



Méthodes de régression spatiale : un grand bol d'R

Philippe Apparicio

Jean Dubé

Jérémy Gelb

Joan Carles Martori

2025-01-26

Table des matières

Préface	1
Un manuel sous la forme d'une ressource éducative libre	2
Comment lire ce manuel?	4
Comment utiliser les données du livre pour reproduire les exemples?	4
Liste des <i>packages</i> utilisés	5
Bref historique sur les modèles de régressions spatiales en économie et en géographie	5
Structure du livre	6
Remerciements	7
À propos des auteurs	8
Partie 1. Notions de base	9
1 Notions de base	10
1.1 Description des jeux de données utilisés dans le manuel	10
1.1.1 Jeu de données sur l'agglomération de Lyon	11
1.1.2 Jeu de données sur la région métropolitaine de Montréal	13
1.1.3 Jeu de données sur Barcelone	13
1.1.4 Jeu de données sur la région métropolitaine de Montréal	13
1.1.5 Jeu de données sur la densité pour six villes espagnoles	13
1.1.6 Jeu de données sur la densité pour six régions métropolitaines espagnoles	13
1.2 Matrices des pondérations spatiales	13
1.2.1 Matrices de contiguïté	13
1.2.2 Matrices de proximité	15
1.2.2.1 Matrice de distance binaire (de connectivité)	18
1.2.2.2 Matrices basées sur la distance	18
1.2.2.3 Matrices selon le critère des plus proches voisins	19
1.2.3 Standardisation des matrices de pondération spatiale en ligne	20
1.2.4 Mise en œuvre dans R	22
1.2.4.1 Matrices de pondération spatiale selon la contiguïté	22
1.2.4.2 Matrices de pondération spatiale selon la contiguïté et un ordre d'adjacence	25
1.2.4.3 Matrice de connectivité (matrice distance binaire)	25
1.2.4.4 Matrices de pondération spatiale selon l'inverse de la distance et l'inverse de la distance au carré	25
1.2.4.5 Matrices de pondération spatiale selon le critère des plus proches voisins	29
1.3 Variable spatialement décalée	31
1.4 Autocorrélation spatiale	33
1.4.1 Formulation du I de Moran	34

1.4.2	Interprétation du I de Moran	35
1.4.3	Significativité du I de Moran	36
1.4.4	Mise en œuvre dans R	36
1.4.4.1	Étape 1. Construction des matrices de pondération spatiale	36
1.4.4.2	Étape 2. Calcul du I de Moran et des trois tests de significativité	37
1.4.4.3	Étape 3. Identification de la plus forte autocorrélation spatiale selon les différentes matrices	39
1.4.4.4	Étape 4. Comparaison des valeurs du I de Moran pour plusieurs variables avec la même matrice	42
1.5	Retour sur la régression linéaire multiple	44
1.6	Pourquoi recourir à des régressions spatiales?	47
1.6.1	Dépendance spatiale	47
1.6.2	Hétérogénéité spatiale	51
Partie 2. Économétrie spatiale		52
2	Modèles d'économétrie spatiale	53
2.1	Les différents modèles spatiaux autorégressifs	54
2.1.1	Modèle SLX : prise en compte des caractéristiques des voisins	54
2.1.1.1	Description du Modèle SLX	54
2.1.1.2	Mise en œuvre dans R	56
2.1.2	Modèle SAR : autocorrélation spatiale sur la variable dépendante	61
2.1.3	Modèle SEM : autocorrélation spatiale sur le terme d'erreur	66
2.1.4	Modèle SDM : autocorrélation spatiale sur la variable dépendante et les variables indépendantes	67
2.1.5	Modèle SDEM : autocorrélation spatiale sur les variables indépendantes et sur le terme d'erreur	70
2.2	Quel modèle choisir?	73
2.2.1	Tests du multiplicateur de Lagrange sur le modèle MCO	73
2.2.2	Comparaison des modèles mixtes et non mixtes	74
2.2.3	Mesures AIC et BIC et dépendance spatiale	75
2.3	Quiz de révision	78
2.4	Exercices de révision	78
3	Modèles d'économétrie spatiale par panel	80
3.1	Bref retour sur les modèles en panel	81
3.2	Formulation des différents modèles spatiaux par panel	81
3.2.1	Description des différents modèles	81
3.2.2	Modèle SLPDM : autocorrélation sur la variable dépendante	81
3.2.3	Modèle SEPDM : autocorrélation sur le terme d'erreur	81
3.2.4	Modèle SEPDM : autocorrélation sur la variable dépendante et les variables indépendantes	81
3.3	Sélection du modèle spatial par panel le plus approprié	81
3.4	Mise en œuvre dans R	81
3.5	Quiz de révision	81
3.6	Exercices de révision	81

Partie 3. Régressions géographiquement pondérées	83
4 Régressions géographiquement pondérées classiques	84
4.1 Principe de base	85
4.1.1 Pourquoi recourir à une régression géographiquement pondérée?	85
4.1.2 Formulation de la GWR	85
4.1.3 Exemple applicatif de la GWR	87
4.1.3.1 Résultats du modèle global	87
4.1.3.2 Résultats du modèle GWR	87
4.2 Mise en œuvre et analyse dans R	88
4.2.1 GWR avec le <i>package</i> <code>spgwr</code>	90
4.2.1.1 Définition de la taille de la zone d'influence	90
4.2.1.2 Réalisation de la GWR	92
4.2.1.3 Comparaison des modèles MCO et GWR	93
4.2.1.4 Cartographie des résultats du modèle GWR	97
4.2.2 GWR avec le <i>package</i> <code>GWmodel</code>	107
4.2.2.1 Définition de la taille de la zone d'influence	107
4.2.2.2 Réalisation de la GWR	110
4.2.2.3 Cartographie des résultats du modèle GWR	112
4.3 GWR classiques pour d'autres distributions	115
4.4 Limites et critiques des GWR	115
4.5 Quiz de révision	118
4.6 Exercices de révision	118
5 Extensions de la régression géographiquement pondérée	120
5.1 Régression géographiquement pondérée mixte	122
5.1.1 Principe de base de la GWR mixte	122
5.1.1.1 Pourquoi recourir à une GWR mixte	122
5.1.1.2 Formulation de la GWR mixte	122
5.1.2 Mise en oeuvre de la GWR mixte dans R	122
5.2 Régression géographiquement pondérée multiéchelle	122
5.2.1 Principe de base de la MGWR	122
5.2.1.1 Pourquoi recourir à une GWR mixte	122
5.2.1.2 Formulation de la GWR mixte	122
5.2.2 Mise en oeuvre de la MGWR dans R	122
5.3 Régression géographiquement pondérée mixte avec des variables spatialement décalée	122
5.3.1 Principe de base de la MGWR-SAR	122
5.3.1.1 Pourquoi recourir à une MGWR-SAR	122
5.3.1.2 Formulation de la MGWR-SAR	122
5.3.2 Mise en oeuvre de la MGWR-SAR dans R	122
5.4 Quiz de révision	122
5.5 Exercices de révision	122

Partie 4. Autres approches de modélisations spatiales	124
6 Modèles généralisées additifs	125
6.1 Modèles généralisés additifs (GAM) avec une <i>spline</i> bivariée sur les coordonnées géographiques	125
6.1.1 Principe de base d'un GAM intégrant l'espace	126
6.1.1.1 Formulation d'un modèle GAM vec une spline bivariée sur les coordonnées géographiques	126
6.1.1.2 Pourquoi recourir à un modèle GAM avec une spline bivariée sur les coordonnées géographiques	126
6.1.2 Mise en oeuvre et analyse dans R	126
6.1.2.1 Réalisation du modèle GAM	126
6.1.2.2 Visualisation de l'effet de l'espace	129
6.1.2.3 Dépendance spatiale du modèle GAM	132
6.2 GAM avec un lissage par champ aléatoire de Markov	133
6.2.1 Principe de base d'un avec un champ aléatoire de Markov	133
6.2.2 Mise en oeuvre et analyse dans R	133
6.3 Quiz de révision	135
6.4 Exercices de révision	135
7 Modèles avec des vecteurs propres spatiaux	136
 Partie 7. Exercices et bibliographie	 137
8 Correction des exercices	138
8.1 Exercices du chapitre 1	138
8.1.1 Exercice 1	138
8.1.2 Exercice 2	138
8.1.3 Exercice 3	138
8.2 Exercices du chapitre 2	138
8.2.1 Exercice 1	138
8.2.2 Exercice 2	138
8.2.3 Exercice 3	138
8.3 Exercices du chapitre 3	138
8.3.1 Exercice 1	138
8.3.2 Exercice 2	138
8.3.3 Exercice 3	138
8.4 Exercices du chapitre 4	138
8.4.1 Exercice 1	138
8.4.2 Exercice 2	139
8.4.3 Exercice 3	140
8.5 Exercices du chapitre 5	142
8.5.1 Exercice 1	142
8.5.2 Exercice 2	142
8.5.3 Exercice 3	142
 Bibliographie	 143

Liste des figures

1	Licence Creative Commons du livre	3
2	Téléchargement de l'intégralité du livre	5
1.1	Cartographie des variables du jeu de données LyonIris	12
1.2	Relation topologique entre des entités spatiales polygonales	14
1.3	Relations de voisinage et évaluation de la contiguïté	16
1.4	Les différents types de distance	17
1.5	Illustration de la connectivité basée sur la distance	18
1.6	Comparaison des matrices inverse de la distance et inverse de la distance au carré	19
1.7	Arrondissements de la ville de Sherbrooke	20
1.8	Illustration du calcul d'une variable spatialement décalée	31
1.9	Exemple de variable spatialement décalée	33
1.10	Valeurs du I de Moran selon les différentes matrices de pondération spatiale	42
1.11	Valeurs du I de Moran pour les quatre variables	44
1.12	Cartographie des résidus d'un modèle MCO	50
2.1	Cartographie des résidus du modèle SLX	60
2.2	Cartographie des résidus du modèle SAR	65
2.3	Comparaison des différents modèles	77
4.1	Fonctions kernel pour définir la matrice de pondération $W(i)$	86
4.2	Cartographie des valeurs de t : moins de 15 ans (%)	89
4.3	Variables indépendantes significatives du modèle GWR	90
4.4	Cartographie des résidus des modèles MCO et GWR	97
4.5	Cartographie des R carrés locaux de la GWR	100
4.6	Cartographie des coefficients de régression de la GWR	102
4.7	Cartographie des valeurs de t de la GWR	104
4.8	Nombre de variables significatives aux seuils de 5% et 1%	105
4.9	Variable indépendante la plus significative au seuil de 5 %	106
4.10	Exemple de cartographie avec les résultats de la GWR obtenus avec le <i>package GWmodel</i>	114
4.11	Corrélation de Pearson entre les coefficients de régression locaux de la GWR	117
6.1	Visualisation des prédictions dans l'espace avec la fonction <code>vis.gam</code>	130
6.2	Visualisation de l'effet de la localisation centrée sur zéro	132

Listes des tableaux

1.1	Statistiques descriptives du jeu de données LyonIris	11
1.2	Matrices de pondération spatiale selon la géométrie	13
1.3	Standardisation de matrices de pondération spatiale	21
1.4	Résultats du I de Moran selon les différentes matrices	41
4.1	Résultats du modèle global (régression linéaire multiple)	87
4.2	Analyse de variance entre les modèles global et GWR	88

Préface

Résumé : Ce livre vise à décrire une panoplie de méthodes de régression spatiale avec le logiciel ouvert R. La philosophie de ce livre est de donner toutes les clefs de compréhension et de mise en œuvre des méthodes abordées dans R. La présentation des méthodes est basée sur une approche compréhensive et intuitive plutôt que mathématique, sans pour autant négliger la rigueur statistique.

Remerciements : Ce manuel a été réalisé avec le soutien de la fabriqueREL. Fondée en 2019, la fabriqueREL est portée par divers établissements d'enseignement supérieur du Québec et agit en collaboration avec les services de soutien pédagogique et les bibliothèques. Son but est de faire des ressources éducatives libres (REL) le matériel privilégié en enseignement supérieur au Québec.

Maquette de la page couverture et identité graphique du livre : Andrés Henao Florez.

Mise en page : Philippe Apparicio et Marie-Hélène Gadbois Del Carpio.

Révision linguistique : Denise Latreille.

© Philippe Apparicio, Jean Dubé, Jérémy Gelb et Joan Carles Martori.

Pour citer cet ouvrage : Apparicio P., J. Dubé, J. Gelb et J. C. Martori (2024). *Méthodes de régression spatiale : un grand bol d'R*. Université Laval et Université de Sherbrooke. fabriqueREL. Licence CC BY-SA.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution - Partage dans les mêmes conditions 4.0 International](#).

Vous êtes autorisé-e à :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Selon les conditions suivantes :

Paternité – Vous devez citer le nom des auteurs originaux.

Mêmes conditions – Si vous remixez, transformez, ou créez à partir du matériel composant l'Œuvre originale, vous devez diffuser l'Œuvre modifiée avec la même licence.



Université de
Sherbrooke



fabriqueREL
RESSOURCES ÉDUCATIVES LIBRES

Un manuel sous la forme d'une ressource éducative libre

Pourquoi un manuel sous licence libre?

Les logiciels libres sont aujourd'hui très répandus. Comparativement aux logiciels propriétaires, l'accès au code source permet à quiconque de l'utiliser, de le modifier, de le dupliquer et de le partager. Le logiciel R, dans lequel sont mises en œuvre les méthodes de régression spatiale décrites dans ce livre, est d'ailleurs à la fois un langage de programmation et un logiciel libre (sous la licence publique générale [GNU GPL2](#)). Par analogie aux logiciels libres, il existe aussi des **ressources éducatives libres (REL)** « dont la licence accorde les permissions désignées par les 5R (**R**etenir – **R**éutiliser – **R**éviser – **R**emixer – **R**edistribuer) et donc permet nécessairement la modification » ([fabriqueREL](#)). La licence de ce livre, CC BY-SA (figure 1), permet donc de :

- **Retenir**, c'est-à-dire télécharger et imprimer gratuitement le livre. Notez qu'il aurait été plutôt surprenant d'écrire un livre payant sur un logiciel libre et donc gratuit. Aussi, nous aurions été très embarrassés que des personnes étudiantes avec des ressources financières limitées doivent payer pour avoir accès au livre, sans pour autant savoir préalablement si le contenu est réellement adapté à leurs besoins.
- **Réutiliser**, c'est-à-dire utiliser la totalité ou une section du livre sans limitation et sans compensation financière. Cela permet ainsi à d'autres personnes enseignantes de l'utiliser dans le cadre d'activités pédagogiques.
- **Réviser**, c'est-à-dire modifier, adapter et traduire le contenu en fonction d'un besoin pédagogique précis puisqu'aucun manuel n'est parfait, tant s'en faut! Le livre a d'ailleurs été écrit intégralement dans R avec [Quarto](#). Quiconque peut ainsi télécharger gratuitement le code source du livre sur [github](#) et le modifier à sa guise (voir l'encadré intitulé *Suggestions d'adaptation du manuel*).
- **Remixer**, c'est-à-dire « combiner la ressource avec d'autres ressources dont la licence le permet aussi pour créer une nouvelle ressource intégrée » ([fabriqueREL](#)).
- **Redistribuer**, c'est-à-dire distribuer, en totalité ou en partie le manuel ou une version révisée sur d'autres canaux que le site Web du livre (par exemple, sur le site Moodle de votre université ou en faire une version imprimée).

La licence de ce livre, CC BY-SA (figure 1), oblige donc à :

- Attribuer la paternité de l'auteur dans vos versions dérivées, ainsi qu'une mention concernant les grandes modifications apportées, en utilisant la formulation suivante : Apparicio Philippe et Jérémy Gelb (2024). *Méthodes d'analyses spatiales : un grand bol d'R*. Université de Sherbrooke. [fabriqueREL](#). Licence CC BY-SA.
- Utiliser la même licence ou une licence similaire à toutes versions dérivées.

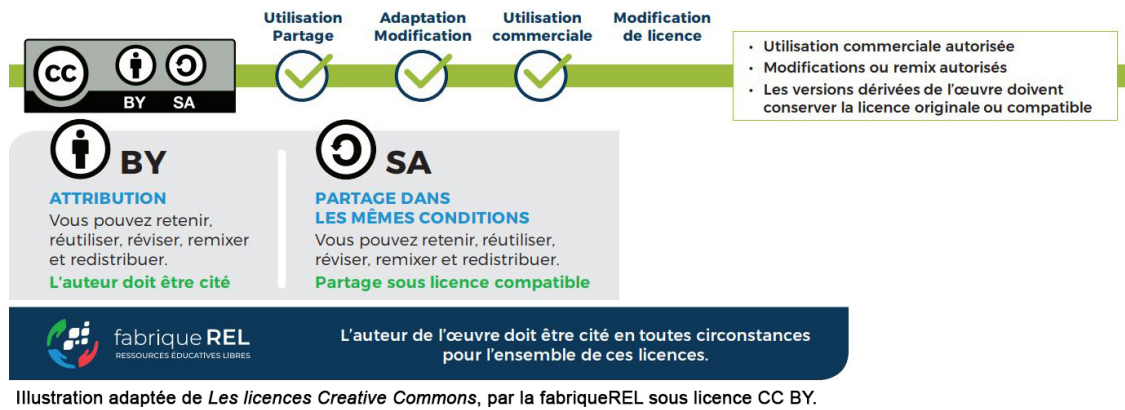


Figure 1: Licence Creative Commons du livre

💡 Astuce

Suggestions d'adaptation du manuel

Pour chaque méthode d'analyse spatiale abordée dans le livre, une description détaillée et une mise en œuvre dans R sont disponibles. Par conséquent, plusieurs adaptations du manuel sont possibles :

- Conserver uniquement les chapitres sur les méthodes ciblées dans votre cours.
- En faire une version imprimée et la distribuer aux personnes étudiantes.
- Modifier la description d'une ou de plusieurs méthodes en effectuant les mises à jour directement dans les chapitres.
- Insérer ses propres jeux de données dans les sections intitulées *Mise en œuvre dans R*.
- Modifier les tableaux et figures.
- Ajouter une série d'exercices.
- Modifier les quiz de révision.
- Rédiger un nouveau chapitre.
- Modifier des syntaxes R. Plusieurs *packages* R peuvent être utilisés pour mettre en œuvre telle ou telle méthode. Ces derniers évoluent aussi très vite et de nouveaux *packages* sont proposés fréquemment! Par conséquent, il peut être judicieux de modifier une syntaxe R du livre en fonction de ses habitudes de programmation dans R (utilisation d'autres *packages* que ceux utilisés dans le manuel par exemple) ou de bien mettre à jour une syntaxe à la suite de la parution d'un nouveau *package* plus performant ou intéressant.
- Toute autre adaptation qui permet de répondre au mieux à un besoin pédagogique.

Comment lire ce manuel?

Le livre comprend plusieurs types de blocs de texte qui en facilitent la lecture.

Package

Bloc *packages*

Habituellement localisé au début d'un chapitre, il comprend la liste des *packages* R utilisés pour un chapitre.

Objectif

Bloc objectif

Il comprend une description des objectifs d'un chapitre ou d'une section.

Note

Bloc notes

Il comprend une information secondaire sur une notion, une idée abordée dans une section.

Aller plus loin

Bloc pour aller plus loin

Il comprend des références ou des extensions d'une méthode abordée dans une section.

Astuce

Bloc astuce

Il décrit un élément qui vous facilitera la vie : une propriété statistique, un *package*, une fonction, une syntaxe R.

Attention

Bloc attention

Il comprend une notion ou un élément important à bien maîtriser.

Exercice

Bloc exercice

Il comprend un court exercice de révision à la fin de chaque chapitre.

Comment utiliser les données du livre pour reproduire les exemples?

Ce livre comprend des exemples détaillés et appliqués dans R pour chacune des méthodes abordées. Ces exemples se basent sur des jeux de données structurés et mis à disposition avec le livre. Ils sont disponibles sur le *repo github* dans le sous-dossier *data*, à l'adresse <https://github.com/SerieBoldR/RegressionsSpatiales/tree/main/data>.

Une autre option est de télécharger le *repo* complet du livre directement sur *github* (<https://github.com/SerieBoldR/RegressionsSpatiales>) en cliquant sur le bouton Code, puis le bouton Download ZIP (figure 2). Les données se trouvent

alors dans le sous-dossier nommé **data**.

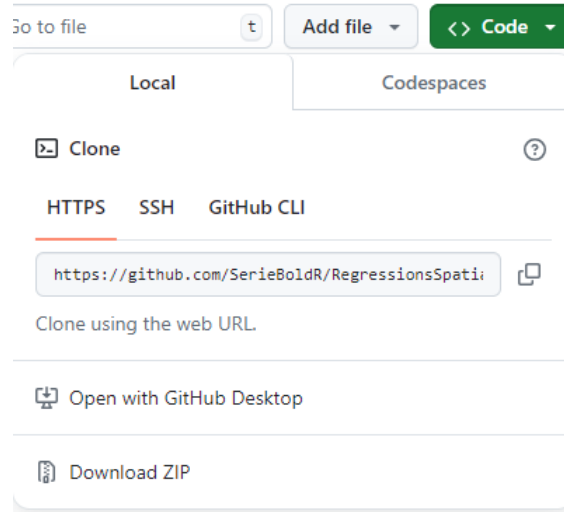


Figure 2: Téléchargement de l'intégralité du livre

Liste des packages utilisés

Dans ce livre, nous utilisons de nombreux *packages* R que vous pouvez installer avec le code ci-dessous.

```
# Liste des packages
ListePackages <- c("dplyr", "mgwrsar", "plm", "RColorBrewer", "rgeoda", "sf", "spatialreg",
                  "spfilter", "splm", "spdep", "spgwr")
# Packages non installés dans la liste
PackagesNonInstalles <- ListePackages[!(ListePackages %in% installed.packages()[,"Package"])]
# Installation des packages manquants
if(length(new.packages)) install.packages(PackagesNonInstalles)
```

Bref historique sur les modèles de régressions spatiales en économie et en géographie

Depuis une cinquantaine d'années, les économistes et les géographes conçoivent et utilisent largement des méthodes de régression prenant en compte la dimension spatiale : modèles d'économétrie spatiale, régressions géographiquement pondérées, modèles généralisés additifs intégrant une dimension spatiale, modèles avec des vecteurs propres spatiaux, etc.

La notion d'autocorrélation spatiale, étroitement liée aux régressions spatiales, remonte à une période encore plus ancienne. Elle a été introduite notamment par les travaux de Patrick Moran, d'abord en 1948 puis en 1950, suivis par ceux de Robert Geary en 1954 et de Nathan Mantel en 1967 (Moran 1948; Moran 1950; Geary 1954; Mantel 1967). La formalisation des relations spatiales trouve son origine, plus précisément, dans la première loi de la géographie énoncée par Waldo

Tobler en 1970 qui stipule que « tout interagit avec tout, mais les objets proches ont plus de chance de le faire que les objets éloignés [*Everything is related to everything else, but near things are more related than distant things*] » (1970).

En économie, le terme « économétrie spatiale » a été utilisé pour la première fois par Jean Paelinck (1978) à la fin des années 1970. Cependant, il est resté relativement peu exploité jusqu'aux travaux de Luc Anselin, qui publie en 1988 un ouvrage de référence sur le sujet intitulé *Spatial econometrics: methods and models* (Anselin 1988). Il faut néanmoins attendre le développement des outils informatiques avant de voir les applications affluées. Ainsi, au début des années 1990, l'économétrie spatiale a véritablement pris son essor (voir Anselin (2010) pour un historique détaillé). Depuis, ce domaine a gagné en importance, s'établissant presque comme un incontournable pour la prise en compte des effets spatiaux, mais aussi, et surtout, des fameux efforts de débordements (nous y reviendrons).

En géographie, Daniel Griffith est certainement l'un des géographes les plus actifs dans le domaine des régressions spatiales, y allant de plusieurs propositions dans le but de contrôler pour la présence d'autocorrélation spatiale résiduelle. Ses premiers travaux remontent aux années 1980, et il a publié depuis un grand nombre d'ouvrages (notamment, Griffith 1988; Griffith 2003; Griffith, Chun et Li 2019). Puis, dans les années 1990, trois géographes – Stewart Fotheringham, Chris Brunsdon et Martin Charlton – ont introduit la régression géographiquement pondérée, une méthode permettant d'analyser localement les relations entre la variable dépendante et les variables indépendantes d'un modèle de régression (Stewart Fotheringham, Charlton et Brunsdon 1996; Brunsdon, Fotheringham et Charlton 1996; Brunsdon, Fotheringham et Charlton 1998). En 2003, la parution de leur ouvrage intitulé *Geographically weighted regression: the analysis of spatially varying relationships* (Fotheringham, Brunsdon et Charlton 2003) a largement contribué à la popularisation de cette méthode.

Structure du livre

Depuis une cinquantaine d'années, les économètres, les épidémiologistes et les géographes conçoivent et utilisent largement des méthodes de régression prenant en compte la dimension spatiale, dont les principales sont présentées dans ce manuel. Ce dernier est structuré en cinq grandes sections.

Partie 1. Notions de base. Dans cette première partie, nous présentons les jeux de données utilisées pour mettre en œuvre les différentes méthodes de régression spatiale présentées dans le livre (chapitre 1). Nous discutons aussi de plusieurs notions fondamentales et méthodes qu'il importe de bien maîtriser avant d'aborder les chapitres suivants consacrés aux méthodes de régression spatiale, notamment : l'autocorrélation spatiale (globale et locale), la notion de variable spatialement décalée et la régression linéaire multiple. Nous concluons ce chapitre en exposant les raisons qui justifient l'utilisation de différentes formes de régressions spatiales pour modéliser des données spatiales ou spatiotemporelles.

Partie 2. Modèles d'économétrie spatiale. Cette seconde partie est consacrée aux modèles d'économétrie spatiale qui vise à modéliser la **dépendance spatiale**. Dans le chapitre 2, nous décrivons ainsi les principaux modèles spatiaux autorégressifs qui permettent d'introduire l'autocorrélation spatiale sur les variables indépendantes (modèle SLX), la variable dépendante (SAR), sur le terme d'erreur (SEM), à la fois sur la variable dépendante et les variables indépendantes (SDM) ou à la fois sur les variables indépendantes et sur le terme d'erreur (SDEM). Dans le chapitre 3, nous abordons une extension des modèles autorégressifs, soit les modèles spatiaux par panel qui permettent de modéliser des données spatiales longitudinales.

Partie 3. Régressions géographiquement pondérées. Dans cette partie, nous abordons les régressions géographiquement pondérées (GWR) qui permettent de modéliser l'**hétérogénéité spatiale**, soit l'instabilité des relations entre la variable dépendante et celles indépendantes. Premièrement, nous décrivons les formes dites classiques de la GWR qui s'appliquent à des variables dépendantes continues, dichotomiques (logistique) et de comptage (Poisson) (chapitre 4).

Deuxièmement, nous abordons des extensions des la GWR, particulièrement la GWR mixte, la GWR multiéchelle et les GWR mixtes avec des variables spatialement décalées (MGWR-SAR) (chapitre 5).

Partie 4. Autres modèles de régressions spatiales. Cette quatrième et dernière partie comprend deux chapitres. Le chapitre 6 est consacré aux modèles généralisés additifs qui permettent d'introduire l'espace deux manières différentes : les modèles GAM avec une *spline* bivariée avec les coordonnées géographiques (x, y) pour capturer les variations continues dans l'espace; les modèles GAM avec un lissage par champ aléatoire de Markov (*random markov field* – RMF) pour modéliser la dépendance spatiale entre les unités spatiales voisines. Dans le chapitre 7, nous nous abordons les modèles linéaires généralisés avec des vecteurs spatiaux proposés par Daniel Griffith (2003; Griffith, Chun et Li 2019).

Remerciements

De nombreuses personnes ont contribué à l'élaboration de ce manuel.

Ce projet a bénéficié du soutien pédagogique et financier de la *fabriqueREL* (ressources éducatives libres). Les différentes rencontres avec le comité de suivi nous ont permis de comprendre l'univers des ressources éducatives libres (REL) et notamment leurs *fameux 5R* (Retenir — Réutiliser — Réviser — Remixer — Redistribuer), de mieux définir le besoin pédagogique visé par ce manuel, d'identifier des ressources pédagogiques et des outils pertinents pour son élaboration. Ainsi, nous remercions chaleureusement les membres de la *fabriqueREL* pour leur soutien inconditionnel :

- Marianne Demers-Desmarais, bibliothécaire, Université Laval.
- Julie-Christine Gagné, conseillère en pédagogie universitaire, Université Laval.
- Claude Potvin, conseiller pédagogique à la fabriqueREL, Université Laval.
- Nadia Villeuneuve, bibliothécaire, Université Laval.

À MODIFIER PLUS TARD. Nous remercions aussi les membres du comité de révision pour leurs commentaires et suggestions très constructifs. Ce comité est composé de quatre personnes étudiantes du *Département de géomatique appliquée* de l'*Université de Sherbrooke* :

- Gaël Machemin et Loek Pascaud, étudiants à la *maîtrise en géomatique appliquée et télédétection (type recherche)*.
- Rosemarie Légaré, étudiante à la maîtrise *en géomatique appliquée et télédétection*.
- Marie-Hélène Gadbois Del Carpio, étudiante à la maîtrise *en géomatique appliquée et télédétection* a participé activement à la mise en page du manuel; elle est désormais une grande spécialiste des feuilles de style en cascade (CSS) et de Quarto.

Finalement, nous remercions Denise Latreille, réviseuse linguistique et chargée de cours à l'Université Sherbrooke, pour la révision du manuel.

À propos des auteurs

Philippe Apparicio est professeur titulaire au Département de géomatique appliquée de l'Université de Sherbrooke. Il y enseigne aux programmes de 1^{er} et 2^e cycles de géomatique les cours *Transport et mobilité durable*, *Modélisation et analyse spatiale* et *Géomatique appliquée à la gestion urbaine*. Géographe de formation, ses intérêts de recherche incluent la justice et l'équité environnementale, la mobilité durable, les pollutions atmosphérique et sonore, et le vélo en ville. Il a publié une centaine d'articles scientifiques dans différents domaines des études urbaines et de la géographie mobilisant la géomatique et l'analyse spatiale.

Jean Dubé est professeur titulaire à l'École Supérieure d'aménagement du territoire et de développement régional (ÉSAD) de l'Université Laval. Économiste de formation, ses intérêts de recherche portent notamment sur l'évaluation d'impact des politiques publiques liées à l'aménagement et au développement. Ses recherches portent sur la mesure des externalités urbaines par le marché immobilier, les décisions de localisation des entreprises et des ménages et les déterminants de la croissance urbaine et régionale. Il est aussi rédacteur de la *Revue canadienne des sciences régionales* (RCSR).

Jérémy Gelb a obtenu un doctorat en études urbaines à l'INRS en 2022 (*L'exposition des cyclistes aux pollutions atmosphérique et sonore en milieu urbain : comparaison empirique de plusieurs villes à travers le monde*), sous la supervision de Philippe Apparicio. Il utilise quotidiennement les systèmes d'information géographique (SIG) et des méthodes d'analyses spatiales. Il est tombé dans la marmite de l'*open source* avec le triptyque QGIS, R et Python au début de sa maîtrise. Il a développé deux *packages* : *geomeans* et *spNetwork*, permettant respectivement d'effectuer des analyses de classification floue non supervisée pondérée spatialement et des estimations de densité par kernel sur réseau. Il travaille actuellement comme conseiller en science des données à l'Autorité régionale de transport métropolitain qui gère la planification du transport collectif dans la région de Montréal. Ces travaux portent sur la qualité des milieux urbains, l'accessibilité spatiale, le transport, l'équité environnementale, les SIG et l'analyse spatiale.

Joan Carles Martori est professeur d'économie appliquée au département d'économie et de commerce de l'Université de Vic - Université centrale de la Catalogne (Barcelone, Espagne). Il enseigne les statistiques appliquées et l'économétrie dans des cours de premier et de deuxième cycle. Il est le chercheur principal du groupe *Data Analysis and Modelling*. Économiste de formation, ses recherches portent sur la ségrégation résidentielle, les inégalités et l'équité environnementale à l'aide des statistiques spatiales et de techniques économétriques. Il a contribué au développement du *package* R *AQuadtree* sur la confidentialité des données spatiales ponctuelles. En tant que statisticien appliqué, il collabore avec d'autres disciplines telles que l'éducation et la santé.

Partie 1. Notions de base

1 Notions de base

Avant d'explorer les différents types de régressions spatiales dans les chapitres suivants, il est primordial de comprendre plusieurs notions clés, comme l'autocorrélation spatiale, la dépendance spatiale et l'hétérogénéité spatiale. Aussi, de nombreuses méthodes de régression spatiale s'appuient sur l'utilisation de matrices de pondération spatiale. Nous proposons également un bref rappel sur la régression linéaire multiple.

Objectif

Objectifs d'apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- Connaître les principales matrices de pondérations spatiales.
- Maîtriser les notions d'autocorrélation spatiale, de dépendance spatiale et d'hétérogénéité spatiale.
- Comprendre la notion de variable spatialement décalée.
- Calculer une mesure d'autocorrélation spatiale globale (I de Moran) dans R.
- Évaluer la dépendance spatiale d'un modèle de régression multiple en calculant le I de Moran sur ses résidus.

Package

Liste des *packages* utilisés dans ce chapitre

- Pour importer et manipuler des fichiers géographiques :
 - `sf` pour importer et manipuler des données vectorielles.
- Pour cartographier des données :
 - `ggplot2` et `ggpubr` pour construire des graphiques.
 - `tmap` pour construire des cartes thématiques.
 - `RColorBrewer` pour construire une palette de couleur.

1.1 Description des jeux de données utilisés dans le manuel

Plusieurs jeux de données sont utilisés dans ce livre et sont déjà structurés et disponibles au format `.Rdata`.

⚠ Attention**Manipulation, structuration et cartographie de données spatiales**

Ce livre n'a pas pour objectif de traiter la phase préparatoire consistant à structurer les données spatiales dans R avant l'application de modèles de régression spatiale. Nous partons du principe que les lectrices et lecteurs maîtrisent déjà l'utilisation des *packages* `sf` et `tmap` pour manipuler, structurer et cartographier des données spatiales. Si ce n'est pas le cas, nous vous encourageons à consulter le chapitre intitulé *Manipulation des données spatiales dans R* (Apparicio et Gelb 2024).

1.1.1 Jeu de données sur l'agglomération de Lyon

Premièrement, nous utiliserons le jeu de données spatiales `LyonIris` du *package* `geocode`. Ce jeu de données spatiales pour l'agglomération lyonnaise (France) comprend dix variables, dont quatre environnementales (EN) et six socioéconomiques (SE), pour les îlots regroupés pour l'information statistique (IRIS) de l'agglomération lyonnaise (tableau 1.1 et figure 1.1).

Tableau 1.1: Statistiques descriptives du jeu de données `LyonIris`

Nom	Intitulé	Type	Moy.	E.-T.	Min.	Max.
Lden	Bruit routier (Lden dB(A))	EN	55,6	4,9	33,9	71,7
NO2	Dioxyde d'azote (ug/m ³)	EN	28,7	7,9	12,0	60,2
PM25	Particules fines (PM _{2,5})	EN	16,8	2,1	11,3	21,9
VegHautPrt	Canopée (%)	EN	18,7	10,1	1,7	53,8
Pct0_14	Moins de 15 ans (%)	SE	18,5	5,7	0,0	54,0
Pct_65	65 ans et plus (%)	SE	16,2	5,9	0,0	45,1
Pct_Img	Immigrants (%)	SE	14,5	9,1	0,0	59,8
TxChom1564	Taux de chômage	SE	14,8	8,1	0,0	98,8
Pct_brevet	Personnes à faible scolarité (%)	SE	23,5	12,6	0,0	100,0
NivVieMed	Médiane du niveau de vie (milliers d'euros)	SE	21,8	4,9	11,3	38,7

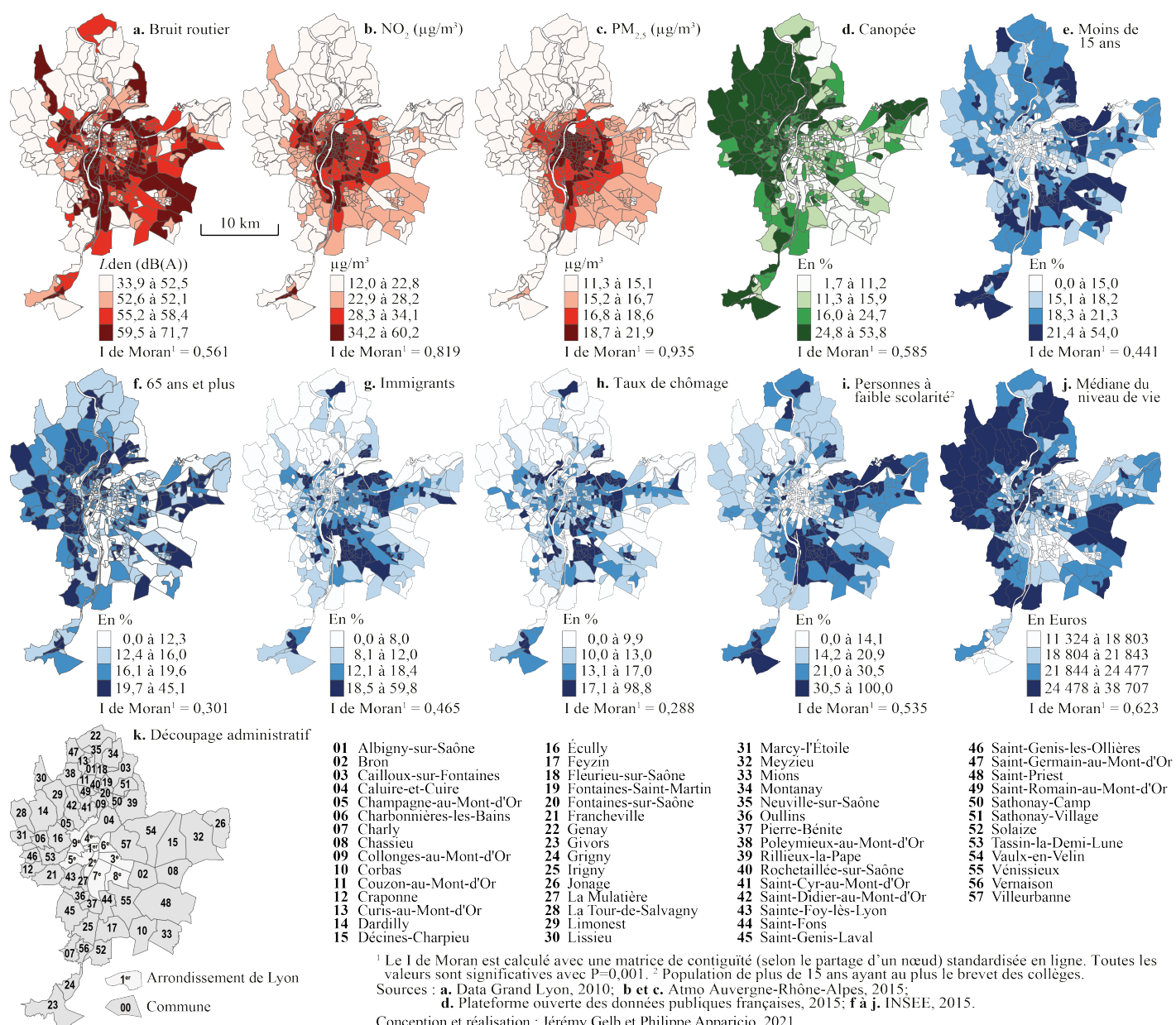


Figure 1.1: Cartographie des variables du jeu de données LyonIris

1.1.2 Jeu de données sur la région métropolitaine de Montréal

1.1.3 Jeu de données sur Barcelone

1.1.4 Jeu de données sur la région métropolitaine de Montréal

1.1.5 Jeu de données sur la densité pour six villes espagnoles

1.1.6 Jeu de données sur la densité pour six régions métropolitaines espagnoles

1.2 Matrices des pondérations spatiales

🎯 Objectif

Utilité des matrices de pondérations spatiales

Ces matrices permettent de définir les relations spatiales entre les entités spatiales d'une couche géographique, et plus spécifiquement la manière de mesurer leur relation d'adjacence (voisinage) ou de proximité (distance). Nous verrons qu'elles sont largement utilisées dans les régressions spatiales, notamment dans les modèles d'économétrie spatiale abordées dans la seconde partie du livre. Elles sont aussi nécessaires au calcul des mesures d'autocorrélation globales ou locales section 1.4.2 et à la construction des variables spatialement décalées section 1.3.

Il existe huit principales matrices de pondération spatiale regroupées en deux grandes catégories : celles de contiguïté (basées sur l'adjacence) et celles de proximité (basées sur la distance) (tableau 1.2). Lorsque la couche géographique est composée de points, seules les matrices de proximité peuvent être utilisées.

Tableau 1.2: Matrices de pondération spatiale selon la géométrie

Matrice	Points	Lignes	Polyg.	Raster
Matrices de contiguïté (basées sur l'adjacence)				
Partage d'un nœud (Queen)		X	X	X
Partage d'un segment (Rook)		X	X	X
Partage d'un nœud et ordre d'adjacence (Queen)		X	X	X
Partage d'un segment et ordre d'adjacence (Rook)		X	X	X
Matrices de proximité (basées sur la distance)				
Connectivité selon la distance	X	X	X	X
Inverse de la distance	X	X	X	X
Inverse de la distance au carré	X	X	X	X
Nombre de plus proches voisins	X	X	X	X

1.2.1 Matrices de contiguïté

La relation d'adjacence (de contiguïté) vise à déterminer si deux entités spatiales sont ou non voisines selon le partage soit d'un nœud, soit d'un segment (frontière commune). La contiguïté est liée à la notion de topologie qui prend en compte les relations de voisinage entre des entités spatiales, sans tenir compte de leurs tailles et de leurs formes géométriques.

Elle peut être représentée à partir d'une **matrice de contiguïté** (avec une valeur de 1 quand deux entités sont voisines et de 0 pour une situation inverse) ou d'un **graphe** (formé de points représentant les entités spatiales et de lignes reliant les entités voisines) (figure 1.2).

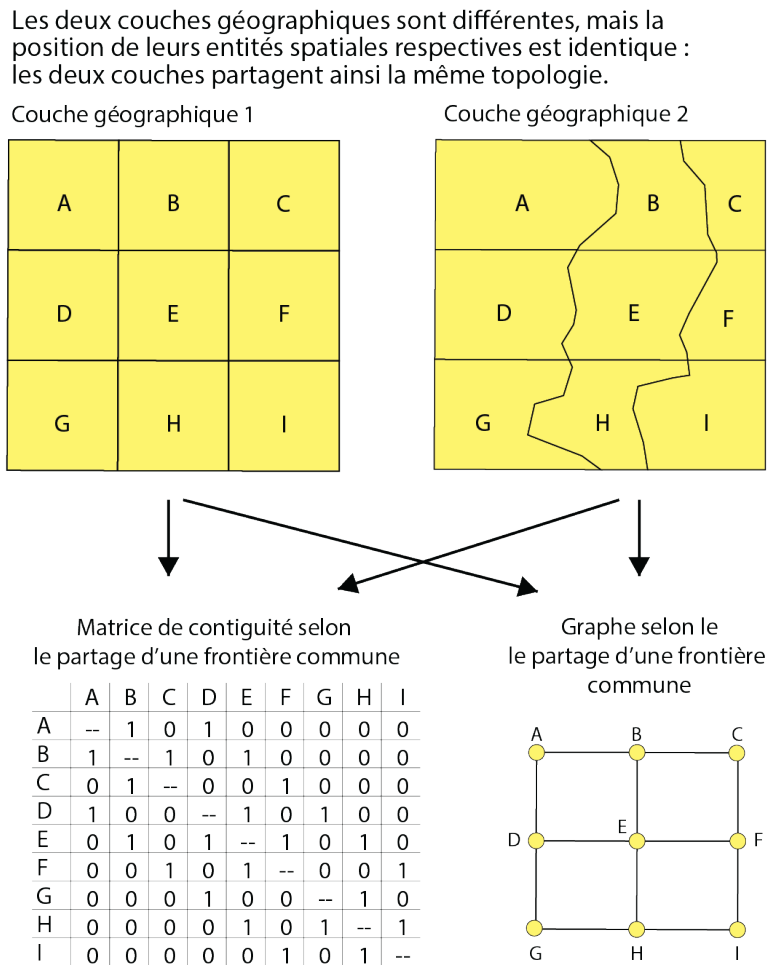


Figure 1.2: Relation topologique entre des entités spatiales polygonales

Trois évaluations de la contiguïté sont représentées à la figure 1.3 :

- **Adjacence selon le partage d'un segment**, soit d'une frontière commune entre les polygones (A).
- **Adjacence selon le partage d'un nœud** (B).
- **Ordre d'adjacence selon le partage d'un segment** (C). L'ordre d'adjacence indique le nombre de frontières à traverser pour se rendre à l'entité spatiale contiguë, soit :
 - **Ordre 1** : une frontière à traverser pour se rendre dans l'entité spatiale adjacente.
 - **Ordre 2** : deux frontières à traverser pour atteindre les entités de la deuxième couronne.
 - **Ordre 3** : trois frontières à traverser pour atteindre les entités de la troisième couronne.
 - Etc.

Bien entendu, les ordres d'adjacence peuvent être également définis selon le partage d'un nœud commun.

Habituellement appelée W , la matrice de contiguïté est binaire selon le partage tant d'un nœud (*Queen* en anglais) (équation 1.1) que d'un segment commun (*Rook* en anglais) (équation 1.2).

$$w_{ij} = \begin{cases} 1 & \text{si les entités spatiales } i \text{ et } j \text{ ont au moins un nœud commun; } i \neq j \\ 0 & \text{sinon} \end{cases} \quad (1.1)$$

$$w_{ij} = \begin{cases} 1 & \text{si les entités spatiales } i \text{ et } j \text{ partagent une frontière commune; } i \neq j \\ 0 & \text{sinon} \end{cases} \quad (1.2)$$

1.2.2 Matrices de proximité

Pour construire une matrice de pondération spatiale selon la proximité, nous pouvons utiliser plusieurs types de distance (Apparicio et al. 2017) : certaines sont cartésiennes, d'autres, dites réticulaires, sont calculées à partir d'un réseau de rues (figure 1.4).

Les distances cartésiennes – euclidienne et de Manhattan (équation 1.3 et équation 1.4) – sont facilement calculables à partir des coordonnées géographiques (x, y) lorsque la couche géographique est dans un système de projection plane (figure 1.4, a). Si la projection de la couche est sphérique (longitude/latitude), il convient d'utiliser la formule de haversine (basée sur la trigonométrie sphérique) pour obtenir la distance à vol d'oiseau (équation 1.5).

Par contre, comme leurs noms l'indiquent, le calcul des distances réticulaires (figure 1.4, b) nécessite d'avoir un réseau de rues dans un système d'information géographique (par exemple, l'extension *Network Analyst* d'ArcGIS Pro) ou dans R (notamment avec le *package* R5R) pour calculer le chemin le plus rapide (voir le chapitre intitulé *Mesures d'accessibilité spatiale selon différents modes de transport* (Apparicio et Gelb 2024)).

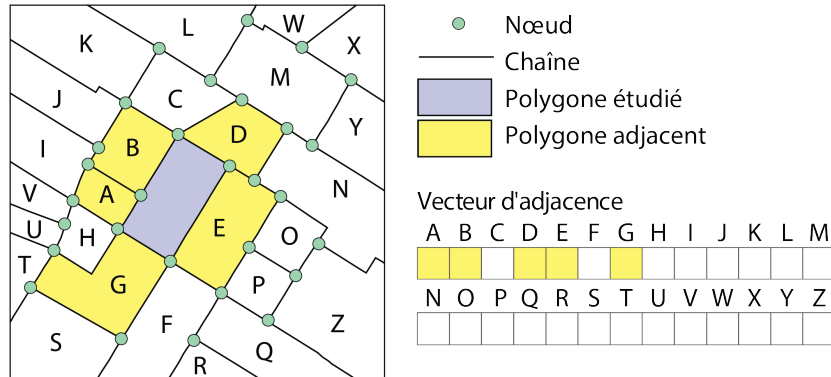
$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (1.3)$$

$$d_{ij} = |x_i - x_j| + |y_i - y_j| \quad (1.4)$$

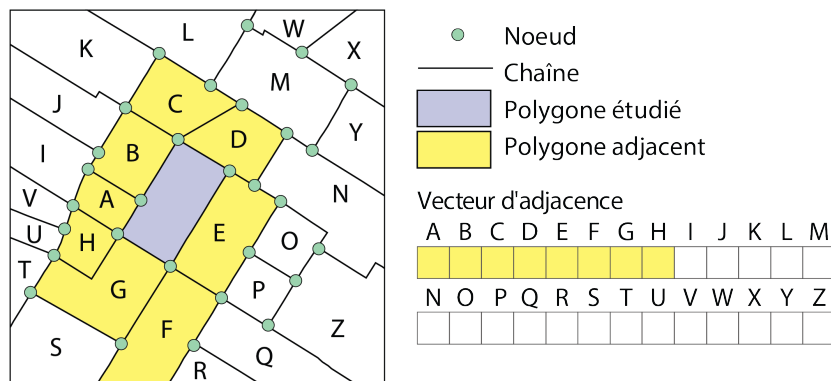
$$d_{ij} = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\delta_i - \delta_j}{2} \right) + \cos \delta_i \cdot \cos \delta_j \cdot \sin^2 \left(\frac{\phi_i - \phi_j}{2} \right)} \right) \quad (1.5)$$

avec R étant le rayon de la terre; δ_i et δ_j les coordonnées de longitude pour les points i et j ; ϕ_i et ϕ_j les coordonnées de latitude pour les points i et j .

A. Adjacence selon le partage d'une frontière commune



B. Adjacence selon le partage d'un nœud



C. Ordres d'adjacence selon le partage d'une frontière commune

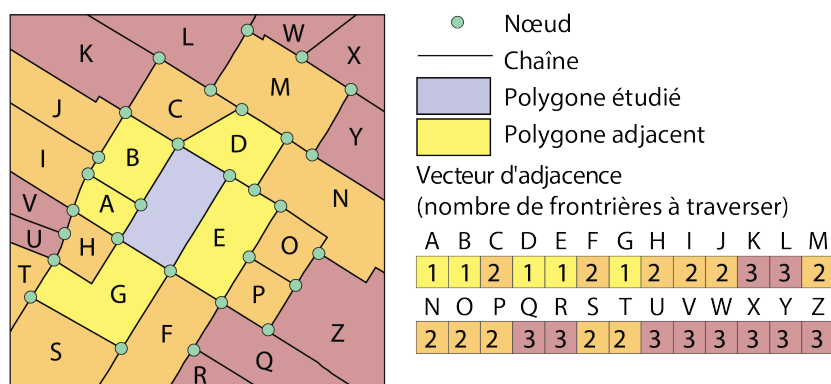
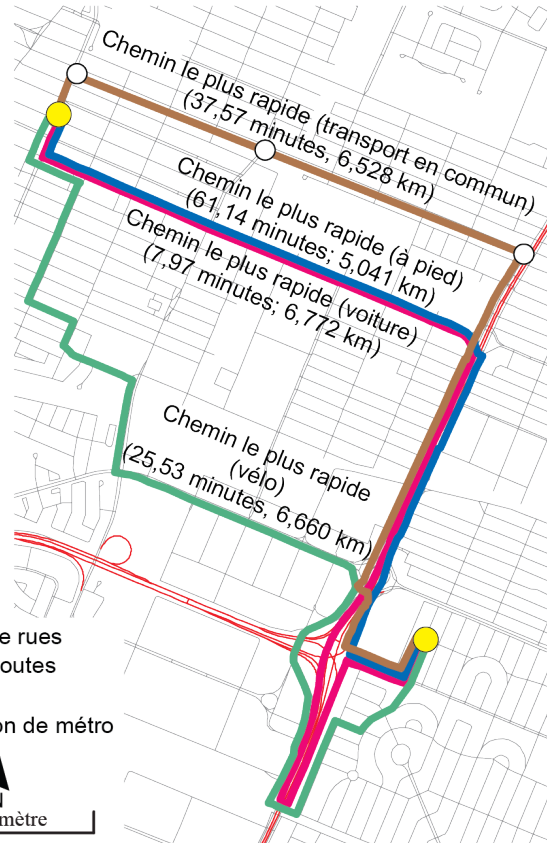


Figure 1.3: Relations de voisinage et évaluation de la contiguïté

(a) Distances cartésiennes



(b) Distances sur un réseau de rues



Figurée adaptée de Apparicio et al. (2017).

Figure 1.4: Les différents types de distance

1.2.2.1 Matrice de distance binaire (de connectivité)

À partir d'une matrice de distance entre les entités spatiales d'une couche géographique, il est possible de créer une matrice de pondération binaire (équation 1.6). Ce type de matrice est habituellement appelée **matrice de connectivité**. Il convient alors de fixer un seuil de distance maximal. Par exemple, avec un seuil de 500 mètres, $w_{ij} = 1$ si la distance entre les entités spatiales i et j est inférieure ou égale à 500 mètres; sinon $w_{ij} = 0$. Notez que pour des lignes et des polygones, la distance est habituellement calculée à partir de leurs centroïdes.

$$w_{ij} = \begin{cases} 1 & \text{si } d_{ij} \leq \bar{d}; i \neq j \\ 0 & \text{sinon} \end{cases} \quad (1.6)$$

avec d_{ij} étant la distance entre les entités spatiales i et j , et $\bar{d}$ étant un seuil de distance maximal fixé par la personne utilisatrice (par exemple, 500 mètres).

En guise d'exemple, à la figure 1.5, seuls les polygones jaunes seraient considérés comme voisins du polygone bleu avec un seuil de distance maximal fixé à 2,5 kilomètres (valeur de 1); les roses se verraient affecter la valeur de 0.

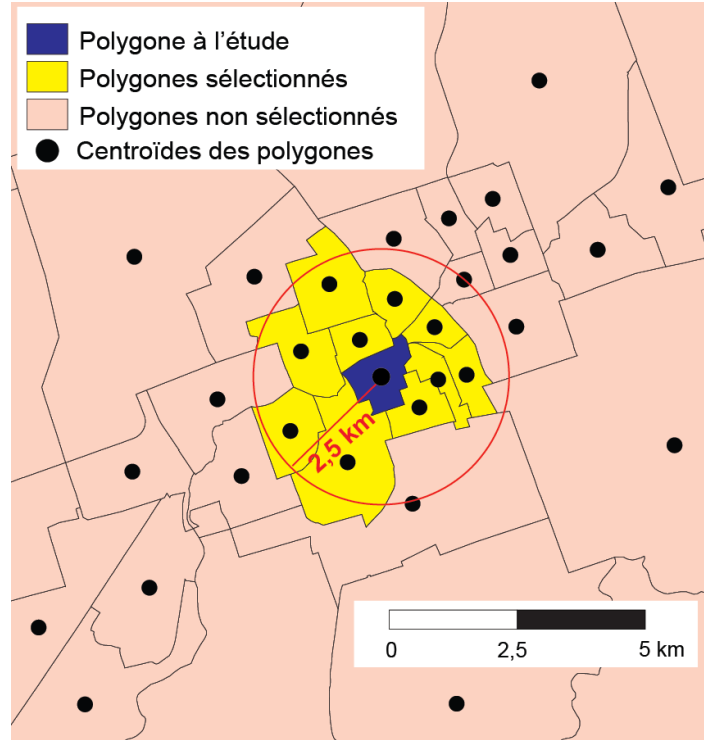


Figure 1.5: Illustration de la connectivité basée sur la distance

1.2.2.2 Matrices basées sur la distance

Une fois les distances calculées entre les entités spatiales, les pondérations peuvent être calculées avec l'inverse de la distance ($1/d_{ij}$) ou l'inverse de la distance au carré ($1/d_{ij}^2$) (équation 1.7).

$$w_{ij} = \begin{cases} \frac{1}{d_{ij}^\gamma} & \text{si } i \neq j \\ 0 & \text{si } i = j \end{cases} \quad (1.7)$$

avec $\gamma = 1$ pour une matrice de l'inverse de la distance et $\gamma = 2$ pour l'inverse de la distance au carré.

Analysons le graphique à la figure 1.6. Premièrement, nous constatons que plus la distance est grande, plus la valeur de la pondération est faible et inversement. De la sorte, nous accordons un rôle plus important aux entités spatiales proches les unes des autres qu'à celles éloignées. Deuxièmement, les pondérations chutent beaucoup plus rapidement avec l'inverse de la distance au carré qu'avec l'inverse de la distance. Autrement dit, le recours à une matrice de pondération calculée avec l'inverse de la distance au carré a comme effet d'accorder un poids plus important aux entités géographiques très proches.

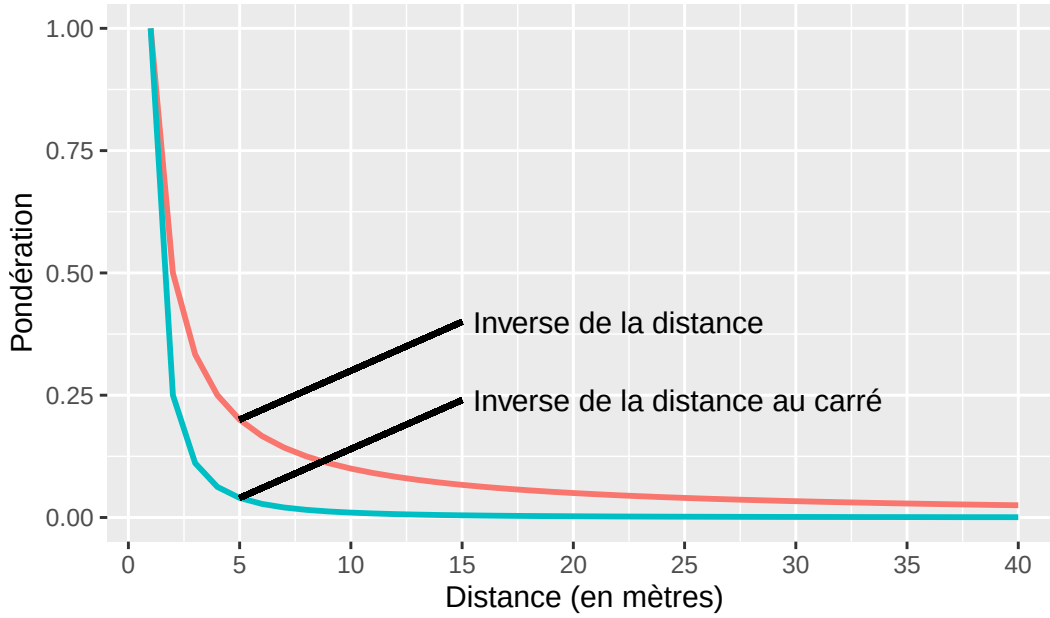


Figure 1.6: Comparaison des matrices inverse de la distance et inverse de la distance au carré

Notez que l'équation 1.7 peut être légèrement modifiée en introduisant un seuil maximal de la distance au-delà duquel les pondérations sont mises à 0 (équation 1.8). Autrement dit, cela permet de ne pas tenir compte des entités spatiales distantes à plus d'un seuil fixé par l'analyste, ce qui est particulièrement intéressant lorsque vous analysez un phénomène dont la diffusion (ou propagation) cesse au-delà d'une certaine distance.

$$w_{ij} = \begin{cases} \frac{1}{d_{ij}^\gamma} & \text{si } d_{ij} \leq \bar{d} \\ 0 & \text{si } d_{ij} > \bar{d} \\ 0 & \text{si } i = j \end{cases} \quad (1.8)$$

1.2.2.3 Matrices selon le critère des plus proches voisins

Une autre façon très utilisée pour définir une matrice de proximité à partir d'une matrice de distance consiste à retenir uniquement les n plus proches voisins. La matrice est aussi binaire avec les valeurs de 1 si les observations sont parmi

les n plus proches de l'entité spatiale i et de 0 pour une situation inverse.

1.2.3 Standardisation des matrices de pondération spatiale en ligne

Il est recommandé de standardiser les matrices de pondération en ligne. La somme de la matrice de pondération sera alors égale au nombre d'entités spatiales de la couche géographique.

⚠ Attention

Quel est l'intérêt de la standardisation?

Nous verrons dans les sections suivantes que ces matrices sont utilisées pour évaluer le degré d'autocorrélation spatiale globale et locale. Or, il est fréquent de comparer les valeurs des mesures d'autocorrélation spatiale obtenues avec différentes matrices d'adjacence et de proximité (contiguïté selon le partage d'un nœud, d'une frontière commune; inverse de la distance, inverse de la distance au carré, etc.). Autrement dit, la standardisation des matrices de pondération spatiale permet de vérifier si le degré de (dis)ressemblance des entités spatiales en fonction d'une variable donnée est plus fort avec une matrice de contiguïté, d'inverse de la distance ou encore d'inverse de la distance au carré, etc.

Pour illustrer comment réaliser une standardisation, nous utilisons une couche géographique comprenant peu d'entités spatiales, soit celle des quatre arrondissements de la ville de Sherbrooke (figure 1.7).

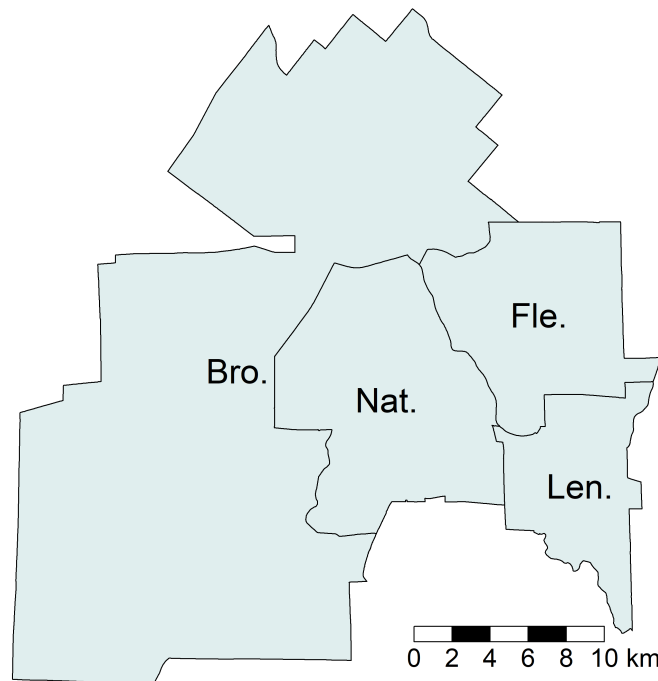


Figure 1.7: Arrondissements de la ville de Sherbrooke

Au tableau 1.3, différentes matrices de contiguïté et de distance ont été calculées, puis standardisées. Voici comment interpréter les différentes sections du tableau :

- **Contiguïté selon le partage d'une frontière commune.** La valeur de 1 signale que deux arrondissements sont voisins, sinon la valeur est à 0. Tel qu'indiqué aux équation 1.1 et équation 1.2, un arrondissement ne peut être voisin de lui-même (ex.: valeur de 0 pour la cellule **Bro.** et **Bro.**). L'arrondissement de Brompton–Rock Forest–Saint-Élie–Deauville (**Bro.**) a deux voisins, soit ceux des Nations et de Fleurimont (**Nat.** et **Fle.**), comme indiqué par la valeur 2 dans la colonne **total**. Par contre, les arrondissements des Nations et de Fleurimont sont voisins de tous les autres (valeur de 3 dans la colonne **total**).
- **Standardisation de la matrice de contiguïté.** Il suffit de diviser chaque valeur de la matrice de contiguïté par la somme de la ligne correspondante. De la sorte, la somme de chaque ligne est égale à 1 et la somme de l'ensemble des valeurs de la matrice est égale au nombre d'entités spatiales (ici 4).
- **Distance (km).** Nous avons calculé la distance euclidienne en kilomètres entre les centroïdes des arrondissements.
- **Inverse de la distance.** Les valeurs sont obtenues avec la formule $1/d_{ij}$. Par exemple, entre **Bro.** et **Nat.**, nous avons $1/7,9930 = 0,1251$.
- **Inverse de la distance au carré.** Les valeurs sont obtenues avec la formule $1/d_{ij}^2$. Par exemple, entre **Bro.** et **Nat.**, nous avons $1/7,9930^2 = 0,0160$.
- **Standardisation de l'inverse de la distance.** Comme précédemment, il suffit de diviser chaque valeur de la matrice par la somme de la ligne correspondante. Par exemple, pour **Bro.** et **Nat.**, nous avons $0,1251/0,3241 = 0,3860$. Remarquez que la somme des lignes est bien égale à 1.
- **Standardisation de l'inverse de la distance au carré.** Comme précédemment, il suffit de diviser chaque valeur de la matrice par la somme de la ligne correspondante. Par exemple, pour **Bro.** et **Nat.**, nous avons $0,0160/0,0360 = 0,4440$. Remarquez que la somme des lignes est bien égale à 1.

Tableau 1.3: Standardisation de matrices de pondération spatiale

Arrondissement	Bro.	Nat.	Len.	Fle.	Somme (lignes)
Matrice de contiguïté selon le partage d'une frontière commune					
Bro.	0,0000	1,0000	0,0000	1,0000	2,0000
Nat.	1,0000	0,0000	1,0000	1,0000	3,0000
Len.	0,0000	1,0000	0,0000	1,0000	2,0000
Fle.	1,0000	1,0000	1,0000	0,0000	3,0000
Standardisation de la matrice de contiguïté					
Bro.	0,0000	0,5000	0,0000	0,5000	1,0000
Nat.	0,3330	0,0000	0,3330	0,3330	1,0000
Len.	0,0000	0,5000	0,0000	0,5000	1,0000
Fle.	0,3330	0,3330	0,3330	0,0000	1,0000
Distance (km)					
Bro.	0,0000	7,9930	18,9940	16,1140	
Nat.	7,9930	0,0000	11,1190	9,1650	
Len.	18,9940	11,1190	0,0000	9,2590	
Fle.	16,1140	9,1650	9,2590	0,0000	
Matrice selon l'inverse de la distance					
Bro.	0,0000	0,1251	0,0526	0,0621	0,2398
Nat.	0,1251	0,0000	0,0899	0,1091	0,3241

Len.	0,0526	0,0899	0,0000	0,1080	0,2505
Fle.	0,0621	0,1091	0,1080	0,0000	0,2792
Matrice selon l'inverse de la distance au carré					
Bro.	0,0000	0,0160	0,0030	0,0040	0,0230
Nat.	0,0160	0,0000	0,0080	0,0120	0,0360
Len.	0,0030	0,0080	0,0000	0,0120	0,0230
Fle.	0,0040	0,0120	0,0120	0,0000	0,0280
Standardisation de l'inverse de la distance					
Bro.	0,0000	0,5220	0,2190	0,2590	1,0000
Nat.	0,3860	0,0000	0,2770	0,3370	1,0000
Len.	0,2100	0,3590	0,0000	0,4310	1,0000
Fle.	0,2220	0,3910	0,3870	0,0000	1,0000
Standardisation de l'inverse de la distance au carré					
Bro.	0,0000	0,6960	0,1300	0,1740	1,0000
Nat.	0,4440	0,0000	0,2220	0,3330	1,0000
Len.	0,1300	0,3480	0,0000	0,5220	1,0000
Fle.	0,1430	0,4290	0,4290	0,0000	1,0000

1.2.4 Mise en œuvre dans R

⚠ Attention

Construction des matrices dans R avec le *packagespdep*.

Le package *spdep* dispose de différentes fonctions pour construire des matrices de contiguïté, de connectivité et de distance :

- `poly2nb` pour des matrices de contiguïté.
- `nblag` et `nblag_cumul` pour des matrices de contiguïté avec des ordres d'adjacence.
- `dnearneigh` pour des matrices de connectivité.
- `as.matrix(dist(coords))` et `mat2listw` pour des matrices de distance.
- `knn2nb` pour des matrices selon le critère des plus proches voisins.

1.2.4.1 Matrices de pondération spatiale selon la contiguïté

Pour créer des matrices de pondération spatiale selon la contiguïté, nous utilisons deux fonctions du *package spdep* :

- `poly2nb(Nom de l'objet sf, queen=TRUE)` crée une matrice de contiguïté sous la forme d'une classe `nb` (*A neighbours list with class nb*). Avec le paramètre `queen=TRUE`, la contiguïté est évaluée selon le partage d'un nœud; avec `queen=FALSE`, la contiguïté est évaluée selon le partage d'un segment (frontière). La matrice spatiale comprend une ligne par polygone avec les index de ceux qui sont adjacents. Par exemple, `Queen[[1]]` renvoie la liste des polygones voisins à la première entité spatiale, soit 24 36 44 73, c'est-à-dire quatre voisins.

- `nb2listw(objet nb, zero.policy=TRUE, style = "W")` crée une matrice de pondération spatiale à partir de n'importe quelle matrice spatiale (de contiguïté ou de distance). Le paramètre `style = "W"`, qui est par défaut, permet de standardiser la matrice en ligne. Par exemple, `W.Queen$weights[[1]]` renvoie les valeurs des pondérations pour la première entité spatiale, soit 0.25 0.25 0.25 0.25 (0,25 = 1 / 4 voisins). Pour obtenir une matrice non standardisée, vous devez écrire `style = "B"`, alors `W.Queen$weights[[1]]` renverra les valeurs de 1 1 1 1.

```
library(sf)      # pour importer des couches géographiques
library(spdep)  # pour construire les matrices de pondération
## Utilisation du jeu de données sur l'agglomération de Lyon
load("data/Lyon.Rdata")
## Matrice selon le partage d'un nœud (Queen)
# Création de la matrice spatiale
Queen <- poly2nb(LyonIris, queen=TRUE)
# Affichage de la première ligne de la matrice
Queen[[1]]
```

```
[1] 27 36 44 73
```

```
# Création de la matrice de pondération avec une standardisation en ligne
W.Queen <- nb2listw(Queen, zero.policy=TRUE, style = "W")
# Affichage de la première ligne des pondérations standardisée
W.Queen$weights[[1]]
```

```
[1] 0.25 0.25 0.25 0.25
```

```
cat("La somme de la première ligne de la matrice de pondération est égale à",
    sum(W.Queen$weights[[1]]))
```

La somme de la première ligne de la matrice de pondération est égale à 1

```
# Affichage de la première ligne des pondérations non standardisée
nb2listw(Queen, zero.policy=TRUE, style = "B")$weights[[1]]
```

```
[1] 1 1 1 1
```

```
## Matrice selon le partage d'un segment (Rook)
Rook <- poly2nb(LyonIris, queen=FALSE)
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")
## Comparaison des deux matrices de contiguïté
# Résultat de la matrice de pondération (Queen)
summary(W.Queen)
```

Characteristics of weights list object:

Neighbour list object:

Number of regions: 506

Number of nonzero links: 2854

Percentage nonzero weights: 1.114687

Average number of links: 5.640316

Link number distribution:

	2	3	4	5	6	7	8	9	10	11	12	14	17
	13	50	84	108	103	70	41	22	7	5	1	1	1

13 least connected regions:

81 91 105 148 160 174 183 325 425 468 480 489 506 with 2 links

1 most connected region:

154 with 17 links

Weights style: W

Weights constants summary:

	n	nn	S0	S1	S2
W	506	256036	506	192.1431	2097.903

Résultat de la matrice de pondération (Rook)

`summary(W.Rook)`

Characteristics of weights list object:

Neighbour list object:

Number of regions: 506

Number of nonzero links: 2660

Percentage nonzero weights: 1.038916

Average number of links: 5.256917

Link number distribution:

	2	3	4	5	6	7	8	9	10	11	12	15
	14	60	104	126	97	58	24	13	6	2	1	1

14 least connected regions:

81 91 105 148 160 174 183 325 376 425 468 480 489 506 with 2 links

1 most connected region:

154 with 15 links

Weights style: W

Weights constants summary:

	n	nn	S0	S1	S2
W	506	256036	506	204.0416	2099.405

1.2.4.2 Matrices de pondération spatiale selon la contiguïté et un ordre d'adjacence

Le code ci-dessous génère les matrices de pondération spatiale standardisées en ligne selon les ordre d'adjacence de 1 à 3.

```
# Création des matrices d'ordre 1, 2 et 3
Queen1 <- poly2nb(LyonIris, queen=TRUE)
Queen2 <- nblag_cumul(nblag(Queen1, 2))
Queen3 <- nblag_cumul(nblag(Queen1, 3))
# Création des matrices de pondération spatiale standardisées en lignes
W.Queen1 <- nb2listw(Queen, zero.policy=TRUE, style = "W")
W.Queen2 <- nb2listw(Queen2, zero.policy=TRUE, style = "W")
W.Queen3 <- nb2listw(Queen3, zero.policy=TRUE, style = "W")
```

1.2.4.3 Matrice de connectivité (matrice distance binaire)

La fonction `dnearneigh(sf points, d1=, d2=)` crée une matrice de connectivité à partir d'une couche de points. Les paramètres `d1` et `d2` permettent de spécifier le rayon de recherche (ex. : avec `d1 = 0` et `d2 = 2500`, le seuil maximal de distance est de 2500 mètres). Si votre couche `sf` comprend des lignes ou des polygones, utilisez la fonction `st_centroid` ou `st_point_on_surface()` pour les convertir en points (section 1.2.2).

```
## Conversion des polygones en points avec st_centroid
IRIS.centroides <- st_centroid(LyonIris)
## Matrice binaire avec un seuil de 2500 mètres
Connect2500m <- dnearneigh(IRIS.centroides, d1 = 0, d2 = 2500)
## Matrice de pondération spatiale standardisée en ligne
W.Connect2500m <- nb2listw(Connect2500m, zero.policy=TRUE, style = "W")
```

1.2.4.4 Matrices de pondération spatiale selon l'inverse de la distance et l'inverse de la distance au carré

Le code ci-dessous, qui permet de les créer les matrices de l'inverse de la distance et de l'inverse de la distance au carré, comprend les étapes suivantes :

- Récupération des coordonnées géographiques des entités spatiales.
- Création de la matrice de distance euclidienne $n \times n$ (n étant le nombre d'entités spatiales de la couche).
- Calcul des matrices d'inverse de la distance et d'inverse de la distance au carré.
- Standardisation de ces deux matrices et transformation dans des objets `listw` avec la fonction `mat2listw`.

```
## Coordonnées des centroïdes des entités spatiales
coords <- st_coordinates(IRIS.centroides)
## Création de la matrice de distance
distances <- as.matrix(dist(coords, method = "euclidean"))
# S'assurer que la diagonale de la matrice est à 0
```

```
diag(distances) <- 0
## Matrices inverse de la distance et inverse de la distance au carré
InvDistances <- ifelse(distances!=0, 1/distances, distances)
InvDistances2 <- ifelse(distances!=0, 1/distances^2, distances)
## Matrices de pondération spatiale standardisées en ligne
W_InvDistances <- mat2listw(InvDistances, style="W")
W_InvDistances2 <- mat2listw(InvDistances2, style="W")
```

💡 Astuce

Intégration d'autres types de distance

À la section 1.2.2, nous avons vu que plusieurs types de distances peuvent être utilisés : cartésiennes (euclidienne et de Manhattan) et réticulaires (chemin le plus rapide à pied, à vélo, en automobile et en transport en commun). Pour construire une matrice de distance de Manhattan, vous devez changer la valeur du paramètre `method` de la fonction `dist` comme suit : `as.matrix(dist(coords, method = "manhattan"))`.

Pour intégrer une distance réticulaire, vous devez la calculer, soit dans R (Apparicio et Gelb 2024), soit dans un logiciel de système d'information géographique (QGIS ou ArcGIS Pro avec l'extension *Network Analyst* par exemple) et l'importer dans R. Le reste du code sera alors identique.

Nous avons vu qu'il est possible d'utiliser une matrice de distance en fixant une distance maximale au-delà de laquelle les pondérations sont mises à 0 (équation 1.8). Le code ci-dessous permet de créer des matrices de pondération standardisées avec l'inverse de la distance et l'inverse de la distance au carré avec des seuils de 2500 et de 5000 mètres.

```
## Coordonnées des centroïdes des entités spatiales
coords <- st_coordinates(IRIS.centroides)
## Création de la matrice de distance
distances <- as.matrix(dist(coords, method = "euclidean"))
## Création de différentes matrices avec différents seuils
InvDistances.1000m <- ifelse(distances<=1000 & distances!=0, 1/distances, 0)
InvDistances.2500m <- ifelse(distances<=2500 & distances!=0, 1/distances, 0)
InvDistances.5000m <- ifelse(distances<=5000 & distances!=0, 1/distances, 0)
InvDistances2.2500m <- ifelse(distances<=2500 & distances!=0, 1/distances^2, 0)
InvDistances2.5000m <- ifelse(distances<=5000 & distances!=0, 1/distances^2, 0)
## Matrices de pondération spatiale standardisées en ligne
W_InvDistances.1000 <- mat2listw(InvDistances.1000m, style="W", zero.policy = TRUE)
W_InvDistances.2500 <- mat2listw(InvDistances.2500m, style="W", zero.policy = TRUE)
W_InvDistances.5000 <- mat2listw(InvDistances.5000m, style="W", zero.policy = TRUE)
W_InvDistances2.2500 <- mat2listw(InvDistances2.2500m, style="W", zero.policy = TRUE)
W_InvDistances2.5000 <- mat2listw(InvDistances2.5000m, style="W", zero.policy = TRUE)
```

Spécifier un seuil de distance trop réduit peut toutefois être problématique. Par exemple, sur les 506 IRIS de l'agglomération de Lyon, 68 n'ont pas de voisins à 1000 mètres, indiqués par le résultat suivant : 68 regions with no links.

```
summary(W_InvDistances.1000, zero.policy=TRUE)
```

Characteristics of weights list object:

Neighbour list object:

Number of regions: 506

Number of nonzero links: 3992

Percentage nonzero weights: 1.559156

Average number of links: 7.889328

68 regions with no links:

5 9 11 16 20 22 26 29 39 46 49 51 59 64 67 69 79 81 88 97 98 99 103 105

108 113 114 117 118 121 124 126 128 129 130 131 133 135 137 141 143 151

153 157 158 160 161 162 163 165 166 167 168 169 171 176 177 178 179 181

184 185 186 370 377 387 415 506

86 disjoint connected subgraphs

Link number distribution:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

68 39 25 28 27 30 34 40 38 14 18 12 13 16 8 10 11 11 10 13 7 11 7 3 5 5

26 28

2 1

39 least connected regions:

7 8 13 14 15 25 28 35 41 42 43 48 52 56 61 65 71 72 85 86 91 93 94 96 106 116 138 140 148 155 172 174 183 187 22

1 most connected region:

424 with 28 links

Weights style: W

Weights constants summary:

n	nn	S0	S1	S2
W	438	191844	438	203.0076 1789.585

Attention

Réduction de la taille des matrices de distance

Plusieurs logiciels (notamment ArcGIS Pro et GeoDa) réduisent par défaut la taille des matrices de distance de la façon suivante : 1) construction d'une matrice de distance uniquement pour l'entité la plus proche (la matrice résultante est donc de dimension $n \times 1$); 2) obtention de la distance maximale dans cette matrice, soit la distance la plus grande entre une entité spatiale et celle la plus proche; 3) construction de la matrice de distance finale avec comme seuil la distance maximale obtenue à l'étape précédente.

Cette réduction procure deux avantages importants :

- **Une diminution considérable des temps de calcul**, surtout pour les couches géographiques comprenant un nombre très élevé d'entités spatiales. Par exemple, avec une couche de 50 entités spatiales, la matrice des distances comprendra 2500 valeurs ($50 \times 50 = 2500$) tandis qu'avec 1000 entités spatiales, elle en comprendra un million ($1000 \times 1000 = 1\,000\,000$).
- Comme décrit plus haut, il est préférable d'**éviter d'avoir une matrice de distances avec des entités spatiales sans voisins**, puisque cela a un impact négatif sur les mesures d'autocorrélation spatiale.

La syntaxe ci-dessous permet ainsi de construire des matrices de pondération (inverse de la distance et inverse de la distance au carré) à partir de la distance maximale et un SR et son voisin le plus proche.

```
## Coordonnées des centroïdes des entités spatiales
coords <- st_coordinates(IRIS.centroides)
## Trouver le plus proche voisin
k1 <- knn2nb(knearneigh(coords))
## Affichage des distances pour les 506 IRUS au le plus proche
round(unlist(nbdists(k1,coords)),0)
```

```
[1] 506 739 776 451 1000 284 547 629 1206 449 1145 603 727 965 370
[16] 1835 389 334 725 1264 523 1543 449 710 990 1692 617 643 1145 509
[31] 489 488 600 334 841 407 770 301 1072 884 426 985 899 417 249
[46] 1316 588 426 1067 402 1366 718 452 657 696 600 759 482 1841 225
[61] 999 452 229 1616 418 220 1144 547 1482 284 985 926 417 716 678
[76] 355 345 603 1020 188 2040 325 841 770 767 727 767 1082 614 622
[91] 746 225 974 841 348 926 1108 1354 1290 545 220 355 1723 338 1686
[106] 370 188 1130 249 451 348 340 1063 1256 471 746 1094 1507 303 471
[121] 2332 831 301 1129 229 1638 677 2237 1092 2157 1050 452 1058 389 2523
[136] 682 1385 418 692 629 1097 459 1543 520 340 325 617 718 303 495
[151] 2218 369 1175 619 993 392 3366 1072 453 1050 1464 1385 1780 657 1181
[166] 1170 1324 1170 1477 352 1477 752 394 852 611 1562 1279 1279 1539 352
[181] 1035 611 394 1834 1300 1057 852 249 460 427 370 272 410 343 319
[196] 346 235 271 491 159 398 386 272 403 512 325 209 286 270 261
[211] 286 157 832 452 450 464 280 492 236 561 276 186 804 219 576
[226] 290 275 560 321 221 583 252 456 460 325 191 314 756 346 482
[241] 472 523 863 259 271 247 238 341 383 699 423 247 221 509 358
[256] 385 316 539 219 602 368 883 599 346 393 245 599 492 940 398
[271] 458 290 414 475 321 286 438 276 347 660 394 155 269 249 328
[286] 393 503 270 392 630 269 281 725 394 408 259 493 517 523 459
[301] 284 438 390 280 311 407 489 534 329 413 301 368 311 462 289
[316] 251 276 387 293 815 467 340 492 261 165 475 602 235 404 384
[331] 976 300 407 486 309 211 319 275 341 756 283 395 299 469 360
[346] 360 870 261 159 211 530 236 292 209 329 346 519 531 597 394
[361] 290 290 165 270 531 343 175 335 281 1046 398 390 267 267 507
[376] 358 1071 341 249 361 292 540 373 157 273 286 1173 253 423 442
[391] 472 274 299 318 335 345 487 503 278 442 341 660 249 238 238
[406] 511 295 725 469 220 481 422 416 456 1038 416 509 318 462 245
[421] 658 341 570 175 291 374 269 355 408 312 514 287 786 520 380
[436] 286 360 520 800 444 475 343 438 641 383 444 508 325 530 448
[451] 533 562 490 290 293 284 155 252 318 289 348 429 210 318 447
[466] 318 297 546 495 311 376 376 443 546 220 277 327 688 362 146
[481] 830 460 431 620 603 725 396 495 362 277 146 298 422 389 389
[496] 422 327 678 688 425 607 289 522 396 431 2365
```

```
## Trouver la distance maximale
plusprochevoisin.max <- max(unlist(nbdists(k1,coords)))
cat("Distance maximale au plus proche voisin :", round(plusprochevoisin.max,0), "mètres")
```

Distance maximale au plus proche voisin : 3366 mètres

```
## Matrice de distance avec la valeur maximale
# les voisins les plus proches avec le seuil de distance maximal
Voisins.DistMax <- dnearneigh(coords, 0, plusprochevoisin.max)
# Distances avec le seuil maximum
distances <- nbdists(Voisins.DistMax, coords)
# Inverse de la distance
InvDistances <- lapply(distances, function(x) (1/x))
# Inverse de la distance au carré
InvDistances2 <- lapply(distances, function(x) (1/x^2))
## Matrices de pondération spatiale standardisées en ligne
W_InvDistances <- nb2listw(Voisins.DistMax, glist = InvDistances, style = "W", zero.policy = TRUE)
W_InvDistances2 <- nb2listw(Voisins.DistMax, glist = InvDistances2, style = "W", zero.policy = TRUE)
```

1.2.4.5 Matrices de pondération spatiale selon le critère des plus proches voisins

La fonction `knearneigh` du *package* `spdep` crée des matrices de distance selon le critère des plus proches voisins, dont le nombre est fixé avec le paramètre `k`.

```
## Coordonnées géographiques des centroïdes des polygones
coords <- st_coordinates(st_centroid(LyonIris))
## Matrices des plus proches voisins de 2 à 5
k2 <- knn2nb(knearneigh(coords, k = 2))
k3 <- knn2nb(knearneigh(coords, k = 3))
k4 <- knn2nb(knearneigh(coords, k = 4))
k5 <- knn2nb(knearneigh(coords, k = 5))
## Matrices de pondération spatiale standardisées en ligne
W.k2 <- nb2listw(k2, zero.policy=FALSE, style = "W")
W.k3 <- nb2listw(k3, zero.policy=FALSE, style = "W")
W.k4 <- nb2listw(k4, zero.policy=FALSE, style = "W")
W.k5 <- nb2listw(k5, zero.policy=FALSE, style = "W")
```


💡 Astuce

L'ensemble des matrices en quelques lignes de code!

```
library(dplyr)
# Nom de la couche sf départ à changer au besoin
Couche.sf <- LyonIris

# Matrice de contiguïté
Queen <- poly2nb(Couche.sf, queen=TRUE)
Rook <- poly2nb(Couche.sf, queen=FALSE)
W.Queen <- nb2listw(Queen, zero.policy=TRUE, style = "W")
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")

# Matrice de contiguïté (ordre 2 à 5)
W.Rook2 <- nblag_cumul(nblag(Rook, 2)) %>% nb2listw(zero.policy=TRUE, style = "W")
W.Rook3 <- nblag_cumul(nblag(Rook, 3)) %>% nb2listw(zero.policy=TRUE, style = "W")
W.Rook4 <- nblag_cumul(nblag(Rook, 4)) %>% nb2listw(zero.policy=TRUE, style = "W")
W.Rook5 <- nblag_cumul(nblag(Rook, 5)) %>% nb2listw(zero.policy=TRUE, style = "W")

# Matrice de connectivité binaire
Centroides <- st_centroid(Couche.sf)
W.Connect2500m <- dnearneigh(Centroides, d1 = 0, d2 = 2500) %>%
  nb2listw(zero.policy=TRUE, style = "W")

# Inverse de la distance et inverse de la distance au carré (matrices complètes)
distances <- as.matrix(dist(st_coordinates(Centroides), method = "euclidean"))
diag(distances) <- 0
W_InvDistances <- ifelse(distances!=0, 1/distances, distances) %>% mat2listw(style="W")
W_InvDistances2 <- ifelse(distances!=0, 1/distances^2, distances) %>% mat2listw(style="W")

# Inverse de la distance et inverse de la distance au carré (matrices réduites)
coords <- st_coordinates(Centroides)
plusprochevoisin.max <- max(unlist(nbdists(knn2nb(knearneigh(coords)), coords)))
dists <- nbdists(dnearneigh(coords, 0, plusprochevoisin.max), coords)
W_InvDistancesReduite <- lapply(dists, function(x) (1/x)) %>%
  nb2listw(Voisins.DistMax, glist = ., style = "W", zero.policy = TRUE)
W_InvDistances2Reduite <- lapply(dists, function(x) (1/x^2)) %>%
  nb2listw(Voisins.DistMax, glist = ., style = "W", zero.policy = TRUE)

# Inverse de la distance et inverse de la distance au carré avec un seuil de distance
W_InvDistances.1000m <- ifelse(distances<=1000 & distances!=0, 1/distances, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances.2500m <- ifelse(distances<=2500 & distances!=0, 1/distances, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances.5000m <- ifelse(distances<=5000 & distances!=0, 1/distances, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances2.2500m <- ifelse(distances<=2500 & distances!=0, 1/distances^2, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances2.5000m <- ifelse(distances<=5000 & distances!=0, 1/distances^2, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
```

Matrices des plus proches voisins de 2 à 5

```
W.k2 <- knn2nb(knearneigh(coords, k = 2)) %>% nb2listw(zero.policy=FALSE, style = "W")
W.k3 <- knn2nb(knearneigh(coords, k = 3)) %>% nb2listw(zero.policy=FALSE, style = "W")
W.k4 <- knn2nb(knearneigh(coords, k = 4)) %>% nb2listw(zero.policy=FALSE, style = "W")
```

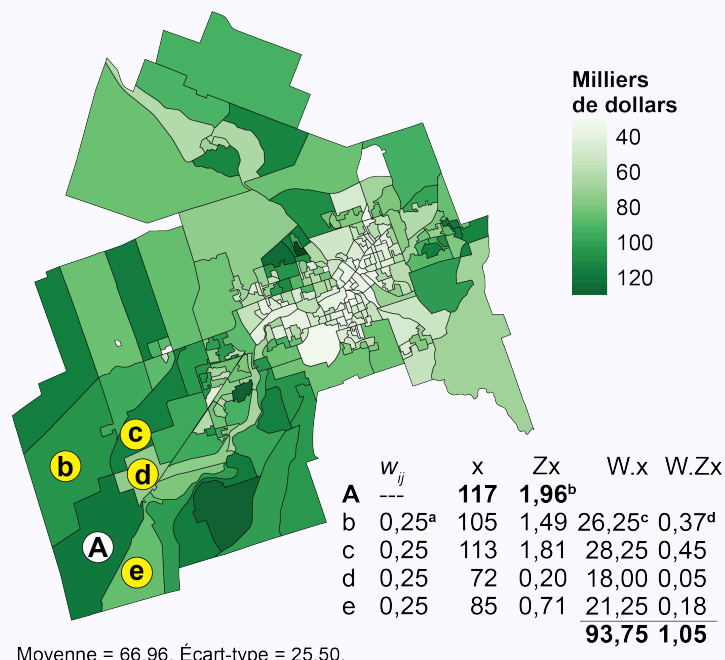
1.3 Variable spatialement décalée

À partir des matrices de pondérations standardisées en ligne, qu'elles soient de contiguïté ou de proximité, il est possible de calculer pour une version spatialement décalée d'une variable initiale. De la sorte, pour une entité spatiale i , il est possible de connaître la valeur moyenne d'une variable pour ses entités voisines ou proches (dépendamment si la matrice est définie selon la contiguïté ou la proximité). Au chapitre 2, nous verrons que ces variables spatiales décalées sont utilisées dans plusieurs modèles d'économétrie spatiale.

i Note

Comment calculer une variable spatialement décalée avec une matrice de pondération spatiale?

À la figure 1.8, nous détaillons le calcul de la valeur d'une variable spatialement décalée pour l'entité spatiale **A** à partir d'une matrice de contiguïté (selon le partage d'un segment) standardisée en ligne. Notez que **A** est adjacente à quatre entités spatiales (b, c, d et e).



Moyenne = 66,96. Écart-type = 25,50.

^a w_{ij} = poids standardisés en ligne de la matrice de contiguïté, soit 1/4 voisins.

^b cote $z = (117 - 66,96) / 25,50 = 1,96$

^c $105 \times 0,25 = 26,25$ ^d $1,49 \times 0,25 = 0,37$

$W.x_a = 93,75$, soit la moyenne de X pour les entités spatiales adjacentes à **A**

$W.Zx_a = 1,05$, soit la moyenne de la cote Z pour les entités spatiales adjacentes à **A**

Figure 1.8: Illustration du calcul d'une variable spatialement décalée

La fonction `lag.listw` du *package* `spdep` permet de créer une variable spatialement décalée en spécifiant la matrice de pondération spatiale et un vecteur numérique : `wzx <- lag.listw(listw, zx)`. Dans la syntaxe ci-dessous, nous créons une version spatiale décalée de la variable *dioxyde d'azote* (**N02**) de la couche **sf** *LyonIris* avec une matrice de pondérations spatiales standardisées en ligne (définie selon la contiguïté avec le partage d'un segment, **W.Rook**), puis nous cartographions les deux versions de la variable (initiale et spatialement décalée) à la figure 1.9.

```

# Chargement des données
library(tmap)
library(spdep)
load("data/Lyon.Rdata")
# Matrice de pondération spatiale standardisée en ligne selon la contiguïté
W.Rook <- nb2listw(poly2nb(LyonIris, queen=FALSE), zero.policy=TRUE, style = "W")
# Variable spatialement décalée
LyonIris$W_N02 <- lag.listw(W.Rook, LyonIris$N02)
# Cartographie avec tmap
tmap_mode("plot")
legende.parametres <- list(text.separator = "à", decimal.mark = ",", big.mark = " ")
Carte1 <- tm_shape(LyonIris)+
  tm_borders(col="gray25", lwd=.2)+
  tm_fill(col="N02", palette="Reds", n=5, style="quantile",
          legend.format = legende.parametres,
          title = "(a) Variable initiale")+
  tm_layout(frame=FALSE)+tm_scale_bar(breaks=c(0,5))
Carte2 <- tm_shape(LyonIris)+
  tm_borders(col="gray25", lwd=.2)+
  tm_fill(col="W_N02", palette="Reds", n=5, style="quantile",
          legend.format = legende.parametres,
          title = "(b) Spatialement décalée")+
  tm_layout(frame=FALSE)
tmap_arrange(Carte1, Carte2, ncol = 2, nrow = 1)

```

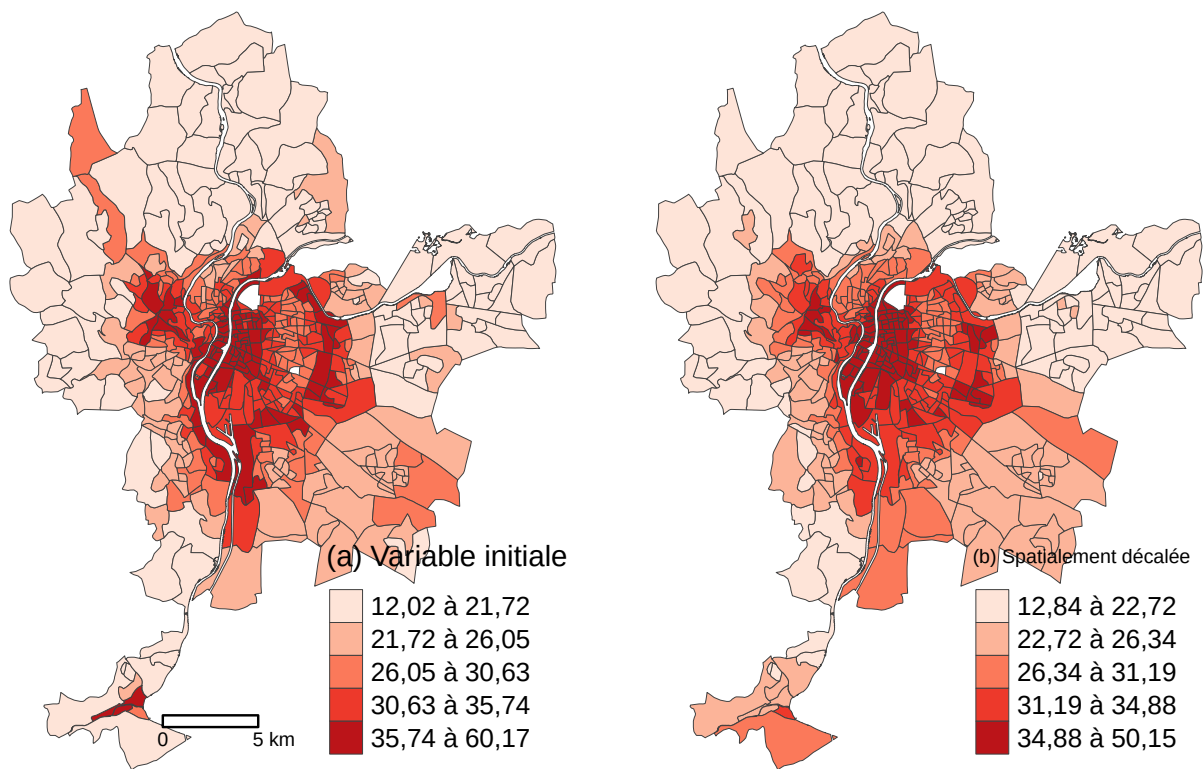


Figure 1.9: Exemple de variable spatialement décalée

1.4 Autocorrélation spatiale

L'autocorrélation spatiale permet d'estimer la corrélation d'une variable par rapport à sa localisation dans l'espace. Autrement dit, elle permet de vérifier si les entités proches ou voisines ont tendance à être (dis)semblables en fonction d'un phénomène donné (soit une variable). On distingue trois formes d'autocorrélation spatiale :

- **(a) Autocorrélation spatiale positive** : lorsque les entités spatiales voisines ou proches se ressemblent davantage que celles non contiguës ou éloignées. Cela renvoie ainsi à la première loi de la géographie : « tout interagit avec tout, mais les objets proches ont plus de chance de le faire que les objets éloignés » (traduction libre) (Tobler 1970).
- **(b) Autocorrélation spatiale négative** : lorsque les entités spatiales voisines ou proches ont tendance à être dissemblables, comparativement à celles non contiguës ou éloignées.
- **(c) Absence d'autocorrélation spatiale** : lorsque les valeurs de la variable sont distribuées aléatoirement dans l'espace; autrement dit, lorsqu'il n'y a pas de relation entre le voisinage ou la proximité des entités spatiales et leur degré de ressemblance.

🎯 Objectif

Différentes mesures d'autocorrélation spatiale

On distingue habituellement les statistiques d'autocorrélation spatiale globales et locales :

- Les statistiques d'autocorrélation spatiale globales qui renvoient une valeur pour l'ensemble de l'espace d'étude. Succinctement, elles permettent de vérifier si les entités proches ou voisines d'une couche géographique ont tendance à être (dis)semblables en fonction d'un phénomène donné (soit une variable).
- Les mesures d'autocorrélation spatiale locales renvoient des valeurs pour chacune des entités spatiales. Succinctement, il s'agit de vérifier si chaque entité spatiale est significativement (dis)semblable de celles voisines ou proches.

Nous aborderons ici uniquement une **statistique d'autocorrélation spatiale globale**, soit le ***I* de Moran**, pour évaluer l'autocorrélation spatiale d'une variable continue. Pour aborder une panoplie de mesures globales ou locales, nous invitons à consulter le chapitre intitulé *Autocorrélation spatiale* (Apparicio et Gelb 2024).

1.4.1 Formulation du *I* de Moran

Pour évaluer le degré d'autocorrélation spatiale d'une variable continue, les deux principales statistiques utilisées sont le *I* de Moran (1950) et le *c* de Geary (1954). Nous présentons ici uniquement le *I* de Moran pour deux raisons principales. Premièrement, étant basée sur la covariance, son interprétation est bien plus facile que celle du *c* de Geary (basé sur la variance des écarts), c'est-à-dire qu'elle est très similaire au bien connu **coefficient de corrélation de Pearson** (Apparicio et Gelb 2022). Deuxièmement, elle constitue la mesure la plus utilisée. Le *I* de Moran s'écrit :

$$I = \frac{n}{\sum_{i=1}^n \sum_{j=1}^n w_{ij}} \frac{\sum_{i=1}^n \sum_{j=1}^n w_{ij} (x_i - \bar{x})(x_j - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2} \text{ avec :} \quad (1.9)$$

- n , le nombre d'entités spatiales dans la couche géographique;
- w_{ij} , la valeur de la pondération spatiale entre les entités spatiales i et j ;
- x_i et x_j , les valeurs de la variable continue pour les entités spatiales i et j ;
- $\bar{x}$, la valeur moyenne de la variable X à l'étude.

Note**Standardisation de la matrice de pondération et I de Moran**

Nous avons vu que si la matrice de pondération spatiale est standardisée en ligne (section 1.2.3), alors chaque ligne de la matrice vaut 1 et la somme de l'ensemble des valeurs de la matrice est égale au nombre d'entités spatiales (n). Or, dans l'équation 1.9), $\sum_{i=1}^n \sum_{j=1}^n w_{ij}$ représente la somme des pondérations de la matrice, soit n si elles sont standardisées en ligne. Puisque $\frac{n}{n} = 1$, alors l'équation du I de Moran est simplifiée comme suit :

$$I = \frac{\sum_{i=1}^n \sum_{j=1}^n w_{ij} (x_i - \bar{x})(x_j - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (1.10)$$

Comme évoqué dans la section 1.2.3, cela démontre l'intérêt de la standardisation : la comparaison des valeurs du I de Moran obtenues avec différentes matrices de contiguïté afin de sélectionner (éventuellement) celle avec laquelle l'autocorrélation spatiale est la plus forte.

1.4.2 Interprétation du I de Moran

Avec une matrice standardisée, la statistique du I de Moran varie de -1 à 1 et s'interprète de la façon suivante :

- quand $I > 0$, l'autocorrélation est positive, c'est-à-dire que les entités géographiques ont tendance à se ressembler d'autant plus qu'elles sont voisines ou proches;
- quand $I = 0$, l'autocorrélation est nulle, c'est-à-dire que la contiguïté ou la proximité spatiale des zones ne jouent aucun rôle;
- quand $I < 0$, l'autocorrélation est négative, c'est-à-dire que les entités géographiques ont tendance à être dissemblables d'autant plus qu'elles sont voisines ou proches.

Les limites de -1 et 1 sont les maximums théoriques du I de Moran. Dans la pratique, elles sont limitées par la matrice spatiale utilisée dans le calcul. En effet, selon la matrice spatiale, le maximum du I de Moran peut être inférieur à 1, et son minimum supérieur à -1. Le calcul de ces bornes propres à chaque matrice spatiale peut se faire en utilisant les maximums et minimums des valeurs propres de $\frac{W+W^T}{2}$, quand la matrice spatiale est standardisée. À titre d'exemple, nous calculons ci-dessous les maximums et minimums possibles pour une matrice de contiguïté selon le partage d'un nœud (*Queen*) pour les 506 IRIS de l'agglomération de Lyon. Il apparaît ainsi que pour la matrice de contiguïté selon le partage d'un nœud (*Queen*), quelle soit la variable analysée, la valeur de I de Moran ne pourra pas dépasser les limites -0,64 et 1,04.

```
# Matrice de contiguïté selon le partage d'un nœud (Queen)
Queen <- poly2nb(LyonIris, queen = TRUE)
WQueen <- nb2listw(Queen, style = 'W')
QueenMat <- listw2mat(WQueen)
values <- range(eigen((QueenMat + t(QueenMat))/2)$values)
print(round(values,2))
```

```
[1] -0.69  1.04
```

1.4.3 Significativité du I de Moran

Comme pour le coefficient de corrélation calculé entre deux variables, il est possible de tester la significativité de la valeur du I de Moran obtenue. Sans que nous détaillions les calculs de significativité, notez qu'il existe trois manières de tester la significativité :

- selon l'hypothèse de la normalité;
- selon l'hypothèse de la randomisation;
- selon des permutations Monte-Carlo (habituellement avec 999 échantillons).

Aller plus loin

Comment calculer les trois tests de significativité du I de Moran?

Pour une description détaillée du calcul des trois tests, consultez l'ouvrage de Jean Dubé et Diego Legros (2014).

1.4.4 Mise en œuvre dans R

Objectif

Calcul du I de Moran dans R

Pour illustrer le calcul de I de Moran dans R, nous utilisons la couche des IRIS de l'agglomération de Lyon. Les étapes suivantes sont réalisées :

1. Construire une panoplie de matrices de pondération spatiale selon la contiguïté, la connectivité, la proximité et le critère des plus proches voisins.
2. Comparer les valeurs de significativité (p) pour une variable continue (N02).
3. Pour cette même variable, trouver avec quelle matrice la valeur du I de Moran est la plus forte.

1.4.4.1 Étape 1. Construction des matrices de pondération spatiale

```
library(dplyr)
library(sf)
library(spdep)
load("data/Lyon.Rdata")
# Matrice de contiguïté
Queen <- poly2nb(LyonIris, queen=TRUE)
Rook <- poly2nb(LyonIris, queen=FALSE)
W.Queen <- nb2listw(Queen, zero.policy=TRUE, style = "W")
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")
# Matrice de contiguïté (ordre 2 à 5)
W.Rook2 <- nblag_cumul(nblag(Rook, 2)) %>% nb2listw(zero.policy=TRUE, style = "W")
W.Rook3 <- nblag_cumul(nblag(Rook, 3)) %>% nb2listw(zero.policy=TRUE, style = "W")
W.Rook4 <- nblag_cumul(nblag(Rook, 4)) %>% nb2listw(zero.policy=TRUE, style = "W")
```

```

W.Rook5 <- nb1ag_cumul(nb1ag(Rook, 5)) %>% nb2listw(zero.policy=TRUE, style = "W")
# Matrice de connectivité binaire
Centroides <- st_centroid(LyonIris)
W.Connect2500m <- dnearneigh(Centroides, d1 = 0, d2 = 2500) %>%
  nb2listw(zero.policy=TRUE, style = "W")
# Inverse de la distance et inverse de la distance au carré (matrices complètes)
distances <- as.matrix(dist(st_coordinates(Centroides), method = "euclidean"))
diag(distances) <- 0
W_InvDistances <- ifelse(distances!=0, 1/distances, distances) %>% mat2listw(style="W")
W_InvDistances2 <- ifelse(distances!=0, 1/distances^2, distances) %>% mat2listw(style="W")
# Inverse de la distance et inverse de la distance au carré (matrices réduites)
coords <- st_coordinates(Centroides)
plusprochevoisin.max <- max(unlist(nbdists(knn2nb(knearneigh(coords)), coords)))
dists <- nbdists(dnearneigh(coords, 0, plusprochevoisin.max), coords)
W_InvDistancesReduite <- lapply(dists, function(x) (1/x)) %>%
  nb2listw(Voisins.DistMax, glist = ., style = "W", zero.policy = TRUE)
W_InvDistances2Reduite <- lapply(dists, function(x) (1/x^2)) %>%
  nb2listw(Voisins.DistMax, glist = ., style = "W", zero.policy = TRUE)
# Inverse de la distance et inverse de la distance au carré avec un seuil de distance
W_InvDistances.1000m <- ifelse(distances<=1000 & distances!=0, 1/distances, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances.2500m <- ifelse(distances<=2500 & distances!=0, 1/distances, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances.5000m <- ifelse(distances<=5000 & distances!=0, 1/distances, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances2.2500m <- ifelse(distances<=2500 & distances!=0, 1/distances^2, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
W_InvDistances2.5000m <- ifelse(distances<=5000 & distances!=0, 1/distances^2, 0) %>%
  mat2listw(style="W", zero.policy = TRUE)
## Matrices des plus proches voisins de 2 à 5
W.k2 <- knn2nb(knearneigh(coords, k = 2)) %>% nb2listw(zero.policy=FALSE, style = "W")
W.k3 <- knn2nb(knearneigh(coords, k = 3)) %>% nb2listw(zero.policy=FALSE, style = "W")
W.k4 <- knn2nb(knearneigh(coords, k = 4)) %>% nb2listw(zero.policy=FALSE, style = "W")
W.k5 <- knn2nb(knearneigh(coords, k = 5)) %>% nb2listw(zero.policy=FALSE, style = "W")

```

1.4.4.2 Étape 2. Calcul du I de Moran et des trois tests de significativité

Les fonctions `moran.test` et `moran.mc` du *package* `spdep` permettent de calculer le *I* de Moran selon les trois façons de tester la significativité :

- selon l'hypothèse de la normalité avec le paramètre `randomisation = FALSE`
 - `moran.test(ObjetSf$Variable, listw=MatriceW, zero.policy=TRUE, randomisation = FALSE)`
- selon l'hypothèse de la randomisation avec le paramètre `randomisation = TRUE`

- `moran.test(ObjetSf$Variable, listw=MatriceW, zero.policy=TRUE, randomisation = TRUE)`
- selon des permutations Monte-Carlo (ci-dessus avec 999 permutations)
 - `moran.mc(ObjetSf$Variable, listw=MatriceW, zero.policy=TRUE, nsim=999)`

Bien entendu, dans les sorties des trois méthodes, la valeur du I de Moran est la même, contrairement à la valeur de p qui peut varier.

```
moran.test(LyonIris$N02,      # nom de l'objet sf et de la variable continue
            listw=W.Queen,    # nom de la matrice de pondération spatiale
            zero.policy=TRUE,
            randomisation = FALSE) # significativité selon l'hypothèse de la normalité
```

Moran I test under normality

```
data: LyonIris$N02
weights: W.Queen
```

```
Moran I statistic standard deviate = 30.135, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:
```

Moran I statistic	Expectation	Variance
0.8189249687	-0.0019801980	0.0007420591

```
moran.test(LyonIris$N02,      # nom de l'objet sf et de la variable continue
            listw=W.Queen,    # nom de la matrice de pondération spatiale
            zero.policy=TRUE,
            randomisation = TRUE) # significativité selon l'hypothèse de la randomisation
```

Moran I test under randomisation

```
data: LyonIris$N02
weights: W.Queen
```

```
Moran I statistic standard deviate = 30.135, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:
```

Moran I statistic	Expectation	Variance
0.8189249687	-0.0019801980	0.0007420917

```
moran.mc(LyonIris$N02,      # nom de l'objet sf et de la variable continue
          listw=W.Queen,    # nom de la matrice de pondération spatiale
          zero.policy=TRUE,
          nsim=999)         # 999 permutations
```

Monte-Carlo simulation of Moran I

```
data: LyonIris$NO2
weights: W.Queen
number of simulations + 1: 1000

statistic = 0.81892, observed rank = 1000, p-value = 0.001
alternative hypothesis: greater
```

La statistique du I de Moran ($I = 0,82$, $p < 0,001$) indique que la variable *dioxyde d'azote* a une forte autocorrélation spatiale positive.

1.4.4.3 Étape 3. Identification de la plus forte autocorrélation spatiale selon les différentes matrices

La syntaxe ci-dessous permet de calculer la statistique du I de Moran avec plusieurs matrices de pondération spatiale.

```
## Création d'une liste pour toutes les matrices
ListeMatrices <- list(W.Queen = W.Queen,
                      W.Rook = W.Rook,
                      W.Rook2 = W.Rook2,
                      W.Rook3 = W.Rook3,
                      W.Rook4 = W.Rook4,
                      W.Rook5 = W.Rook5,
                      W.Connect2500m = W.Connect2500m,
                      W_InvDistances = W_InvDistances,
                      W_InvDistances2 = W_InvDistances2,
                      W_InvDistancesReduite = W_InvDistancesReduite,
                      W_InvDistances2Reduite = W_InvDistances2Reduite,
                      W_InvDistances.1000m = W_InvDistances.1000m,
                      W_InvDistances.2500m = W_InvDistances.2500m,
                      W_InvDistances2.2500m = W_InvDistances2.2500m,
                      W_InvDistances2.5000m = W_InvDistances2.5000m,
                      W.k2 = W.k2,
                      W.k3 = W.k3,
                      W.k4 = W.k4,
                      W.k5 = W.k5)

## Vecteur pour le I de Moran et la valeur de p
MoranI <- c()
Pvalue <- c()
i<-0
## Boucle pour calculer le I de Moran avec la liste des matrices
for (e in ListeMatrices){
  i<-i+1
  Test <-moran.mc(LyonIris$NO2,
                  listw=e,
```

```

      zero.policy=TRUE,
      nsim=999)
MoranI[i] <- Test$statistic
Pvalue[i] <- Test$p.value
}
# Création d'un DataFrame avec les valeurs du I de Moran et de p
MoranData1 <- data.frame(Matrices=names(ListeMatrices),
      MoranIs=round(MoranI, 4),
      Pvalues=Pvalue)
print(MoranData1)

```

	Matrices	MoranIs	Pvalues
1	W.Queen	0.8189	0.001
2	W.Rook	0.8244	0.001
3	W.Rook2	0.7174	0.001
4	W.Rook3	0.6215	0.001
5	W.Rook4	0.5192	0.001
6	W.Rook5	0.4194	0.001
7	W.Connect2500m	0.6327	0.001
8	W_InvDistances	0.1946	0.001
9	W_InvDistances2	0.4769	0.001
10	W_InvDistancesReduite	0.6301	0.001
11	W_InvDistances2Reduite	0.7011	0.001
12	W_InvDistances.1000m	0.6302	0.001
13	W_InvDistances.2500m	0.6766	0.001
14	W_InvDistances2.2500m	0.7235	0.001
15	W_InvDistances2.5000m	0.6554	0.001
16	W.k2	0.8083	0.001
17	W.k3	0.7903	0.001
18	W.k4	0.7865	0.001
19	W.k5	0.7844	0.001

La lecture détaillée du tableau 1.4 permet d'avancer plusieurs constats intéressants :

- D'emblée, signalons que toutes les valeurs sont positives et significatives, témoignant d'une autocorrélation spatiale positive.
- Concernant les **matrices de contiguïté**, l'autocorrélation spatiale est plus forte selon le partage d'un segment que d'un nœud (0,8244 contre 0,8189). Par conséquent, si nous devons choisir une matrice de contiguïté, il serait préférable d'utiliser celle définie selon le partage d'une chaîne (*Rook*).
- Sans surprise, plus nous ajoutons des **ordres d'adjacence**, plus la valeur de la statistique du *I* de Moran est faible, passant de 0,7174 à 0,4194 du deuxième au cinquième ordre.
- Concernant les **matrices de proximité**, la méthode de l'inverse de la distance au carré, qui accorde un poids plus important aux entités spatiales très proches (comparativement à l'inverse de la distance), renvoie des valeurs toujours plus élevées, et ce, que la matrice soit complète ou réduite. Aussi, les matrices de distance réduites présentent toujours des valeurs plus fortes que celles complètes.

- Concernant les matrices **selon le critère des plus proches voisins**, l'autocorrélation spatiale diminue légèrement de $k = 2$ à $k = 5$. D'ailleurs, la valeur la plus forte est pour deux voisins ($I = 0,8083$).

Tableau 1.4: Résultats du I de Moran selon les différentes matrices

Nom	Description	I de Moran	p (999 permutations)
Matrices de contiguïté			
W.Queen	Partage d'un nœud	0,8189	0,001
W.Rook	Partage d'un segment	0,8244	0,001
Matrices de contiguïté selon le partage d'un segment et ordre d'adjacence			
W.Rook2	Ordre 2	0,7174	0,001
W.Rook3	Ordre 3	0,6215	0,001
W.Rook4	Ordre 4	0,5192	0,001
W.Rook5	Ordre 5	0,4194	0,001
Matrices de connectivité			
W.Connect2500m	2500 mètres	0,6327	0,001
Matrices de distance (complètes)			
W_InvDistances	Inverse de la distance	0,1946	0,001
W_InvDistances2	Inverse de la distance au carré	0,4769	0,001
Matrices de distance (réduites)			
W_InvDistancesReduite	Inverse de la distance	0,6301	0,001
W_InvDistances2Reduite	Inverse de la distance au carré	0,7011	0,001
Matrices de distance avec un seuil maximal			
W_InvDistances.1000m	Inverse de la distance (2500 mètres)	0,6302	0,001
W_InvDistances.2500m	Inverse de la distance (5000 mètres)	0,6766	0,001
W_InvDistances2.2500m	Inverse de la distance au carré (2500 mètres)	0,7235	0,001
W_InvDistances2.5000m	Inverse de la distance au carré (5000 mètres)	0,6554	0,001
Matrices selon le critère des plus proches voisins			
W.k2	2 voisins	0,8083	0,001
W.k3	3 voisins	0,7903	0,001
W.k4	4 voisins	0,7865	0,001
W.k5	5 voisins	0,7844	0,001

Quelle est la matrice de pondérations spatiales avec laquelle la dépendance spatiale de la variable est la plus forte?

Pour la trouver, nous construisons un graphique avec les valeurs du I de Moran triées par ordre décroissant. La valeur la plus forte est obtenue avec la matrice de contiguïté selon le partage d'un segment.

```
library(ggplot2)
ggplot(data=MoranData1, aes(x=reorder(Matrices,MoranIs), y=MoranIs)) +
  geom_segment(aes(x=reorder(Matrices,MoranIs),
                    xend=reorder(Matrices,MoranIs),
                    y=0, yend=MoranIs)) +
  geom_point(size=4, fill="red", shape=21)+
```

```
xlab("Matrice de pondération spatiale") +
ylab("I de Moran")+
coord_flip()
```

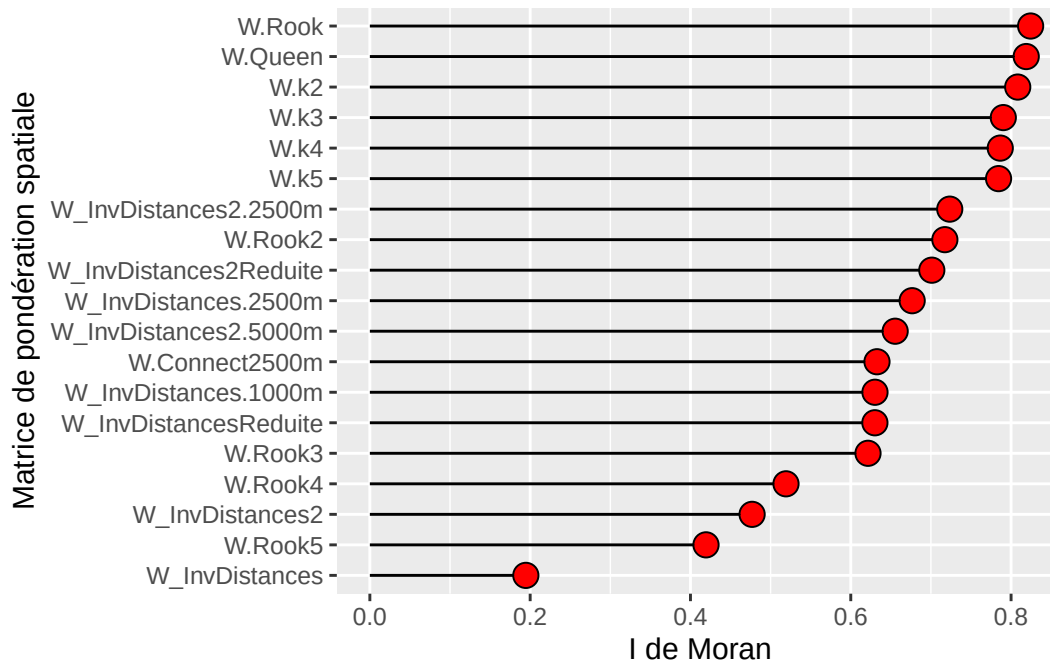


Figure 1.10: Valeurs du I de Moran selon les différentes matrices de pondération spatiale

1.4.4.4 Étape 4. Comparaison des valeurs du I de Moran pour plusieurs variables avec la même matrice

La syntaxe ci-dessous permet de calculer la statistique du I de Moran pour plusieurs variables avec la même matrice de pondération spatiale (ici, matrice de contiguïté selon le partage d'un segment).

```
## Vecteur pour les variables à analyser
listeVars <- c("Lden", "N02", "PM25", "VegHautPrt",
               "Pct0_14", "Pct_65", "Pct_Img", "TxChom1564", "Pct_brevet", "NivVieMed")

## Boucle pour calculer le I de Moran pour les différentes variables
MoranData2 <- t(sapply(listeVars, function(e){
  Test <- moran.mc(LyonIris[[e]],
                   listw=W.Rook,
                   zero.policy=TRUE,
                   nsim=999)
  result <- c(round(Test$statistic,4),
              Test$p.value)
}))

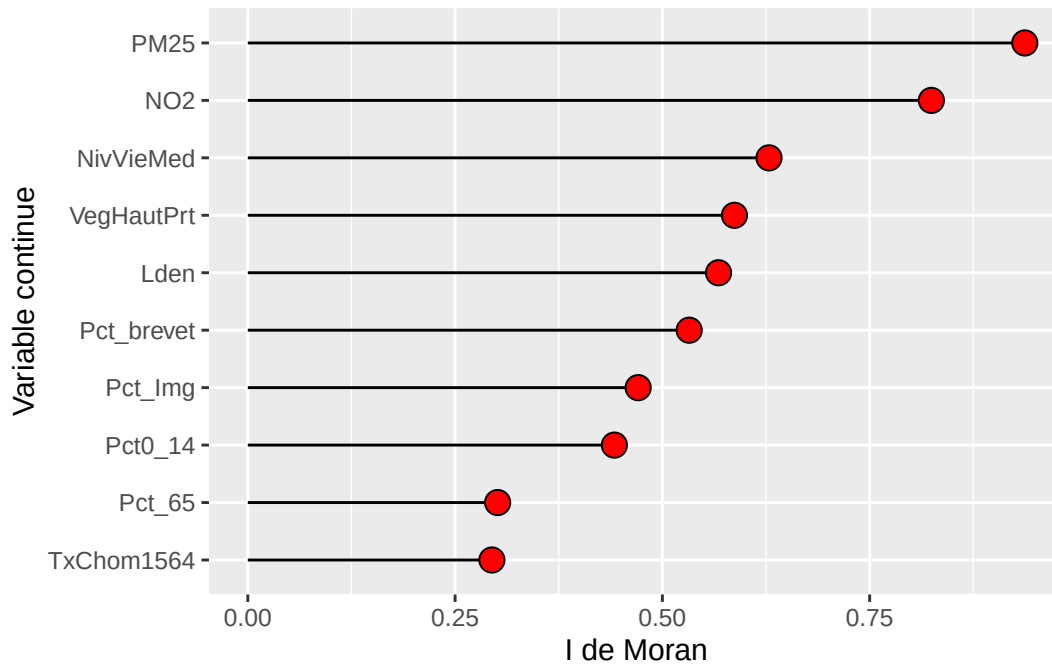
MoranData2 <- data.frame(MoranData2)
```

```
names(MoranData2) <- c('MoranIs', 'Pvalues')
MoranData2$Variable <- listeVars
rownames(MoranData2) <- NULL
print(MoranData2)
```

	MoranIs	Pvalues	Variable
1	0.5677	0.001	Lden
2	0.8244	0.001	N02
3	0.9371	0.001	PM25
4	0.5869	0.001	VegHautPrt
5	0.4421	0.001	Pct0_14
6	0.3013	0.001	Pct_65
7	0.4708	0.001	Pct_Img
8	0.2944	0.001	TxChom1564
9	0.5323	0.001	Pct_brevet
10	0.6286	0.001	NivVieMed

De nouveau, en quelques lignes de code, il est aisé de réaliser un graphique pour comparer les valeurs du I de Moran pour les différentes variables (figure 1.11).

```
library(ggplot2)
ggplot(data=MoranData2, aes(x=reorder(Variable,MoranIs), y=MoranIs)) +
  geom_segment( aes(x=reorder(Variable,MoranIs), xend=reorder(Variable,MoranIs), y=0, yend=MoranIs)) +
  geom_point( size=4, fill="red", shape=21)+
  xlab("Variable continue") + ylab("I de Moran")+
  coord_flip()
```

Figure 1.11: Valeurs du I de Moran pour les quatre variables

1.5 Retour sur la régression linéaire multiple

À titre de rappel, la régression linéaire multiple permet d'exprimer une variable d'intérêt, habituellement notée par le terme variable dépendante (y) en fonction de plusieurs variables que l'on pense liées, théoriquement, à la variable d'intérêt. Celles-ci sont habituellement désignées comme des variables explicatives ou indépendantes (notés x_k).

L'équation de régression s'écrit alors :

$$y_i = \alpha + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_K x_{Ki} + \varepsilon_i \quad (1.11)$$

Où :

- y_i , est la valeur de la variable dépendante y pour l'observation spatiale i .
- x_{ki} , est la valeur de la variable indépendante x_k pour l'observation spatiale i .
- α , est le paramètre d'ordonnée à l'origine, ou encore la constante. Elle représente la valeur moyenne de la variable dépendante lorsque l'ensemble des variables indépendantes (ou explicatives) sont nulles.
- $k = 1, 2, \dots, K$ est le nombre de variables indépendantes.
- β_1 à β_K , sont les coefficients de régression liés à chacune des variables indépendantes.
- ε_i , est le terme d'erreur, soit la partie de la valeur de la variable dépendante qui n'est pas expliquée par le modèle de régression. Le terme d'erreur est habituellement supposé de moyenne nulle, de variance homogène et indépendant.

À noter que le terme d'erreur est, par définition, inobservable. En revanche, il est possible d'approximer cette valeur en calculant le résidu, $\hat{\varepsilon}_i$, parfois aussi noté $\hat{e}_i$ (équations 1.12 et 1.13).

$$\hat{\varepsilon}_i = \hat{y}_i - y_i \quad (1.12)$$

$$\hat{\varepsilon}_i = y_i - \left[\hat{\alpha} + \hat{\beta}_1 x_{1i} + \hat{\beta}_2 x_{2i} + \cdots + \hat{\beta}_K x_{Ki} \right] \quad (1.13)$$

Il existe plusieurs manières d'exprimer cette relation sous forme plus compacte. L'une de celle-ci consiste à introduire le terme de sommation dans l'équation :

$$y_i = \alpha + \sum_{k=1}^K \beta_k x_{ki} + \varepsilon_i \quad (1.14)$$

Une autre approche, plus pratique et compacte, consiste à utiliser la notation matricielle :

$$y = \iota\alpha + X\beta + \varepsilon \quad (1.15)$$

Où :

- y , est un vecteur de dimension $(N \times 1)$ où N est le nombre total d'observations mobilisées dans l'analyse, avec $i = 1, 2, \dots, N$.
- ι est un vecteur de dimension $(N \times 1)$ comportant uniquement des valeurs de 1.
- X , une matrice de dimension $(N \times K)$ renfermant l'ensemble des K vecteurs de variables explicatives en plus d'une colonne de 1, dans la première colonne de la matrice, pour la constante, d'où $(K + 1)$.
- α est un scalaire identifiant l'ordonnée à l'origine.
- β , est un vecteur de dimension $(K + 1)$ qui renferme l'ensemble des coefficients de régression, pour les K variables indépendantes et la constante.
- ε , un vecteur de dimension $(N \times 1)$ des termes d'erreurs.

De manière plus explicite, les différents vecteurs et matrices prennent les formes suivantes :

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_i \\ \vdots \\ y_N \end{bmatrix} \quad (1.16)$$

$$\iota = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \\ \vdots \\ 1 \end{bmatrix} \quad (1.17)$$

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} & \cdots & x_{1k} & \cdots & x_{1K} \\ x_{21} & x_{22} & x_{23} & \cdots & x_{2k} & \cdots & x_{2K} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{i1} & x_{i2} & x_{i3} & \cdots & x_{ik} & \cdots & x_{iK} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & x_{N3} & \cdots & x_{Nk} & \cdots & x_{NK} \end{bmatrix} \quad (1.18)$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_k \\ \vdots \\ \beta_K \end{bmatrix} \quad (1.19)$$

Les modèles de régression linéaire sont largement mobilisés lorsqu'on s'intéresse à l'effet d'une variable spécifique sur le comportement de la variable d'intérêt. Le terme « effet marginal » est souvent associé aux coefficients estimés, β_k . Les coefficients permettent de simuler l'effet d'une variation d'une variable dépendante, Δx_k , sur la variation du comportement de la variable expliquée, Δy .

En termes plus techniques, les effets marginaux permettent d'exprimer l'effet de la dérivée partielle, c'est-à-dire d'isoler comment la variable d'intérêt bouge lorsqu'une des variables indépendantes bouge :

$$\frac{\partial y}{\partial x_k} = \beta_k \quad (1.20)$$

Ces effets sont habituellement l'essence de l'intérêt des modèles de régression. Par exemple, une variation de deux unités de la variable x_k devrait entraîner une variation dans y de $2\beta_k$ unités. Cette analyse est souvent le centre d'intérêt des modèles de régression, bien que l'interprétation causale ne soit pas nécessairement directe (Dubé et Legros 2014).

Aller plus loin

Régression linéaire multiple

Afin de bien maîtriser les principes de base et les hypothèses de la régression linéaire multiple, mesurer la qualité d'ajustement du modèle, introduire des variables explicatives particulières (variable qualitative dichotomique ou polytomique, variable d'interaction, etc.), interpréter les résultats d'un modèle de régression la mettre en œuvre dans R, nous vous invitons vivement à lire le [chapitre suivant](#) (Apparicio et Gelb 2022).

1.6 Pourquoi recourir à des régressions spatiales?

Deux motivations principales peuvent conduire à réaliser des régressions spatiales: la **dépendance spatiale** ou l'**hétérogénéité** du modèle de régression spatiale (Chi et Zhu 2019, pp. 41-43).

1.6.1 Dépendance spatiale

Dans un modèle, les résidus (ϵ) sont la différence entre les valeurs observées (y_i) et les valeurs prédites par le modèle ($\hat{y}_i$). Une des hypothèses de la régression linéaire multiple est que les observations doivent être indépendantes les unes des autres (*indépendance du terme d'erreur*). Le non-respect de cette hypothèse produit des résultats biaisés, notamment pour les coefficients de régression. Un simple test de présence d'autocorrélation spatiale entre les résidus permet de vérifier si cette hypothèse est respectée.

Le rejet de cette hypothèse d'indépendance rend les coefficients estimés, selon la forme fonctionnelle postulée (ou processus générateur des données – PGD), par le modèle biaisés et/ou biaise le calcul des écarts-types, invalidant ainsi les tests de significativité usuels. La présence d'autocorrélation spatiale entre les résidus est donc un premier test nécessaire pour s'assurer de la qualité des résultats issus du modèle de régression linéaire.

À noter qu'une autocorrélation spatiale entre certaines variables observables, dépendante ou indépendantes, est possible sans pour autant que les résidus soient autocorrélés. Dans ce cas, les relations entre les variables indépendantes permettent de capter l'effet spatial contenu dans la variable dépendante et les résultats d'estimations sont valides si les résidus sont homoscedastiques, c'est-à-dire de variance homogène (autrement, le problème peut facilement être réglé par un simple ajout d'option dans la commande du modèle de régression). Ce n'est donc pas sur les variables observables que le test de détection d'autocorrélation spatiale est important, mais bien sur les résidus du modèle.

Pour être valides, les résultats d'un modèle de régression linéaire construit avec des données spatiales ne devraient pas avoir des résidus spatialement autocorrélés. Si c'est le cas, alors certaines options sont envisageables pour tenter de remédier à la situation. La plus simple étant l'ajout de variables indépendantes spatialement structurées, afin de capter l'autocorrélation résiduelle. Les méthodes plus avancées proposent d'intégrer formellement cette relation spatiale dans le PGD.

Pour illustrer le problème de dépendance spatiale d'un modèle, nous calculons le modèle MCO suivant avec le jeu de données des IRIS de l'agglomération lyonnaise : `lm(PM25 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed, data = LyonIris)`.

```
load("data/Lyon.Rdata")
## Modèle MCO
Modele.MCO <- lm(PM25 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed, data = LyonIris)
summary(Modele.MCO)
```

Call:

```
lm(formula = PM25 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
    NivVieMed, data = LyonIris)
```

Residuals:

```
Min      1Q  Median      3Q      Max
```

```
-6.8510 -1.1962 -0.0094 1.2337 7.5446
```

Coefficients:

```
      Estimate Std. Error t value Pr(>|t|)
(Intercept) 23.382778    0.775600  30.148 < 2e-16 ***
Pct0_14      -0.151750    0.016326  -9.295 < 2e-16 ***
Pct_65       -0.049786    0.014571  -3.417 0.000685 ***
Pct_Img       0.088013    0.013240   6.648 7.82e-11 ***
Pct_brevet   -0.076002    0.009635  -7.888 1.94e-14 ***
NivVieMed    -0.113065    0.026357  -4.290 2.15e-05 ***
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.731 on 500 degrees of freedom

Multiple R-squared: 0.3539, Adjusted R-squared: 0.3474

F-statistic: 54.78 on 5 and 500 DF, p-value: < 2.2e-16

Pour vérifier la dépendance spatiale du modèle, nous calculons le I de Moran sur les résidus avec la fonction `lm.morantest` du *package* `spdep` et nous cartographions les résidus avec le *package* `tmap`. La statistique du I de Moran sur les résidus ($I = 0,66$, $p < 0,001$) indique clairement que le modèle OLS a un problème de dépendance spatiale et que ses coefficients de régression reportés plus haut sont certainement biaisés. Aussi, la cartographie des coefficients de régression montre que des résidus positifs dans les IRIS des quartiers centraux, tandis que les résidus négatifs sont concentrés dans les IRIS des quartiers ou des municipalités périphériques (figure 1.12). Par conséquent, le recours à des régressions spatiales est certainement nécessaire pour remédier à ce problème de dépendance spatiale du modèle MCO.

```
library(spdep)
library(tmap)
## Modèle MCO
LyonIris$MCO.Residus <- Modele.MCO$residuals
## Matrice de contiguïté
Rook <- poly2nb(LyonIris, queen=TRUE)
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")
# I de Moran sur les résidus du modèle global (MCO)
lm.morantest(Modele.MCO, W.Rook)
```

Global Moran I for regression residuals

data:

```
model: lm(formula = PM25 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
NivVieMed, data = LyonIris)
```

weights: W.Rook

Moran I statistic standard deviate = 24.453, p-value < 2.2e-16

alternative hypothesis: greater

sample estimates:

Observed Moran I	Expectation	Variance
0.6552777081	-0.0053724124	0.0007299083

```
# Cartographie des résidus
tmap_mode("plot")
tm_shape(LyonIris)+
  tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="MCO.Residus", n = 6, style = "pretty",
          legend.format = list(text.separator = "à",
                                decimal.mark = ","),
          midpoint = 0,
          palette = "-RdBu",
          title = "MCO") +
tm_layout(frame=FALSE)+
tm_scale_bar(breaks = c(0,5))
```

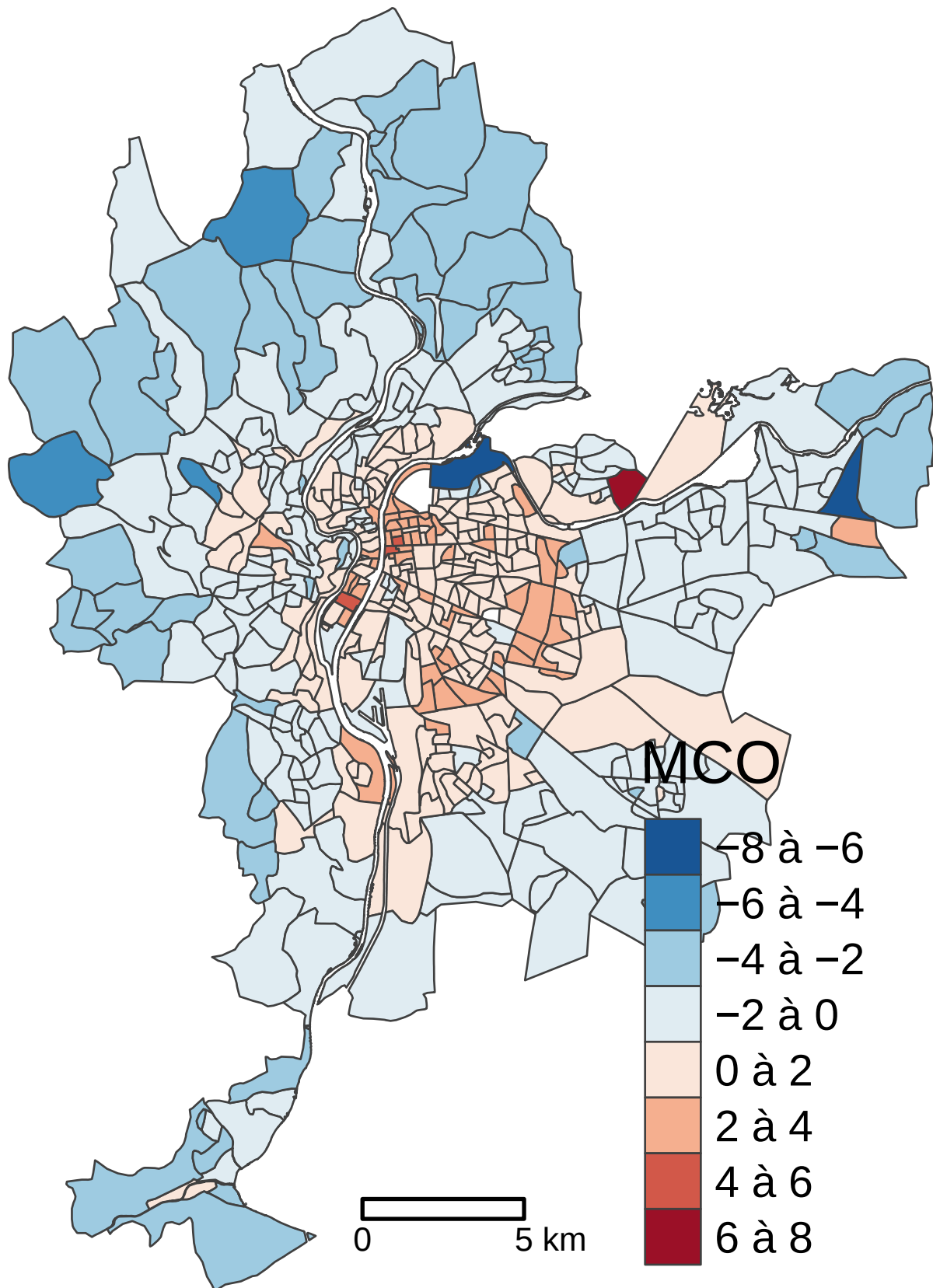


Figure 1.12: Cartographie des résidus d'un modèle MCO

Aller plus loin

Dépendance spatiale et méthodes de régression spatiale : quelles solutions?

Les modèles d'économétrie spatiale (chapitre 2 et chapitre 3) sont des solutions particulièrement intéressantes pour remédier au problème de dépendance spatiale d'un modèle et obtenir des coefficients de régression non biaisés.

Les modèles linéaires généralisés avec des vecteurs spatiaux ([chapitre Chapitre 7] à compléter.

1.6.2 Hétérogénéité spatiale

L'hétérogénéité ou la non-stationnarité (*Spatial heterogeneity* ou *nonstationarity*) se manifeste quand les relations entre la variables dépendante et indépendantes varient dans l'espace (en d'autres termes à travers les entités spatiales du jeu de données). Cela contrevient ainsi à l'hypothèse de l'homogénéité spatiale ou stationnarité spatiale selon laquelle les coefficients du modèles doivent être constant dans l'espace.

Ceci viole l'hypothèse de stationnarité spatiale, qui suppose que les paramètres du modèle (par exemple, les coefficients) sont constants dans l'espace.

Dans le chapitre 2, nous avons vu que les modèles d'économétrie spatiale visent à contrôler la dépendance spatiale d'un modèle de régression classique (MCO), afin d'améliorer l'estimation des coefficients de régression. L'objectif des modèles de régression géographiquement pondérée est différent : ils visent à analyser les variations spatiales de la relation entre la variable dépendante et les variables indépendantes. Autrement dit, les modèles GWR visent à explorer l'instabilité spatiale du modèle MCO afin d'analyser localement la relation entre la variable dépendante et les variables indépendantes. Pour une description détaillée en français de la GWR, consultez Apparicio et al. (2007).

Les modèles linéaires généralisés avec des vecteurs propres ([chapitre Chapitre 6] à compléter plus tard.

Aller plus loin

Hétérogénéité ou non-stationnarité spatiale et méthodes de régression spatiale : quelles solutions?

Les régressions géographiquement pondérées ([chapitre Chapitre 4 et chapitre 5), sont des outils d'exploration particulièrement adaptés pour explorer et cartographier l'instabilité des relations entre les variables dépendante et indépendantes d'un modèle de régression.

Les modèles généralisés additifs ([chapitre Chapitre 6) permettent d'introduire des lissages spatiaux pour capturer les tendances spatiales, soit de manière continues (avec une *spline* bivariée sur les coordonnées géographiques), soit de manière discrète (avec un lissage par champ aléatoire de Markov).

Partie 2. Économétrie spatiale

2 Modèles d'économétrie spatiale

Dans ce chapitre, nous décrivons uniquement les modèles économétriques spatiaux dont la variable dépendante est continue. Sommairement, ces modèles sont des extensions de la régression linéaire multiple, mais dans lequel une spécification explicite des relations spatiales entre les observations est introduite. Cette spécification vise ultimement à capter des effets spatiaux qui influencent le processus générateur de données (PGD), c'est-à-dire le modèle économétrique que l'on spécifie.

Objectif

Objectifs d'apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- À modifier.
- À modifier.
- À modifier.
- Analyser les résultats produits par ces différents modèles.
- Mettre en pratique ces modèles spatiaux par panel dans R.

Package

Liste des *packages* utilisés dans ce chapitre

- Pour importer et manipuler des fichiers géographiques :
 - `sf` pour importer et manipuler des données vectorielles.
- Pour construire des cartes et des graphiques :
 - `tmap` est certainement le meilleur *package* pour la cartographie.
 - `ggplot2` et `ggpubr` pour construire des graphiques.
- Pour construire des modèles spatiaux :
 - `spdep` pour construire des matrices de pondération spatiales et calculer le I de Moran.
 - `spatialreg` pour construire des modèles économétriques spatiaux.

⚠ Attention**Régression linéaire multiple et modèles économétriques spatiaux**

Dans cette section, nous décrivons uniquement les modèles économétriques spatiaux dont la variable dépendante est continue. Sommairement, ces modèles sont des extensions de la régression linéaire multiple, mais dans lequel une spécification explicite des relations spatiales entre les observations est introduite. Cette spécification vise ultimement à capter des effets spatiaux qui influencent le processus générateur de données (PGD), c'est-à-dire le modèle économétrique que l'on spécifie.

Par conséquent, avant de lire cette chapitre, il convient de bien maîtriser la régression linéaire multiple. Une brève récapitulation est proposée dans la section 1.5.

2.1 Les différents modèles spatiaux autorégressifs

Selon Jean Dubé et Diègo Legros, quelques raisons expliquent « le choix d'un modèle autorégressif : la présence d'externalités, les effets d'entraînement, l'omission de variables importantes, la présence d'hétérogénéité spatiale des comportements, les effets mixtes » (2014, 120). Les effets mixtes représentent une combinaison d'effets spatiaux. Dans tous les cas, la prise en compte des effets spatiaux passe par la spécification d'une matrice de pondérations spatiales, notée W et de dimension $(N \times N)$.

FAIRE UN TABLEAU Modèles / Motivation / Intégration de l'autocorrélation spatiale SLX / Externalités : influence des caractéristiques d'un voisin sur le comportement d'intérêt d'une observation donnée / Variables indépendantes (WX)

2.1.1 Modèle SLX : prise en compte des caractéristiques des voisins

2.1.1.1 Description du Modèle SLX

Dans un modèle SLX (*spatial lag of X model*), la dimension spatiale est intégrée par une possible influence des variables indépendantes des observations voisines, ou des caractéristiques des voisins. Cette spécification vise habituellement à intégrer ce que l'on identifie dans la littérature par les effets d'externalités spatiales, c'est-à-dire capter l'influence des caractéristiques d'un voisin sur le comportement d'intérêt d'une observation donnée (voir Figure x).

Plusieurs avantages sont liés à cette spécification. D'une part, cette approche peut se faire simplement en ajoutant des variables à l'équation de départ, soit la spécification du PGD original. La création de variables indépendantes, supposées exogènes, décalées spatialement peut être effectuée de manière simple (voir figure 2.29). D'autre part, ce modèle peut être estimé par moindres carrés ordinaires, ce qui n'est pas possible avec les autres spécifications.

Dans un modèle SLX, l'autocorrélation spatiale est intégrée au niveau des variables indépendantes. Autrement dit, les variables indépendantes spatialement décalées (WX) sont introduites aussi dans le modèle. Par conséquent, la valeur de chaque unité spatiale du modèle est ainsi expliquée à la fois par ses propres caractéristiques et celles dans le voisinage ou à proximité en fonction de la matrice de pondération spatiale (W).

Note**Rappel sur les variables spatialement décalées**

Dans le premier chapitre (section 1.3), nous avons vu comment calculer une variable spatialement décalée avec une matrice de pondérations spatiale spatiale (figure 1.8). À titre de rappel, lorsque cette dernière est standardisée en ligne, elle correspond à la valeur moyenne dans les unités voisines ou proches (dépendamment si la matrice est définie selon la contiguïté ou la proximité).

Le modèle SLX prend la spécification suivante :

$$y = \iota\alpha + X\beta + WX\theta + \varepsilon \quad (2.1)$$

Où :

- y , est la variable dépendante, soit un vecteur de dimension $(N \times 1)$ où N est le nombre total d'observations mobilisées dans l'analyse, avec $i = 1, 2, \dots, N$.
- ι est un vecteur de dimension $(N \times 1)$ comportant uniquement des valeurs de 1.
- X , une matrice de dimension $(N \times K)$ renfermant l'ensemble des K vecteurs de variables explicatives en plus d'une colonne de 1, dans la première colonne de la matrice, pour la constante, d'où $(K + 1)$.
- α est un scalaire identifiant l'ordonnée à l'origine.
- β , est un vecteur de dimension $(K + 1)$ qui renferme l'ensemble des coefficients de régression, pour les K variables indépendantes et la constante.
- WX est une matrice de variables indépendantes décalées spatialement, résultant d'une opération matricielle simple, soit $WX = W \times X$, où W est de dimension $(N \times N)$, X est de dimension $(N \times K)$, d'où WX est de dimension $(N \times K)$, où chaque observations représente la moyenne des valeurs des variables indépendantes autour d'une observation donnée.
- θ est un vecteur de coefficients associés aux valeurs décalées spatialement des variables indépendantes (excluant la constante) de dimension $(K \times 1)$.
- ε , un vecteur de dimension $(N \times 1)$ des termes d'erreurs.

Le PGD suggère que la valeur de chaque unité spatiale du modèle est ainsi expliquée à la fois par ses propres caractéristiques et celles de ses voisins ou des observations à proximité en fonction de la matrice de pondération spatiale (W).

Une des particularités du modèle SLX est qu'il introduit une complexité dans l'interprétation des résultats et le calcul des effets marginaux puisque la variable x_k apparaît à deux endroits dans le modèle : d'abord dans la matrice des variables indépendantes originales (X) et ensuite dans la matrice de variables indépendantes décalées spatialement (WX). Lorsque la matrice de pondérations est standardisée en ligne, le calcul des effets marginaux est alors donné par la dérivée partielle suivante :

$$\frac{\partial y}{\partial x_k} = \mathbf{I}\beta_k + W\theta_k \quad (2.2)$$

Pour une matrice de pondérations standardisées en ligne, cette expression se réduit à la valeur du coefficient associé à la variable indépendante originale, β_k , en plus de la valeur du coefficient associé à la variable décalée spatialement, θ_k . L'expression synthétise l'effet marginal total, alors que le coefficient β_k est habituellement désigné par l'effet direct et le coefficient θ_k synthétise l'effet indirect, puisque venant des voisins.

C'est d'ailleurs la valeur du coefficient θ_k qui introduit ce que l'on qualifie d'externalité spatiale. Il permet d'exprimer comment la variable dépendante varie lorsque les caractéristiques des voisins changent. Les changements dans la variable d'intérêt peuvent venir d'une variation des caractéristiques des voisins.

Ce modèle est aussi désigné sous le terme d'effet spatial local, par opposition à l'effet global (voir modèle SAR, section 2.2.2). La variation des caractéristiques des voisins a une portée limitée sur le changement de valeurs de la variable dépendante. Seulement certaines observations sont affectées : celles dont les caractéristiques des voisins ont changé.

Si le modèle SLX offre une solution intéressante pour régler le potentiel problème d'autocorrélation spatiale des résidus, rien n'assure pour autant qu'il permet de régler tous les problèmes, même si les coefficients associés aux variables décalées spatialement sont statistiquement significatifs. L'inclusion de variables indépendantes additionnelles peut ne pas suffire à capter ce qui, au départ dans la régression par MCO, se cache dans les résidus du modèle. Un test d'autocorrélation spatiale sur les résidus du modèle SLX est un réflexe nécessaire à avoir.

2.1.1.2 Mise en œuvre dans R

Le modèle SLX est construit avec la fonction `lmSLX` du *package* `spatialreg` (Bivand, Millo et Piras 2021). Remarquez, dans le code ci-dessous, le paramètre `listw=W.Rook` qui est utilisé pour spécifier la matrice de pondération spatiale.

```
## Chargement des packages et des données
library(sf)
library(tmap)
library(spdep)
library(spatialreg)
load("data/Lyon.Rdata")
## Matrice de contiguïté selon le partage d'un segment (Rook)
Rook <- poly2nb(LyonIris, queen=FALSE)
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")
## Construction du modèle MCO
Modele.MCO <- lm(NO2 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                 data = LyonIris)
## Construction du modèle
Modele.SLX <- lmSLX(NO2 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                   listw=W.Rook,      # matrice de pondération spatiale
                   data = LyonIris)  # dataframe
## Résultats du modèle
summary(Modele.SLX)
```

Call:

```
lm(formula = formula(paste("y ~ ", paste(colnames(x)[-1], collapse = "+"))),
   data = as.data.frame(x), weights = weights)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.068e+01	4.188e+00	1.210e+01	1.040e-29
Pct0_14	-2.040e-01	6.268e-02	-3.255e+00	1.211e-03
Pct_65	-3.771e-02	5.361e-02	-7.033e-01	4.822e-01
Pct_Img	1.041e-01	4.849e-02	2.146e+00	3.235e-02
Pct_brevet	-7.363e-02	3.550e-02	-2.074e+00	3.857e-02
NivVieMed	-1.844e-01	1.106e-01	-1.667e+00	9.617e-02
lag.Pct0_14	-7.759e-01	1.030e-01	-7.537e+00	2.315e-13
lag.Pct_65	-6.454e-02	9.115e-02	-7.081e-01	4.792e-01
lag.Pct_Img	6.465e-01	8.593e-02	7.524e+00	2.526e-13
lag.Pct_brevet	-3.013e-01	6.157e-02	-4.893e+00	1.344e-06
lag.NivVieMed	-1.805e-02	1.750e-01	-1.031e-01	9.179e-01

⚠ Attention**Effets directs, indirects et totaux**

La formulation d'un modèle SLX implique deux types d'effets pour les variables indépendantes (X) :

- les **effets directs**, soit ceux des caractéristiques des entités spatiales. Ils correspondent aux coefficients β des variables indépendantes (X). Autrement dit, pour une observation i , à chaque augmentation d'une unité d'une caractéristique X , la valeur de y_i va varier (augmenter ou diminuer) en fonction du coefficient β .
- les **effets indirects**, soit ceux des caractéristiques des entités spatiales voisines ou proches définies selon la matrice de pondération spatiale. Ils correspondent aux coefficients θ des variables indépendantes spatialement décalées (WX). Autrement dit, les valeurs de WX des entités spatiales proches ou voisines j de i vont aussi être amenées à varier, impactant alors les valeurs y_j selon les coefficients θ .
- Pour calculer l'impact total, il suffit de sommer son effet direct (β_k) et son effet indirect (θ_k) pour obtenir son effet total.

Le code suivant permet de calculer ces effets directs et indirects.

```
## Effets directs, indirects et totaux (uniquement les coefficients)
impacts(Modele.SLX)
```

Impact measures (SLX, glht):

	Direct	Indirect	Total
Pct0_14	-0.20403803	-0.77590830	-0.9799463
Pct_65	-0.03770918	-0.06453809	-0.1022473
Pct_Img	0.10406359	0.64653923	0.7506028
Pct_brevet	-0.07363272	-0.30128171	-0.3749144
NivVieMed	-0.18440960	-0.01804718	-0.2024568

```
## Effets directs, indirects et totaux (coefficients, valeurs de z et de p)
summary(impacts(Modele.SLX))
```

```
Impact measures (SLX, glht, n-k):
```

	Direct	Indirect	Total
Pct0_14	-0.20403803	-0.77590830	-0.9799463
Pct_65	-0.03770918	-0.06453809	-0.1022473
Pct_Img	0.10406359	0.64653923	0.7506028
Pct_brevet	-0.07363272	-0.30128171	-0.3749144
NivVieMed	-0.18440960	-0.01804718	-0.2024568

```
=====
```

Standard errors:

	Direct	Indirect	Total
Pct0_14	0.06268202	0.10295210	0.10045332
Pct_65	0.05361420	0.09114695	0.08556272
Pct_Img	0.04849085	0.08593145	0.08060028
Pct_brevet	0.03549819	0.06157121	0.05975821
NivVieMed	0.11063207	0.17499339	0.14911021

```
=====
```

Z-values:

	Direct	Indirect	Total
Pct0_14	-3.2551283	-7.5365951	-9.755241
Pct_65	-0.7033432	-0.7080664	-1.194998
Pct_Img	2.1460460	7.5238953	9.312658
Pct_brevet	-2.0742665	-4.8932234	-6.273857
NivVieMed	-1.6668729	-0.1031306	-1.357766

p-values:

	Direct	Indirect	Total
Pct0_14	0.0011334	4.8184e-14	< 2.22e-16
Pct_65	0.4818419	0.47890	0.23209
Pct_Img	0.0318693	5.3069e-14	< 2.22e-16
Pct_brevet	0.0380546	9.9198e-07	3.5221e-10
NivVieMed	0.0955397	0.91786	0.17454

VOIR AVEC JEAN!!! À la lecture des valeurs de p , nous constatons que seule la variable **Pct0_14** a un impact direct et indirect significatif au seuil 0,01. L'augmentation d'un point de pourcentage de la population de moins de 15 ans est associé localement à une réduction de 0,20 de la concentration annuelle du dioxyde d'azote. Chez les entités voisines, cette réduction est de 0,78. L'effet total est donc une réduction de 0,98.

Dépendance spatiale du modèle SLX?

Ce modèle a-t-il corrigé le problème de dépendance spatiale du modèle de régression linéaire classique? Avec une valeur du I de Moran de 0,605 ($p < 0,001$), les résidus sont toujours fortement autocorrélés spatialement (figure 2.1).

```
lm.morantest(Modele.SLX, W.Rook, alternative="two.sided")
```

Global Moran I for regression residuals

```
data:
model: lm(formula = formula(paste("y ~ ", paste(colnames(x)[-1],
collapse = "+"))), data = as.data.frame(x), weights = weights)
weights: W.Rook
```

Moran I statistic standard deviate = 21.951, p-value < 2.2e-16

alternative hypothesis: two.sided

sample estimates:

Observed Moran I	Expectation	Variance
0.6046602748	-0.0072844321	0.0007771643

```
LyonIris$SLX.Residus <- residuals(Modele.SLX)
tm_shape(LyonIris)+
  tm_fill(col="SLX.Residus", n = 5, style = "quantile",
    legend.format = list(text.separator = "à"),
    palette = "-RdBu", title = "Résidus") +
  tm_layout(frame=FALSE) +
  tm_scale_bar(breaks = c(0,5))
```

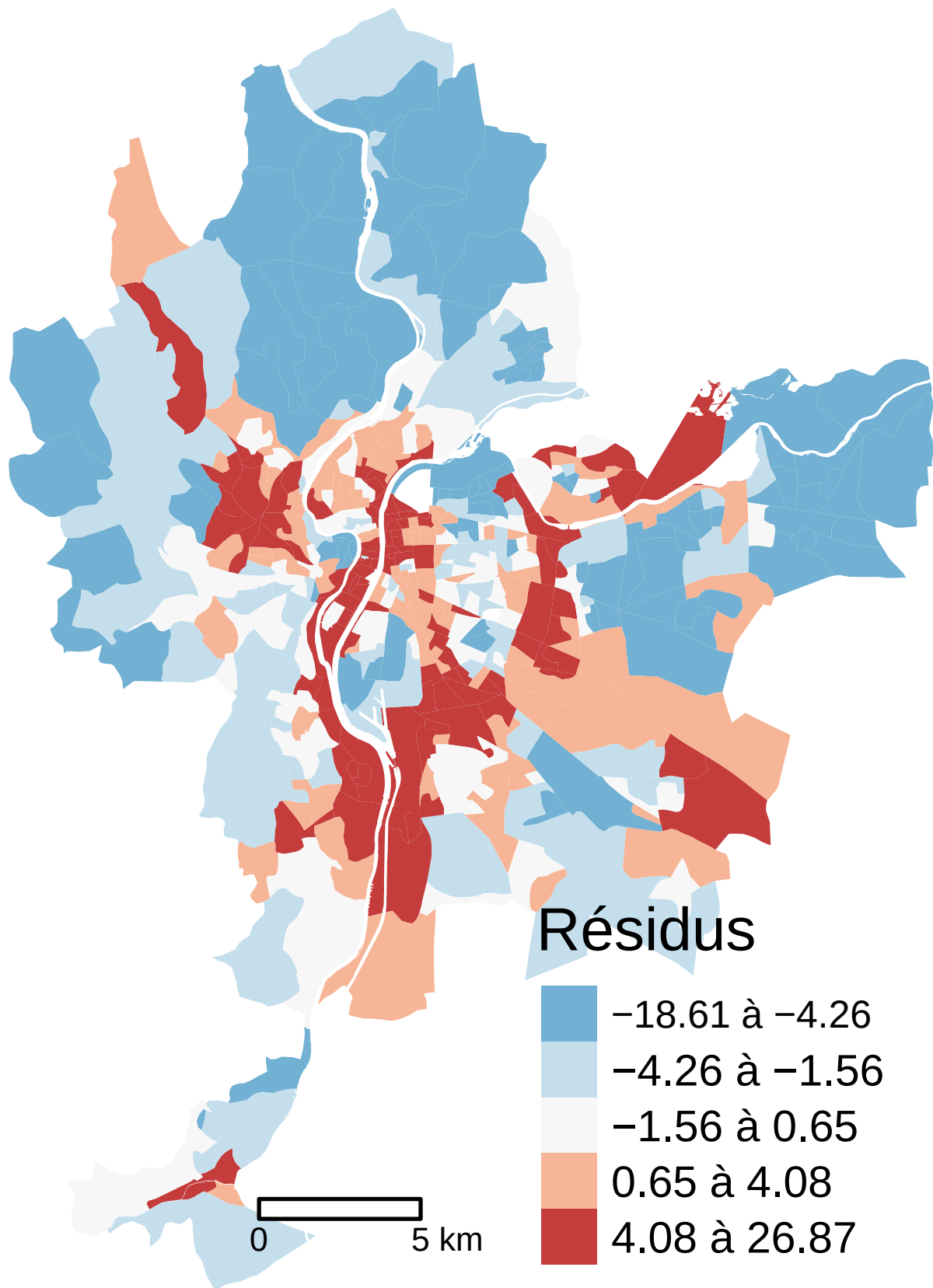


Figure 2.1: Cartographie des résidus du modèle SLX

2.1.2 Modèle SAR : autocorrélation spatiale sur la variable dépendante

Dans le modèle SAR (aussi appelé SAR-LAG), l'autocorrélation spatiale est intégrée au niveau de la variable dépendante (Wy), qui est ainsi spatialement décalée. L'idée générale est que la valeur de la variable dépendante pour une observation (y_i) peut être influencée par les valeurs de y des observations voisines et proches. L'exemple le plus classique est le prix de vente des maisons : il est influencé à la fois par les caractéristiques intrinsèques de la maison (X , par exemple, la superficie habitable, le nombre de chambres à coucher, de salles de bains, etc.) et par le prix de vente des maisons voisines (Wy). Jean Dubé et Diègo Legros (2014) qualifient ce phénomène « **d'effets d'entraînement ou d'effets de débordement** (*spillover effects*) » (2014, 123). L'équation du modèle SAR s'écrit alors :

$$y = Wy\rho + X\beta + \epsilon \quad (2.3)$$

avec :

- y , la variable dépendante.
- W , la matrice de pondération spatiale.
- Wy , la variable dépendante spatialement décalée.
- ρ (prononcez *rho*), le coefficient de la variable dépendante spatialement décalée. Il varie de -1 à 1.
- X , les variables indépendantes.
- β , les coefficients des variables indépendantes.
- ϵ , les résidus.

Construction du modèle SAR dans R

Le modèle SAR est construit avec la fonction `lagsarlm` du *package* `spatialreg`.

```
## Construction du modèle
Modele.SAR <- lagsarlm(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                      listw=W.Rook, # matrice de pondération spatiale
                      data = LyonIris, # dataframe
                      type = 'lag') # Modèle lag par défaut

## Résultats du modèle
summary(Modele.SAR, Nagelkerke=TRUE)
```

```
Call:lagsarlm(formula = N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
  NivVieMed, data = LyonIris, listw = W.Rook, type = "lag")
```

Residuals:

	Min	1Q	Median	3Q	Max
	-12.86859	-1.88111	-0.49760	0.94464	18.21351

Type: lag

Coefficients: (asymptotic standard errors)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	7.838906	1.646232	4.7617	1.919e-06
Pct0_14	-0.098708	0.030554	-3.2306	0.001235

Pct_65	-0.034543	0.026957	-1.2814	0.200044
Pct_Img	0.030241	0.024491	1.2348	0.216917
Pct_brevet	-0.019234	0.017855	-1.0772	0.281384
NivVieMed	-0.098413	0.048985	-2.0090	0.044534

Rho: 0.87939, LR test value: 620.31, p-value: < 2.22e-16

Asymptotic standard error: 0.01942

z-value: 45.283, p-value: < 2.22e-16

Wald statistic: 2050.5, p-value: < 2.22e-16

Log likelihood: -1366.157 for lag model

ML residual variance (sigma squared): 10.181, (sigma: 3.1908)

Nagelkerke pseudo-R-squared: 0.78962

Number of observations: 506

Number of parameters estimated: 8

AIC: 2748.3, (AIC for lm: 3366.6)

LM test for residual autocorrelation

test value: 0.6198, p-value: 0.43112

Dans les résultats ci-dessus, la valeur de ρ est de 0,88 (LR = 620, $p < 0,001$), traduisant un très fort effet d'entraînement. Autrement dit, lorsqu'en moyenne la concentration de dioxyde d'azote augmente dans les IRIS voisines (W_y), elle augmente aussi fortement chaque IRIS (y).

⚠ Attention**Effets directs, indirects et totaux**

Tout comme le modèle SLX vu précédemment, la formulation du modèle SAR-LAG implique des effets particuliers. Reprenons l'exemple d'un modèle prédisant le prix de vente des maisons avec cette fois-ci un modèle de type SAR-LAG :

1. L'augmentation de la superficie du jardin de la maison i va faire augmenter le prix de la maison i (y_i).
2. Cette augmentation de prix de la maison i aura un impact sur les voisins de i , soit les maisons j , car leur prix dépend du prix de la maison i au travers du terme $W_{ij}\rho$ du modèle. Par exemple, si ρ vaut 0,8, alors 80% de l'augmentation du prix de i va se répercuter sur le prix des maisons j .
3. De même, les voisines des maisons j , les maisons k vont aussi être impactées par le changement de prix des maisons j et ainsi de suite de voisins en voisins.
4. Au final, la maison i verra son prix augmenter encore plus, car le prix de ses voisines aura augmenté par effet de rétroaction.

Ce processus de propagation est appelé l'effet d'entraînement ou de débordement (*spillover*) en économétrie. L'effet original de l'augmentation de la taille du jardin sur la maison i , combiné à l'augmentation par rétroaction, est appelé l'effet direct. L'effet cumulé de l'augmentation de la taille du jardin sur toutes les autres maisons ($\neq i$) est appelé l'effet indirect. La somme des effets indirects et des directs est appelée effets totaux. À nouveau, il est possible d'utiliser la fonction `impacts` pour calculer ces effets directs et indirects.

```
## Effets directs, indirects et totaux (uniquement les coefficients)
impacts(Modele.SAR, listw = W.Rook, R = 999)
```

Impact measures (lag, exact):

	Direct	Indirect	Total
Pct0_14	-0.13878038	-0.6796248	-0.8184052
Pct_65	-0.04856624	-0.2378349	-0.2864012
Pct_Img	0.04251743	0.2082131	0.2507306
Pct_brevet	-0.02704205	-0.1324283	-0.1594703
NivVieMed	-0.13836534	-0.6775923	-0.8159576

```
## Effets directs, indirects et totaux (coefficients, valeurs de z et de p)
summary(impacts(Modele.SAR, listw = W.Rook, R = 999), zstats = TRUE, short = TRUE)
```

Impact measures (lag, exact):

	Direct	Indirect	Total
Pct0_14	-0.13878038	-0.6796248	-0.8184052
Pct_65	-0.04856624	-0.2378349	-0.2864012
Pct_Img	0.04251743	0.2082131	0.2507306
Pct_brevet	-0.02704205	-0.1324283	-0.1594703
NivVieMed	-0.13836534	-0.6775923	-0.8159576

Simulation results (variance matrix):

Simulated standard errors

	Direct	Indirect	Total
Pct0_14	0.04234340	0.2391413	0.2772947
Pct_65	0.03958087	0.2042968	0.2429762
Pct_Img	0.03454707	0.1763965	0.2101282
Pct_brevet	0.02488563	0.1289668	0.1533222
NivVieMed	0.07114932	0.3739999	0.4421282

Dépendance spatiale du modèle SAR?

Ce modèle a-t-il corrigé le problème de dépendance spatiale du modèle de régression linéaire classique? Avec une valeur du I de Moran de -0,014 ($p = 0,654$), les résidus ne sont plus spatialement autocorrélés (figure 2.2).

```
## Autocorrélation spatiale des résidus
moran.mc(resid(Modele.SAR), W.Rook, nsim=999)
```

Monte-Carlo simulation of Moran I

```
data: resid(Modele.SAR)
weights: W.Rook
number of simulations + 1: 1000
```

```
statistic = -0.014281, observed rank = 323, p-value = 0.677
alternative hypothesis: greater
```

```
## Cartographie des résidus
LyonIris$SAR.Residus <- resid(Modele.SAR)
tm_shape(LyonIris)+
  tm_fill(col="SAR.Residus", n = 5, style = "quantile",
          legend.format = list(text.separator = "à"),
          palette = "-RdBu", title = "Résidus") +
  tm_layout(frame=FALSE) +
  tm_scale_bar(breaks = c(0,5))
```

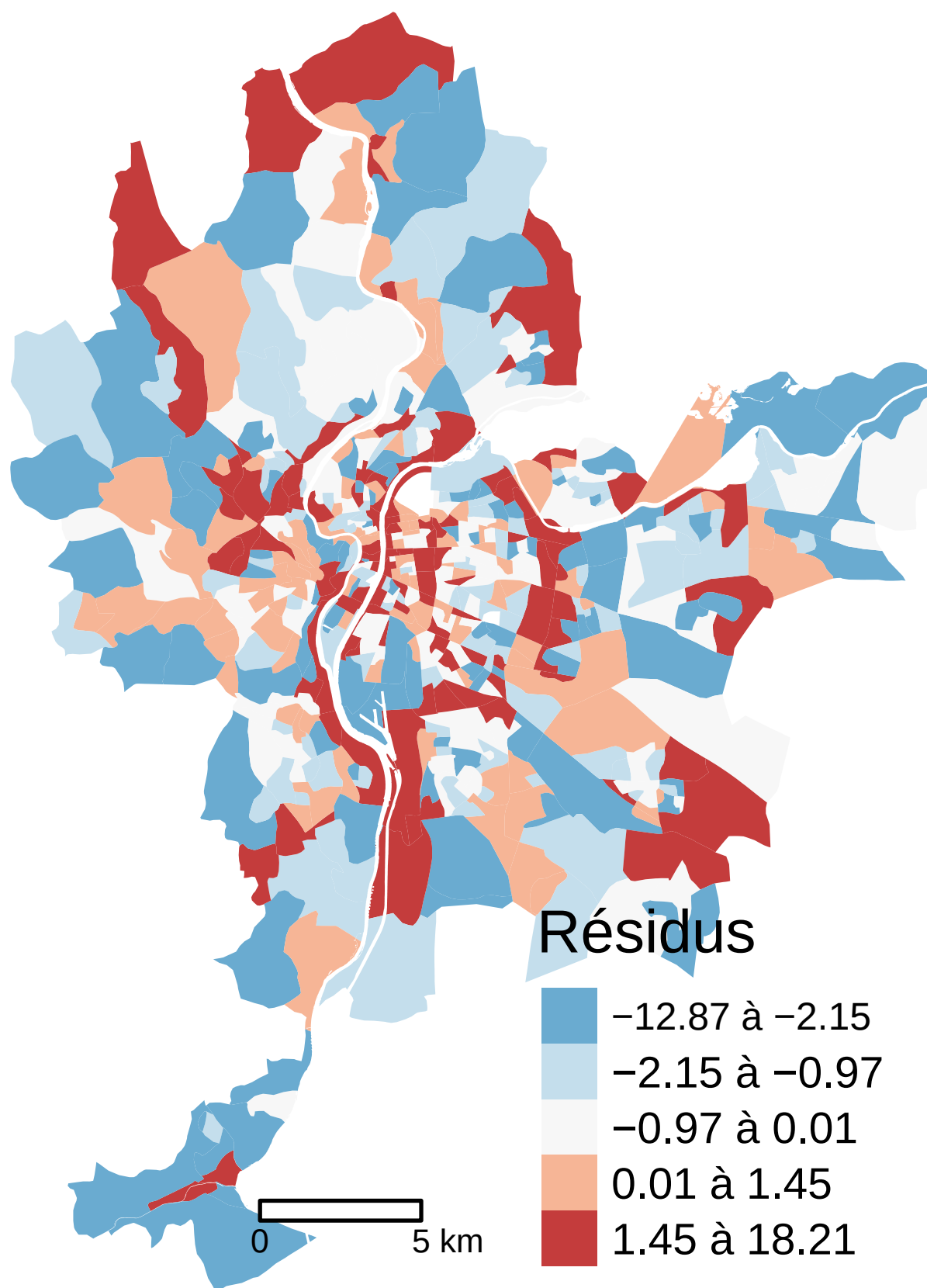


Figure 2.2: Cartographie des résidus du modèle SAR

2.1.3 Modèle SEM : autocorrélation spatiale sur le terme d'erreur

Dans le modèle SEM (*Spatial Error Model*, appelé aussi *SAR-ERROR*), l'intégration de l'autocorrélation spatiale est réalisée sur le terme d'erreur, ce qui pourrait se justifier par l'omission d'une variable dépendante spatialement structurée (Dubé et Legros 2014, 126). L'équation du modèle SEM s'écrit :

$$y = X\beta + u, u = \lambda W u + \epsilon \quad (2.4)$$

avec :

- y , la variable dépendante.
- W , la matrice de pondération spatiale.
- λ (prononcez *lambda*), le coefficient sur le terme d'erreur spatialement décalé. Il varie de -1 à 1.
- X , les variables indépendantes.
- β , les coefficients des variables indépendantes.
- ϵ , les résidus.

Construction du modèle SAR dans R

Le modèle SEM est construit avec la fonction `errorsarlm` du *package* `spatialreg`.

```
## Construction du modèle
Modele.SEM <- errorsarlm(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                        listw=W.Rook,      # matrice de pondération spatiale
                        data = LyonIris) # dataframe
## Résultats du modèle
summary(Modele.SEM, Nagelkerke=TRUE)
```

```
Call:errorsarlm(formula = N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
  NivVieMed, data = LyonIris, listw = W.Rook)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-12.86150	-1.83161	-0.44106	0.91029	17.94924

Type: error

Coefficients: (asymptotic standard errors)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	30.544576	2.358173	12.9526	< 2e-16
Pct0_14	-0.035019	0.033393	-1.0487	0.29431
Pct_65	-0.026039	0.028970	-0.8988	0.36874
Pct_Img	-0.016770	0.026176	-0.6407	0.52175
Pct_brevet	0.023708	0.019074	1.2430	0.21388
NivVieMed	-0.146309	0.060273	-2.4274	0.01521

Lambda: 0.91138, LR test value: 613.15, p-value: < 2.22e-16

```
Asymptotic standard error: 0.01651
  z-value: 55.201, p-value: < 2.22e-16
Wald statistic: 3047.2, p-value: < 2.22e-16

Log likelihood: -1369.737 for error model
ML residual variance (sigma squared): 9.9971, (sigma: 3.1618)
Nagelkerke pseudo-R-squared: 0.78662
Number of observations: 506
Number of parameters estimated: 8
AIC: 2755.5, (AIC for lm: 3366.6)
```

Dans les résultats ci-dessus, la valeur de λ est de 0,91 (LR = 613, $p < 0,001$), traduisant une très forte autocorrélation spatiale sur le terme d'erreur.

Dépendance spatiale du modèle SEM?

Ce modèle a-t-il corrigé le problème de dépendance spatiale du modèle de régression linéaire classique? Avec une valeur du I de Moran de -0,013 ($p = 0,614$), les résidus ne sont plus spatialement autocorrélés.

```
## Autocorrélation spatiale des résidus
moran.mc(resid(Modele.SEM), W.Rook, nsim=999)
```

Monte-Carlo simulation of Moran I

```
data: resid(Modele.SEM)
weights: W.Rook
number of simulations + 1: 1000

statistic = -0.011827, observed rank = 366, p-value = 0.634
alternative hypothesis: greater
```

2.1.4 Modèle SDM : autocorrélation spatiale sur la variable dépendante et les variables indépendantes

Le modèle SDM (*Spatial Durbin Model*) est un modèle mixte qui intègre à la fois l'autocorrélation spatiale sur la variable dépendante (Wy , **effets d'entraînement ou de débordement**) et sur les variables indépendantes (WX , **externalités**). Il s'écrit alors :

$$y = Wy\rho + X\beta + WX\theta + \epsilon \quad (2.5)$$

avec :

- y , la variable dépendante.
- W , la matrice de pondération spatiale.
- Wy , la variable dépendante spatialement décalée.

- ρ , le coefficient de la variable dépendante spatialement décalée.
- X , les variables indépendantes.
- β , les coefficients des variables indépendantes.
- WX , les variables indépendantes spatiales décalées.
- θ , les coefficients des variables indépendantes spatiales décalées.
- ϵ , les résidus.

Construction du modèle SDM dans R

Le modèle SDM est construit avec la fonction `lagsarlm` du *package* `spatialreg`. Notez que le paramètre `type = "mixed"` spécifie l'utilisation d'un modèle mixte.

```
## Construction du modèle
Modele.DurbinSpatial <- lagsarlm(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                                listw = W.Rook,      # matrice de pondération spatiale
                                data = LyonIris,      # dataframe
                                type = "mixed")
## Résultats du modèle
summary(Modele.DurbinSpatial, Nagelkerke=TRUE)
```

```
Call:lagsarlm(formula = N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
              NivVieMed, data = LyonIris, listw = W.Rook, type = "mixed")
```

Residuals:

	Min	1Q	Median	3Q	Max
	-12.60922	-1.77753	-0.43909	0.99252	18.15526

Type: mixed

Coefficients: (asymptotic standard errors)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	8.1130457	2.5671301	3.1604	0.001576
Pct0_14	-0.0574046	0.0344908	-1.6643	0.096043
Pct_65	-0.0238715	0.0293647	-0.8129	0.416256
Pct_Img	0.0048364	0.0266560	0.1814	0.856025
Pct_brevet	0.0112746	0.0195259	0.5774	0.563656
NivVieMed	-0.1463876	0.0605853	-2.4162	0.015682
lag.Pct0_14	-0.1242574	0.0581170	-2.1381	0.032512
lag.Pct_65	0.0255480	0.0499646	0.5113	0.609125
lag.Pct_Img	0.1559952	0.0482138	3.2355	0.001214
lag.Pct_brevet	-0.0883930	0.0342496	-2.5809	0.009856
lag.NivVieMed	0.1032469	0.0960201	1.0753	0.282257

Rho: 0.84127, LR test value: 492.38, p-value: < 2.22e-16

Asymptotic standard error: 0.023363

z-value: 36.009, p-value: < 2.22e-16

Wald statistic: 1296.7, p-value: < 2.22e-16

```
Log likelihood: -1353.106 for mixed model
ML residual variance (sigma squared): 9.9845, (sigma: 3.1598)
Nagelkerke pseudo-R-squared: 0.8002
Number of observations: 506
Number of parameters estimated: 13
AIC: 2732.2, (AIC for lm: 3222.6)
LM test for residual autocorrelation
test value: 0.0748, p-value: 0.78447
```

Effets directs, indirects et totaux

```
# Effets directs, indirects et totaux (uniquement les coefficients)
impacts(Modele.DurbinSpatial, listw = W.Rook, R = 999)
```

```
Impact measures (mixed, exact):
```

	Direct	Indirect	Total
Pct0_14	-0.12369039	-1.02079497	-1.14448536
Pct_65	-0.02177191	0.03233406	0.01056215
Pct_Img	0.06632543	0.94692639	1.01325182
Pct_brevet	-0.01903815	-0.46681402	-0.48585217
NivVieMed	-0.15403413	-0.11775603	-0.27179016

```
# Effets directs, indirects et totaux (coefficients, valeurs de z et de p)
summary(impacts(Modele.DurbinSpatial, listw = W.Rook, R = 999), zstats = TRUE, short = TRUE)
```

```
Impact measures (mixed, exact):
```

	Direct	Indirect	Total
Pct0_14	-0.12369039	-1.02079497	-1.14448536
Pct_65	-0.02177191	0.03233406	0.01056215
Pct_Img	0.06632543	0.94692639	1.01325182
Pct_brevet	-0.01903815	-0.46681402	-0.48585217
NivVieMed	-0.15403413	-0.11775603	-0.27179016

```
=====
Simulation results ( variance matrix):
=====
Simulated standard errors
```

	Direct	Indirect	Total
Pct0_14	0.04298259	0.3447179	0.3729314
Pct_65	0.03582930	0.2818585	0.3045532
Pct_Img	0.03309232	0.2779903	0.2975143
Pct_brevet	0.02463644	0.2018981	0.2175848
NivVieMed	0.06937489	0.5020265	0.5394288

```

Simulated z-values:
```


	Direct	Indirect	Total
Pct0_14	-2.8721057	-2.93768904	-3.04647093
Pct_65	-0.6369231	0.09864112	0.01635947
Pct_Img	1.9678371	3.45000971	3.44248884
Pct_brevet	-0.7833983	-2.32027445	-2.24169615
NivVieMed	-2.2172087	-0.21728644	-0.48737133

Simulated p-values:

	Direct	Indirect	Total
Pct0_14	0.0040775	0.00330669	0.00231545
Pct_65	0.5241749	0.92142322	0.98694761
Pct_Img	0.0490868	0.00056057	0.00057639
Pct_brevet	0.4333933	0.02032604	0.02498102
NivVieMed	0.0266088	0.82798513	0.62599521

Dépendance spatiale du modèle SDM?

```
moran.mc(resid(Modele.DurbinSpatial), W.Rook, nsim=999)
```

Monte-Carlo simulation of Moran I

```
data: resid(Modele.DurbinSpatial)
weights: W.Rook
number of simulations + 1: 1000
```

```
statistic = -0.0046127, observed rank = 496, p-value = 0.504
alternative hypothesis: greater
```

2.1.5 Modèle SDEM : autocorrélation spatiale sur les variables indépendantes et sur le terme d'erreur

Le modèle SDEM (*Spatial Durbin Error Model* en anglais) est un autre modèle mixte qui intègre à la fois l'autocorrélation spatiale sur les valeurs indépendantes (WX , **externalités**) et sur le terme d'erreur ($u = \lambda Wu + \epsilon$). Il s'écrit alors :

$$y = X\beta + WX\theta + u, u = \lambda Wu + \epsilon \quad (2.6)$$

avec :

- y , la variable dépendante.
- W , la matrice de pondération spatiale.
- X , les variables indépendantes.
- β , les coefficients des variables indépendantes.
- WX , les variables dépendantes spatiales décalées.
- θ , les coefficients des variables indépendantes spatiales décalées.

- λ (prononcez *lambda*), le coefficient sur le terme d'erreur spatialement décalé.
- ϵ , les résidus.

Construction du modèle SDEM dans R

Le modèle SDEM est construit avec la fonction `errorsarlm` du *package* `spatialreg`. Notez que le paramètre `etype = "mixed"` spécifie l'utilisation d'un modèle mixte.

```
## Construction du modèle
Modele.DurbinErreur <- errorsarlm(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                                listw=W.Rook,      # matrice de pondération spatiale
                                data = LyonIris,    # dataframe
                                etype = 'emixed')

## Résultats du modèle
summary(Modele.DurbinErreur, Nagelkerke=TRUE)
```

```
Call:errorsarlm(formula = N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
  NivVieMed, data = LyonIris, listw = W.Rook, etype = "emixed")
```

Residuals:

	Min	1Q	Median	3Q	Max
	-12.99324	-1.82407	-0.45644	1.06084	18.21108

Type: error

Coefficients: (asymptotic standard errors)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	37.061010	6.501018	5.7008	1.192e-08
Pct0_14	-0.081998	0.041699	-1.9664	0.04925
Pct_65	-0.026329	0.034714	-0.7585	0.44817
Pct_Img	0.004656	0.031028	0.1501	0.88072
Pct_brevet	0.009785	0.023884	0.4097	0.68203
NivVieMed	-0.167855	0.068005	-2.4683	0.01358
lag.Pct0_14	-0.176747	0.102345	-1.7270	0.08417
lag.Pct_65	0.010533	0.089183	0.1181	0.90599
lag.Pct_Img	0.092785	0.079704	1.1641	0.24437
lag.Pct_brevet	-0.038048	0.056688	-0.6712	0.50211
lag.NivVieMed	-0.102531	0.172405	-0.5947	0.55204

Lambda: 0.8976, LR test value: 464.09, p-value: < 2.22e-16

Asymptotic standard error: 0.018242

z-value: 49.204, p-value: < 2.22e-16

Wald statistic: 2421, p-value: < 2.22e-16

Log likelihood: -1367.25 for error model

ML residual variance (sigma squared): 10.046, (sigma: 3.1696)

Nagelkerke pseudo-R-squared: 0.78871

Number of observations: 506
 Number of parameters estimated: 13
 AIC: 2760.5, (AIC for lm: 3222.6)

Effets directs, indirects et totaux

```
## Effets directs, indirects et totaux (uniquement les coefficients)
impacts(Modele.DurbinErreur, listw = W.Rook, R = 999)
```

Impact measures (SDEM, glht):

	Direct	Indirect	Total
Pct0_14	-0.081997642	-0.17674683	-0.25874447
Pct_65	-0.026329370	0.01053248	-0.01579689
Pct_Img	0.004656039	0.09278511	0.09744115
Pct_brevet	0.009784961	-0.03804813	-0.02826317
NivVieMed	-0.167855498	-0.10253070	-0.27038620

```
## Effets directs, indirects et totaux (coefficients, valeurs de z et de p)
summary(impacts(Modele.DurbinErreur, listw = W.Rook, R = 999), zstats = TRUE, short = TRUE)
```

Impact measures (SDEM, glht, n):

	Direct	Indirect	Total
Pct0_14	-0.081997642	-0.17674683	-0.25874447
Pct_65	-0.026329370	0.01053248	-0.01579689
Pct_Img	0.004656039	0.09278511	0.09744115
Pct_brevet	0.009784961	-0.03804813	-0.02826317
NivVieMed	-0.167855498	-0.10253070	-0.27038620

=====

Standard errors:

	Direct	Indirect	Total
Pct0_14	0.04169878	0.10234506	0.13146453
Pct_65	0.03471367	0.08918350	0.11192948
Pct_Img	0.03102807	0.07970364	0.09949722
Pct_brevet	0.02388387	0.05668833	0.07344175
NivVieMed	0.06800483	0.17240549	0.21172909

=====

Z-values:

	Direct	Indirect	Total
Pct0_14	-1.9664279	-1.7269698	-1.9681695
Pct_65	-0.7584727	0.1180989	-0.1411325
Pct_Img	0.1500589	1.1641264	0.9793354
Pct_brevet	0.4096890	-0.6711810	-0.3848379
NivVieMed	-2.4682880	-0.5947067	-1.2770385

p-values:

	Direct	Indirect	Total
Pct0_14	0.049249	0.084173	0.049049
Pct_65	0.448168	0.905989	0.887765
Pct_Img	0.880718	0.244373	0.327414
Pct_brevet	0.682034	0.502105	0.700358
NivVieMed	0.013576	0.552040	0.201589

Dépendance spatiale du modèle SDEM?

Avec une valeur du I de Moran de -0,010 ($p = 0,619$), les résidus du modèle SDEM ne sont pas spatialement autocorrélés.

```
moran.mc(resid(Modele.DurbinErreur), W.Rook, nsim=999)
```

Monte-Carlo simulation of Moran I

```
data: resid(Modele.DurbinErreur)
weights: W.Rook
number of simulations + 1: 1000
```

```
statistic = -0.010362, observed rank = 396, p-value = 0.604
alternative hypothesis: greater
```

2.2 Quel modèle choisir?

2.2.1 Tests du multiplicateur de Lagrange sur le modèle MCO

L'utilisation des tests du multiplicateur de Lagrange (simple et robuste) a été largement popularisée par Anselin *et al.* (1996) pour vérifier si le recours à un modèle autorégressif est nécessaire, comparativement à un modèle de régression classique (MCO). Les tests sont calculés sur le modèle MCO avec la fonction `lm.LMtests` et une matrice de pondération spatiale. Ces tests permettent aussi de choisir entre les modèles SAR et SEM. La démarche suivante peut être utilisée pour choisir un modèle :

1. Si toutes les valeurs des tests (simples et robustes) sont non significatives ($p > 0,05$), alors le recours à un modèle autorégressif n'est pas nécessaire. Nous pouvons conserver le modèle de régression classique (MCO).
2. Si les valeurs de `LMlag` ou `RLMlag` sont non significatives ($p > 0,05$), alors le recours au modèle SAR n'est pas nécessaire.
3. Si les valeurs de `LMerr` ou `RLMerr` sont non significatives ($p > 0,05$), alors le recours au modèle SEM n'est pas nécessaire.
4. Si les valeurs de `RLMlag` et `RLMerr` sont significatives ($p < 0,001$), nous choisissons le modèle ayant la plus forte statistique.

Dans les résultats ci-dessous, nous ne retenons pas le modèle SEM car la valeur de 0,740 pour le `RLMerr` n'est pas significative ($p = 0,3898$). Par contre, les valeurs de `LMlag` et de `RLMlag` (555 et 123) sont significatives, ce qui justifie la sélection du modèle SAR.

```
summary(lm.LMtests(model = Modele.MCO,
                  listw = W.Rook,
                  test = c("LMlag", "LMerr", "RLMlag", "RLMerr")))
```

Rao's score (a.k.a Lagrange multiplier) diagnostics for spatial dependence

data:

model: `lm(formula = N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed, data = LyonIris)`

test weights: `listw`

	statistic	parameter	p.value
RSlag	554.65778	1	<2e-16 ***
RSerr	432.83282	1	<2e-16 ***
adjRSlag	122.56452	1	<2e-16 ***
adjRSerr	0.73955	1	0.3898

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

2.2.2 Comparaison des modèles mixtes et non mixtes

Nous avons vu qu'il existe deux modèles mixtes (SDM et SDEM). Il convient alors de vérifier si le recours d'un modèle mixte est justifié comparativement à un modèle non mixte. Dans le code ci-dessous, nous vérifions si le modèle SDM est statistiquement différent du modèle SAR avec les fonctions `LR.Sarlm` et `anova`. Les résultats signalent un écart significatif des valeurs du log-vraisemblance (26,101, $p < 0,001$). Par conséquent, ce modèle mixte a un apport significatif.

```
## SDM et SEM sont-ils significativement différents?
LR.Sarlm(Modele.DurbinSpatial, Modele.SAR)
```

Likelihood ratio for spatial linear models

data:

Likelihood ratio = 26.101, df = 5, p-value = 8.528e-05

sample estimates:

Log likelihood of Modele.DurbinSpatial	Log likelihood of Modele.SAR
-1353.106	-1366.157

```
anova(Modele.DurbinSpatial, Modele.SAR)
```

	Model	df	AIC	logLik	Test	L.Ratio	p-value
Modele.DurbinSpatial	1	13	2732.2	-1353.1	1		
Modele.SAR	2	8	2748.3	-1366.2	2	26.101	8.5283e-05

À l'inverse, la différence entre les valeurs du log-vraisemblance des modèles SDEM et SEM n'est pas significative (4,9728, $p = 0,42$), signalant que l'utilisation d'un modèle SDEM comparativement à un modèle SEM n'est pas nécessaire.

```
## SDEM et SEM sont-ils significativement différents?
LR.Sarlm(Modele.DurbinErreur, Modele.SEM)
```

Likelihood ratio for spatial linear models

data:

Likelihood ratio = 4.9728, df = 5, p-value = 0.4192

sample estimates:

Log likelihood of Modele.DurbinErreur	Log likelihood of Modele.SEM
-1367.250	-1369.737

```
anova(Modele.DurbinErreur, Modele.SEM)
```

	Model	df	AIC	logLik	Test	L.Ratio	p-value
Modele.DurbinErreur	1	13	2760.5	-1367.2	1		
Modele.SEM	2	8	2755.5	-1369.7	2	4.9728	0.4192

2.2.3 Mesures AIC et BIC et dépendance spatiale

Le critère d'information d'Akaike (AIC) et le critère d'information bayésien (BIC) sont largement utilisés pour évaluer la qualité d'ajustement du modèle. Plus leurs valeurs sont faibles, meilleur est le modèle. Il est donc possible de comparer leurs valeurs pour les différents modèles (MCO, SLX, SAR, SEM, SDM et SDEM). Nous pouvons aussi comparer l'autocorrélation spatiale des résidus des modèles avec le I de Moran.

```
## Valeurs d'AIC et de BIC
AICs <- AIC(Modele.MCO, Modele.SLX, Modele.SAR, Modele.SEM,
            Modele.DurbinSpatial, Modele.DurbinErreur)
BICs <- BIC(Modele.MCO, Modele.SLX, Modele.SAR, Modele.SEM,
            Modele.DurbinSpatial, Modele.DurbinErreur)

## Autocorrélation spatiale des résidus
IMoran.MCO <- moran.mc(resid(Modele.MCO), W.Rook, nsim=999)
IMoran.SLX <- moran.mc(resid(Modele.SLX), W.Rook, nsim=999)
IMoran.SLM <- moran.mc(resid(Modele.SAR), W.Rook, nsim=999)
IMoran.SEM <- moran.mc(resid(Modele.SEM), W.Rook, nsim=999)
IMoran.DurbinS <- moran.mc(resid(Modele.DurbinSpatial), W.Rook, nsim=999)
IMoran.DurbinE <- moran.mc(resid(Modele.DurbinErreur), W.Rook, nsim=999)
MoranI.s <- c(IMoran.MCO$statistic, IMoran.SLX$statistic,
              IMoran.SLM$statistic, IMoran.SEM$statistic,
              IMoran.DurbinS$statistic, IMoran.DurbinE$statistic)
MoranI.p <- c(IMoran.MCO$p.value, IMoran.SLX$p.value,
```

```

      IMoran.SLM$p.value, IMoran.SEM$p.value,
      IMoran.DurbinS$p.value, IMoran.DurbinE$p.value)
## Tableau
Comparaison <- data.frame(Modele = c("MCO", "SLX", "SAR", "SEM", "Durbin S", "Durbin E"),
                          AIC = AICs$AIC,
                          BIC = BICs$BIC,
                          dl = AICs$df,
                          MoranI = MoranI.s,
                          MoranIp = MoranI.p)
Comparaison

```

	Modele	AIC	BIC	dl	MoranI	MoranIp
1	MCO	3366.626	3396.212	7	0.587312061	0.001
2	SLX	3222.594	3273.313	12	0.604660275	0.001
3	SAR	2748.314	2782.126	8	-0.014281059	0.643
4	SEM	2755.474	2789.286	8	-0.011826605	0.603
5	Durbin S	2732.212	2787.157	13	-0.004612686	0.520
6	Durbin E	2760.501	2815.446	13	-0.010361653	0.608

Quelques lignes de code suffisent pour créer deux graphiques permettant de comparer visuellement les résultats des différents modèles (figure 2.3). Les résultats démontrent que :

- Les modèles MCO et SLX ont un problème de dépendance spatiale puisque leurs résidus sont significativement autocorrélés spatialement. Par conséquent, ils ne devraient pas être retenus.
- Les modèles SDM, SAR et SEM sont les plus performants avec les valeurs d'AIC les plus faibles.

```

library(ggplot2)
library(ggpubr)
## Graphique pour l'autocorrélation spatiale
g1 <- ggplot(data=Comparaison, aes(x=reorder(Modele,MoranI), y=MoranI)) +
  geom_segment(aes(x=reorder(Modele, MoranI),
                           xend=reorder(Modele, MoranI),
                           y=0, yend=MoranI)) +
  geom_point( size=4,fill="red",shape=21)+
  xlab("Modèle") + ylab("I de Moran")+
  labs(title="Autocorrélation spatiale des résidus",
       caption="Plus la valeur du I de Moran est faible, \nmoins il y a d'autocorrélation spatiale.")
## Graphique pour les valeurs d'AIC
g2 <- ggplot(data=Comparaison, aes(x=reorder(Modele,AIC), y=AIC)) +
  geom_segment(aes(x=reorder(Modele, AIC),
                           xend=reorder(Modele, AIC),
                           y=0, yend=AIC)) +
  geom_point( size=4,fill="red",shape=21)+
  xlab("Modèle") + ylab("AIC")+
  labs(title="Qualité d'ajustement du modèle",

```

```
caption="Plus la valeur d'AIC est faible, \nplus le modèle est performant.")
## Figure avec les deux graphiques
ggarrange(g1, g2)
```

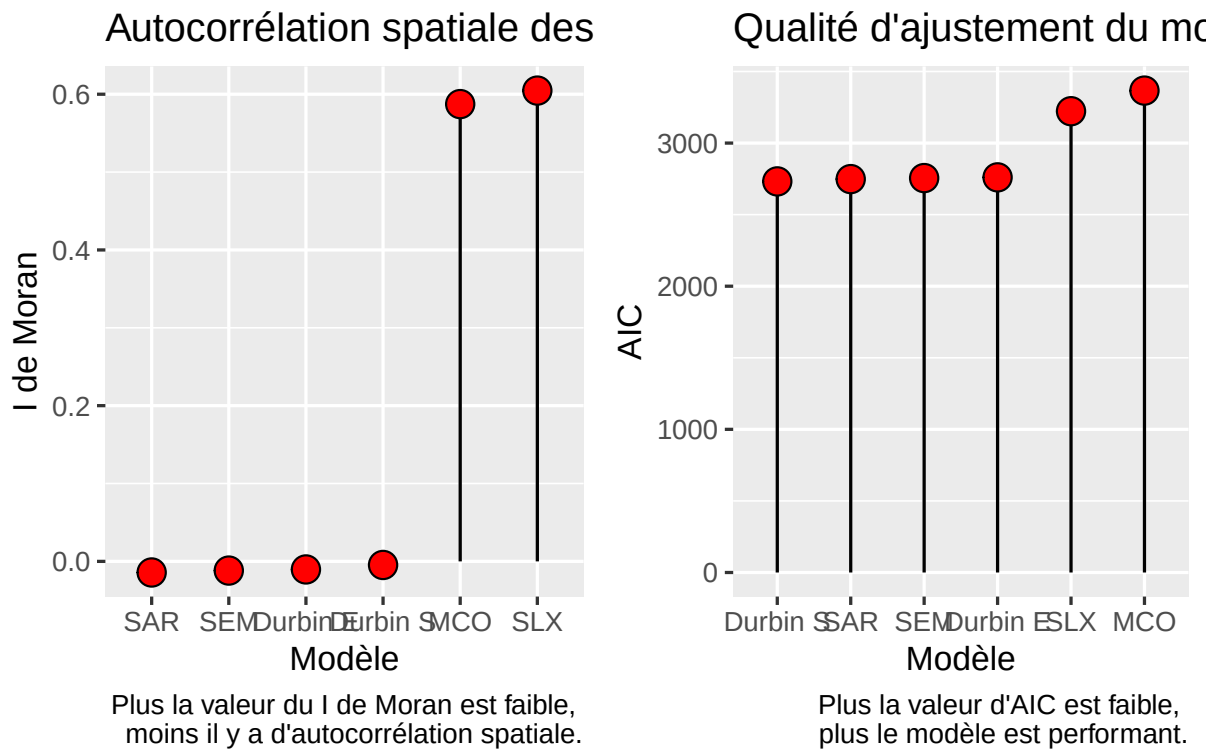


Figure 2.3: Comparaison des différents modèles

2.3 Quiz de révision

2.4 Exercices de révision

Exercice

Exercice 1. Réalisation de modèles de régression autorégressifs spatiaux

```
library(sf)
library(spatialreg)
# Matrice de contiguïté selon le partage d'un segment (Rook)
load("data/chap02/DonneesLyon.Rdata")
Rook <- poly2nb(LyonIris, queen=FALSE)
Rook <- poly2nb(LyonIris, queen=FALSE)
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")
# Modèles
formule <- "PM25 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed"
Modele.SLX <- à compléter
Modele.SAR <- à compléter
Modele.SEM <- à compléter
Modele.DurbinSpatial <- à compléter
Modele.DurbinErreur <- à compléter
```

Correction à la ?@sec-06071.

Exercice

Exercice 2. Réalisation d'un modèle GAM

```
library(sf)
library(mgcv)
load("data/chap02/DonneesLyon.Rdata")
# Ajout des coordonnées x et y
xy <- à compléter
LyonIris$X <- à compléter
LyonIris$Y <- à compléter
# Construction du modèle
formule <- "PM25 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed"
Modele.GAM2 <- gam(NO2 ~ à compléter
                  à compléter,
                  data = LyonIris)
summary(Modele.GAM2)
```

Correction à la ?@sec-06072.

 Exercice**Exercice 2.** Réalisation d'un modèle GWR

```

library(sf)
library(spgwr)
load("data/chap02/DonneesLyon.Rdata")
# Ajout des coordonnées x et y
xy <- à compléter
LyonIris$X <- à compléter
LyonIris$Y <- à compléter
# Optimisation du nombre de voisins avec le CV
formule <- "PM25 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed"
bwaCV.voisins <- gwr.sel(à compléter)
# Optimisation du nombre de voisins avec l'AIC
bwaCV.voisins <- gwr.sel(à compléter)
# Réalisation de la GWR
Modele.GWR <- gwr(à compléter)
# Affichage des résultats
Modele.GWR

```

Correction à la ?@sec-06073.

3 Modèles d'économétrie spatiale par panel

Dans ce chapitre, nous abordons une extension des modèles autorégressifs, soit les modèles spatiaux par panel qui permettent de modéliser des données spatiales longitudinales.

Objectif

Objectifs d'apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- Comprendre les différentes formulations des modèles spatiaux par panel (SLPDM, SEPDM et SEPDM).
- Assimiler les principes fondamentaux de ces différents modèles.
- Identifier le modèle spatial par panel le plus approprié (SLPDM, SEPDM et SEPDM).
- Analyser les résultats produits par ces différents modèles.
- Mettre en pratique ces modèles spatiaux par panel dans R.

Package

Liste des *packages* utilisés dans ce chapitre

- Pour importer et manipuler des fichiers géographiques :
 - `sf` pour importer et manipuler des données vectorielles.
- Pour construire des cartes et des graphiques :
 - `tmap` pour construire des cartes thématiques.
 - `ggplot2` est un *package* pour construire des graphiques.
- Pour les régressions :
 - `spdep` pour construire des matrices spatiales et calculer des mesures d'autocorrélation spatiale.
 - `plm` pour construire des modèles de régression par panel.
 - `splm` pour construire des modèles de régression spatiale par panel.

3.1 Bref retour sur les modèles en panel

3.2 Formulation des différents modèles spatiaux par panel

3.2.1 Description des différents modèles

3.2.2 Modèle SLPDM : autocorrélation sur la variable dépendante

3.2.3 Modèle SEPDM : autocorrélation sur le terme d'erreur

3.2.4 Modèle SEPDM : autocorrélation sur la variable dépendante et les variables indépendantes

3.3 Sélection du modèle spatial par panel le plus approprié

3.4 Mise en œuvre dans R

3.5 Quiz de révision

3.6 Exercices de révision

Exercice

Exercice 1. Calcul du centre moyen et de la distance standard pour les accidents
Complétez le code ci-dessous.

```
library(sf)
library(tmap)
```

Correction à la ?@sec-06031.

Exercice

Exercice 2. Calcul et cartographie de la densité des accidents dans un maillage irrégulier
Pour l'année 2021, complétez le code ci-dessous.

```
library(sf)
library(tmap)
```

Correction à la ?@sec-06032.

Exercice

Exercice 3. Calcul et cartographie de la densité des accidents dans un maillage régulier
Pour l'année 2021, complétez le code ci-dessous.

```
library(sf)
library(spatstat)
library(tmap)
```

Correction à la ?@sec-06033.

Partie 3. Régressions géographiquement pondérées

4 Régressions géographiquement pondérées classiques

La régression géographiquement pondérée (*geographically weighted regression* - **GWR**, en anglais) a été formalisée au milieu des années 1990 par Chris Brunsdon, Steward Fotheringham et Martin Charlton (1996), puis largement décrite dans un ouvrage de référence (Fotheringham, Brunsdon et Charlton 2003). Dans le cadre de ce chapitre, nous abordons uniquement la GWR classique qui s'applique à une variable dépendante continue (GWR gaussienne).

Objectif

Objectifs d'apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- Comprendre pourquoi utiliser une GWR.
- Assimiler les principes fondamentaux d'une GWR classique (distribution gaussienne).
- Analyser les résultats produits par une GWR.
- Saisir que la GWR est un outil exploratoire intéressant, comprenant toutefois plusieurs limites et critiques importantes.
- Mettre en pratique une GWR dans R.

Package

Liste des *packages* utilisés dans ce chapitre

- Pour importer et manipuler des fichiers géographiques :
 - `sf` pour importer et manipuler des données vectorielles.
 - `dplyr` pour manipuler les données.
- Pour construire des cartes et des graphiques :
 - `tmap` pour les cartes.
 - `ggplot2` et `ggpubr` pour construire des graphiques.
 - `corrplot` et `GGally` pour créer des graphiques de matrices de corrélation.
- Pour construire des modèles de régressions :
 - `spgwr` pour construire des GWR gaussienne, logistique et Poisson.
 - `spdep` pour calculer le I de Moran sur les résidus des modèles.

4.1 Principe de base

4.1.1 Pourquoi recourir à une régression géographiquement pondérée?

Dans le chapitre 2, nous avons vu que les modèles d'économétrie spatiale visent à contrôler la **dépendance spatiale** d'un modèle de régression classique (MCO), afin d'améliorer l'estimation des coefficients de régression. L'objectif des modèles de régression géographiquement pondérée est différent : ils visent à analyser les variations spatiales de la relation entre la variable dépendante et les variables indépendantes. Autrement dit, les modèles GWR visent à explorer **l'instabilité spatiale du modèle MCO** afin d'analyser localement la relation entre la variable dépendante et les variables indépendantes. Pour une description détaillée en français de la GWR, consultez Apparicio *et al.* (2007).

4.1.2 Formulation de la GWR

Contrairement à la régression linéaire classique et aux modèles spatiaux autorégressifs qui produisent une équation pour l'ensemble du tableau de données, la GWR produit une équation pour chaque unité spatiale i et ainsi, des valeurs locales de R^2 , β_0 , β_k , t de Student, etc. La résolution de cette équation de régression locale est aussi basée sur la méthode des moindres carrés et sur une matrice de pondération $W(i)$ dont les valeurs décroissent en fonction de la distance séparant les unités i et j . Autrement dit, plus j est proche de i , plus sa pondération est élevée et donc plus son « rôle » dans la détermination de l'équation de régression locale de i est important.

De la sorte, la GWR est une extension de la régression linéaire multiple classique où (u_i, v_i) représente les coordonnées géographiques du centroïde de l'unité spatiale et où les paramètres β_0 et β_k peuvent varier dans l'espace (équation 4.1).

$$y_i = \beta_0(u_i, v_i) + \sum_{j=1}^k \beta_j(u_i, v_i)x_{ij} + \epsilon_i \quad (4.1)$$

avec :

- (u_i, v_i) , les coordonnées géographiques de l'unité spatiale i .
- y_i , la variable dépendante pour l'unité spatiale i .
- $\beta_0(u_i, v_i)$, la constante pour l'unité spatiale i aux coordonnées géographiques (u_i, v_i) .
- $\beta_j(u_i, v_i)$, le coefficient de régression pour la variable x_j (avec k variables indépendantes) pour l'unité spatiale i aux coordonnées géographiques (u_i, v_i) .
- x_{ij} , la valeur de la variable indépendante x_j pour l'unité spatiale i .
- ϵ_i , le terme d'erreur pour l'unité spatiale i .

Fotheringham *et al.* (2003) proposent deux principales fonctions *kernel* pour définir la pondération $W(i)$ dans le modèle GWR : une fonction gaussienne (équation 4.2) et une fonction bicarrée (équation 4.3) où d_{ij} représente la distance euclidienne entre les points i et j et b , le rayon de zone d'influence autour du point i (*bandwidth*). Il existe une différence fondamentale entre les deux : la fonction gaussienne accorde un poids non nul à tous les points de l'espace d'étude aussi loin soient-ils, tandis que la fonction bicarrée ne tient pas compte des points distants à plus de b mètres de i , tel qu'illustré à la figure 4.1 avec une valeur fixée à 5000 mètres en guise d'exemple.

$$w_{ij} = \exp \left(-\frac{1}{2} \left(\frac{d_{ij}}{b} \right)^2 \right) \quad (4.2)$$

$$w_{ij} = \left(1 - \left(\frac{d_{ij}}{b}\right)^2\right)^2 \text{ si } d_{ij} < b, \text{ sinon } w_{ij} = 0 \quad (4.3)$$

🔗 Aller plus loin

Autres fonctions Kernel pour la GWR

Outre les fonctions Kernel gaussienne et bicarrée, il est aussi possible d'utiliser des fonctions exponentielle (équation 4.4) et tricube (équation 4.5), et Boxcar (équation 4.6) implémentées dans les fonctions `bw.gw` et `gwr.basic` du package `GWmodel`.

$$w_{ij} = \exp\left(-\frac{d_{ij}}{b}\right) \quad (4.4)$$

$$w_{ij} = \left(1 - \left(\frac{d_{ij}}{b}\right)^3\right)^3 \text{ si } d_{ij} < b, \text{ sinon } w_{ij} = 0 \quad (4.5)$$

$$w_{ij} = 1 \text{ si } d_{ij} < b, \text{ sinon } w_{ij} = 0 \quad (4.6)$$

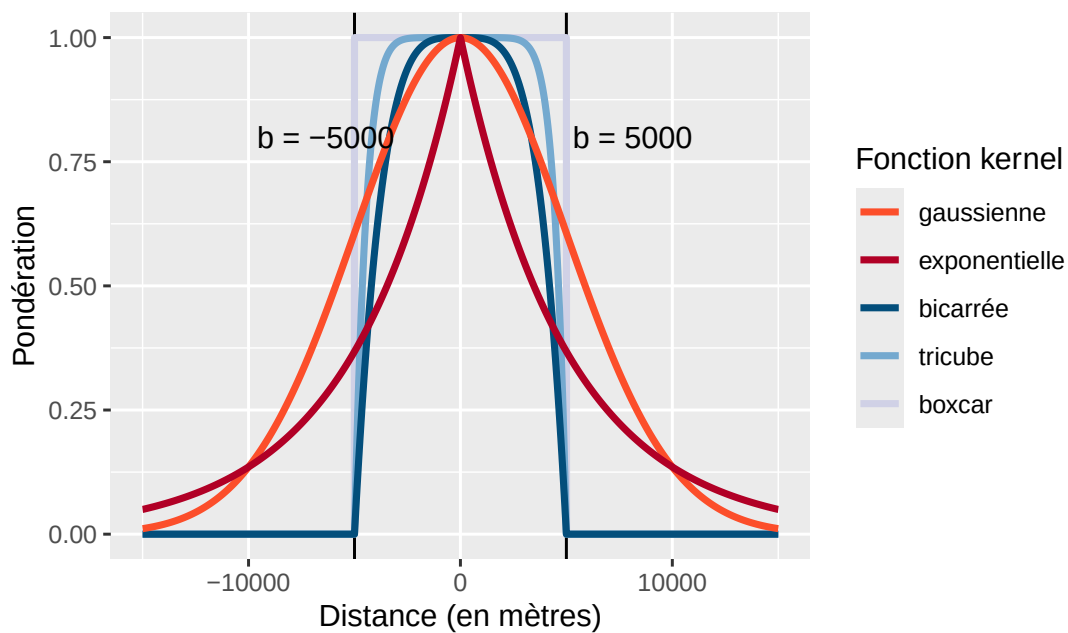


Figure 4.1: Fonctions kernel pour définir la matrice de pondération $W(i)$

Dans le modèle GWR, la valeur de b est soit fixée par la personne utilisatrice, soit optimisée avec la valeur de CV (*cross-validation*) ou celle de l'AIC. Notez qu'il est possible d'optimiser la taille de la zone d'influence à partir de la distance (euclidienne le plus souvent, mais d'autres types de distances peuvent être utilisées) ou du nombre de plus proches voisins.

4.1.3 Exemple applicatif de la GWR

Pour démontrer l'utilité de la GWR comme outil d'exploration de l'instabilité spatiale d'un modèle classique, nous utilisons le jeu de données sur Lyon (section 1.1.1). Les modèles de régression (global et GWR) sont construits avec comme variable dépendante le dioxyde d'azote (NO_2) et cinq variables sociodémographiques comme variables indépendantes (section 1.1.1).

4.1.3.1 Résultats du modèle global

Les résultats de la régression linéaire multiple (modèle global) sont présentés au tableau 4.1. Les variables sociodémographiques (variables indépendantes) introduites dans le modèle expliquent 28,3 % de la variance du dioxyde d'azote (variable dépendante). La constante et tous les coefficients de régression du modèle sont significatifs ($p < 0,001$). Toutes choses étant égales par ailleurs, le pourcentage d'immigrants est associé positivement avec le dioxyde d'azote ($\beta = 0,283$), ce qui traduit une certaine iniquité environnementale pour ce groupe de population. À l'inverse, toutes les autres variables sont associées négativement.

Tableau 4.1: Résultats du modèle global (régression linéaire multiple)

Variable	Coef.	Erreur type	t	p
Constante	49,433	2,995	16,502	0,000
Pct0_14	-0,534	0,063	-8,461	0,000
Pct_65	-0,150	0,056	-2,674	0,008
Pct_Img	0,283	0,051	5,532	0,000
Pct_brevet	-0,240	0,037	-6,451	0,000
NivVieMed	-0,316	0,102	-3,107	0,002

Nombre d'observations = 506. R carré = 0,2832. R carré ajusté = 0,276.

L'objectif de la construction d'un modèle GWR est d'examiner l'instabilité spatiale de ce modèle, en analysant comment les associations observées dans le modèle global entre les cinq variables sociodémographiques et le dioxyde d'azote varient dans l'espace.

4.1.3.2 Résultats du modèle GWR

Définition de la zone d'influence

Puisque la superficie des 506 IRIS varie considérablement (avec de petits IRIS au centre de l'agglomération et de très grands IRIS en périphérie, figure 1.1), nous préférons optimiser la zone d'influence (b) avec le nombre de plus proches voisins et non la distance euclidienne avec l'approche *cross-validation*. Le résultat final de l'optimisation du nombre de plus proches voisins est de 92.

Comparaison des modèles global et GWR

L'analyse de variance entre les résidus des deux modèles démontre que le modèle GWR améliore de façon significative la capacité prédictive du modèle de régression globale (tableau 4.2). Les valeurs de R^2 passent d'ailleurs de 0,283 pour le modèle global à 0,784 pour le modèle GWR.

Tableau 4.2: Analyse de variance entre les modèles global et GWR

	dl	Somme des carrés	Moyenne des carrés	Valeur F
Résidus du modèle global	6,00	22 346,02		
Modèle GWR	101,13	15 581,60	154,08	
Résidus du modèle GWR	398,87	6 764,42	16,96	9,09

R carré du modèle global = 0,2832. R carré du modèle GWR = 0,7838.

Cartographie des valeurs de t

Afin de décrire les variations spatiales des associations entre les variables dépendantes et indépendantes, nous cartographions les valeurs de t pour chacune des cinq variables indépendantes (figure 4.2). Pour ce faire, nous utilisons les seuils de $\pm 1,96$, $2,58$ et $3,29$, indiquant des seuils de signification à 5 %, 1 % et 0,1 %. La cartographie des valeurs de t illustre clairement les variations spatiales de l'association entre les variables indépendantes et le dioxyde d'azote.

Premièrement, le modèle global avait révélé que toutes les variables indépendantes étaient significativement associées au seuil de 0,1 %, toutes choses étant égales par ailleurs. Or, localement certaines variables ne sont plus significatives au seuil de 5 % (valeurs de t comprises entre $-1,96$ et $1,96$), soit les IRIS en gris à la figure 4.2. Par exemple, le pourcentage d'enfants de moins de 15 ans est associé négativement au dioxyde d'azote uniquement dans la partie sud de l'agglomération.

Deuxièmement, le sens de la relation, c'est-à-dire du coefficient et de sa valeur de t associé, change pour certaines variables indépendantes. Cela est particulièrement le cas de la variable *médiane du niveau de vie (milliers d'euros)* qui affiche des valeurs positives dans le centre-ville, mais des valeurs négatives en périphérie. Autrement dit, une hausse du niveau de vie des résidents est associée à une réduction du dioxyde d'azote dans les quartiers et municipalités périphériques de l'agglomération, mais à une augmentation dans les quartiers centraux.

Cartographie du nombre de variables significative et de la variable plus significatives

Pour compléter l'exploration des résultats de la GWR, il est possible de répondre à deux dernières questions avec de nouvelles analyses cartographiques (figure 4.3) :

- Quelle est la variable indépendante la plus significative localement ?
- Comment de variables indépendantes sont localement significatives (au seuil de 5 %) ?

Les deux variables les plus significatives localement sont celles relatives au niveau de vie à la population de moins de 15 ans (toutes deux 124 IRIS). Aussi, 159 modèles locaux ne présentent aucune variable significative au seuil retenu.

4.2 Mise en œuvre et analyse dans R

Il existe deux principaux *packages* pour mettre en œuvre des modèles GWR, soit *spgwr* (Bivand et Yu 2023) et *GWmodel* (Isabella Gollini et al. 2015; Binbin Lu et al. 2014). La construction d'un modèle GWR comprend les étapes suivantes :

1. Sélection de la taille de la zone d'influence (*bandwidth*) optimale.
2. Réalisation de la GWR avec la taille de la zone d'influence optimale.
3. Comparaison des modèles MCO et GWR.
4. Cartographie des résultats du modèle GWR (R^2 , coefficients, valeurs de t , etc.).

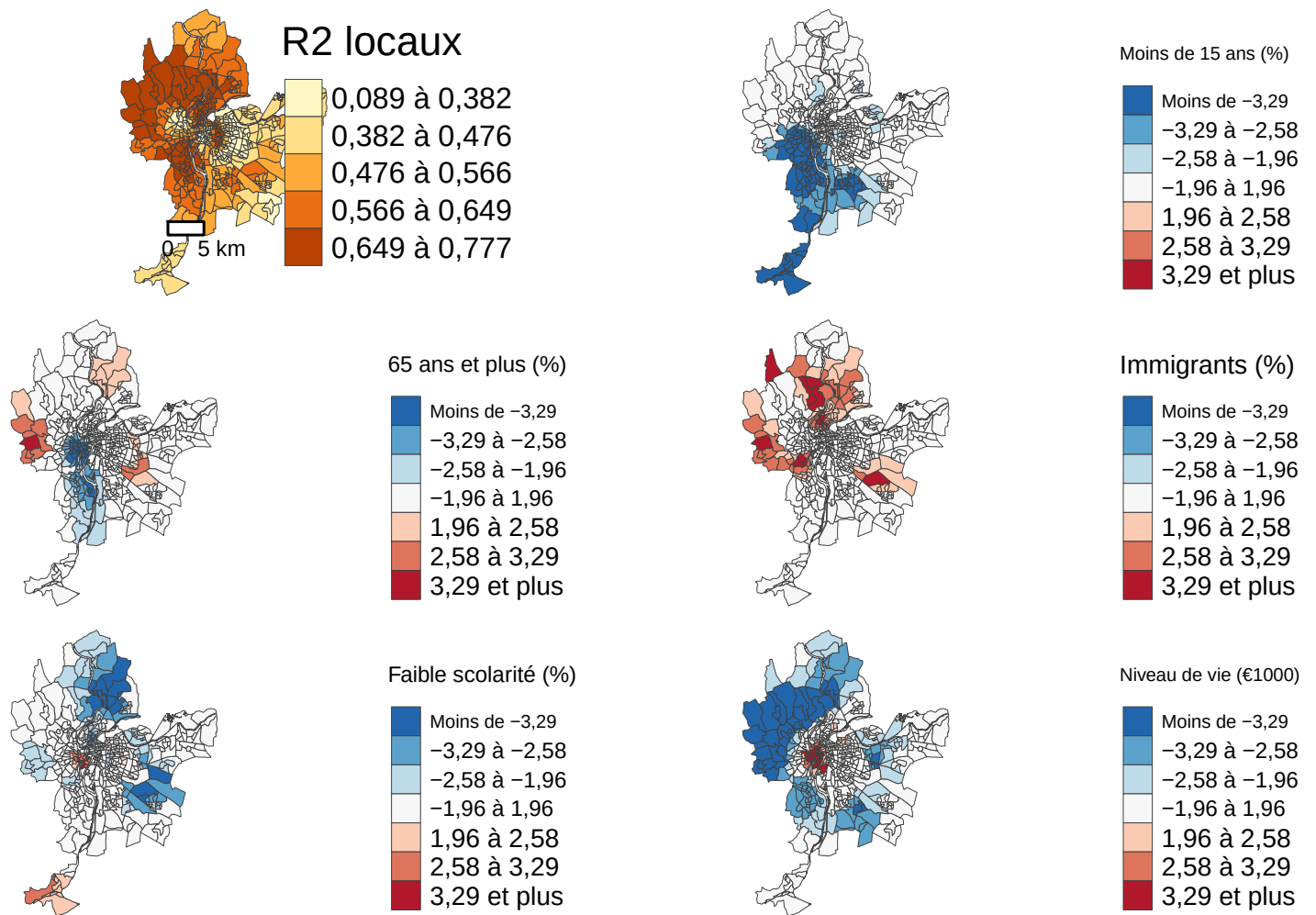


Figure 4.2: Cartographie des valeurs de t : moins de 15 ans (%)

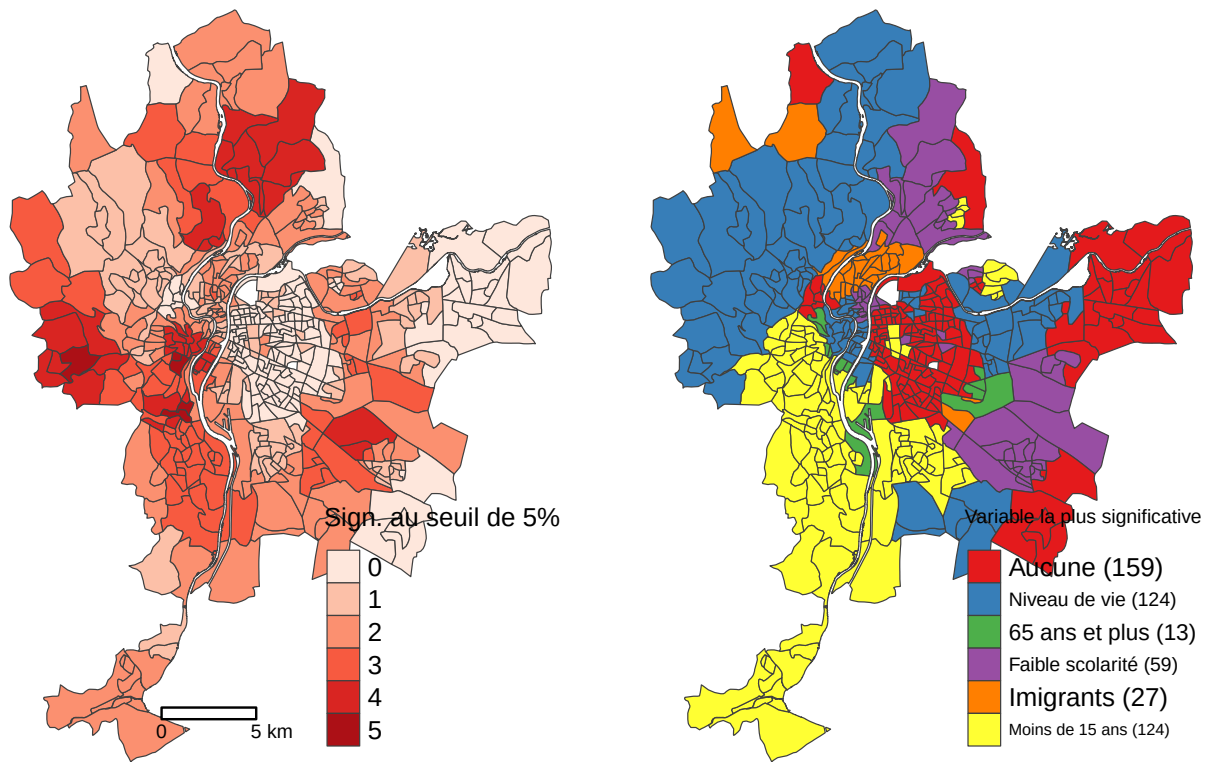


Figure 4.3: Variables indépendantes significatives du modèle GWR

4.2.1 GWR avec le package `spgwr`

4.2.1.1 Définition de la taille de la zone d'influence

La sélection de la taille de la zone d'influence optimale est réalisée avec la fonction `gwr.sel` pour laquelle :

- le paramètre `gweight` permet de spécifier une fonction *kernel* gaussienne (`gwr.gauss`) ou bicarrée (`gwr.gauss`).
- le paramètre `adapt` permet de spécifier si vous optimisez le nombre de plus proches voisins (`adapt=TRUE`) ou la distance (`adapt=FALSE`).

```
## Chargement des packages et des données
library(sf)
library(tmap)
library(spgwr)
load("data/Lyon.Rdata")
## Ajout des coordonnées X et Y dans LyonIris
xy <- st_coordinates(st_centroid(LyonIris))
LyonIris$X <- xy[,1]
LyonIris$Y <- xy[,2]
## Optimisation du nombre de voisins avec le CV
bwCV.voisins <- gwr.sel(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                        data = LyonIris,
```

```

method = "cv",          # Méthode cv ou AIC
gweight=gwr.bisquare,   # gwr.gauss ou gwr.bisquare
adapt=TRUE,
verbose = FALSE,
RMSE = TRUE,
longlat = FALSE,
coords=cbind(LyonIris$X,LyonIris$Y))

## Optimisation du nombre de voisins avec l'AIC
bwaAIC.voisins <- gwr.sel(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
  data = LyonIris,
  method = "AIC",        # Méthode cv ou AIC
  gweight=gwr.bisquare,  # gwr.gauss ou gwr.bisquare
  adapt=TRUE,            # adaptatif
  verbose = FALSE,
  RMSE = TRUE,
  longlat = FALSE,
  coords=cbind(LyonIris$X,LyonIris$Y))

## Optimisation de la distance avec le CV
bwnaCV.dist <- gwr.sel(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
  data = LyonIris,
  method = "cv",        # méthode cv ou AIC
  gweight=gwr.Gauss,    # gwr.gauss ou gwr.bisquare
  adapt=FALSE,          # non adaptatif
  verbose = FALSE,
  RMSE = TRUE,
  longlat = FALSE,
  coords=cbind(LyonIris$X,LyonIris$Y))

## Optimisation de la distance avec l'AIC
bwnaAIC.dist <- gwr.sel(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
  data = LyonIris,
  method = "AIC",       # méthode cv ou AIC
  gweight=gwr.Gauss,    # gwr.gauss ou gwr.bisquare
  adapt=FALSE,          # non adaptatif
  RMSE = TRUE,
  verbose = FALSE,
  longlat = FALSE,
  coords=cbind(LyonIris$X,LyonIris$Y))

## Affichage des résultats d'optimisation
cat("Sélection de la taille de la zone optimale (bandwidth)",
  "\n avec le nombre de plus proches voisins :",
  "\n  CV =", round(bwCV.voisins,4), "nombre de voisins =",
  round(bwCV.voisins*nrow(LyonIris)),
  "\n  AIC =", round(bwaAIC.voisins,4), "nombre de voisins =",
  round(bwaAIC.voisins*nrow(LyonIris)),
  "\nSélection de la taille de la zone optimale (bandwidth) avec la distance :",
  "\n  CV =", round(bwnaCV.dist, 0), "mètres",

```

```
"\n  AIC =", round(bwnaAIC.dist, 0), "mètres")
```

Sélection de la taille de la zone optimale (bandwidth)

avec le nombre de plus proches voisins :

CV = 0.1818 nombre de voisins = 92

AIC = 0.1067 nombre de voisins = 54

Sélection de la taille de la zone optimale (bandwidth) avec la distance :

CV = 1315 mètres

AIC = 1662 mètres

Les résultats ci-dessus montrent que le nombre de plus proches voisins pourrait être de 92 selon l'approche *cross-validation* et de 54 selon la méthode basée sur l'AIC. Si la valeur de b est basée sur la distance, elle serait alors optimale à 1315 et 1662 mètres selon les deux méthodes.

4.2.1.2 Réalisation de la GWR

Avec la fonction `gwr`, nous estimons un modèle GWR avec un *kernel* bicarré et un nombre optimisé de plus voisins selon la méthode CV, soit 92.

```
# Modèle global : régression linéaire multiple
Modele.Global <- lm(N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed, data = LyonIris)
# Modèle GWR
Modele.GWR <- gwr(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
  data = LyonIris,
  adapt=bwCV.voisins,
  gweight=gwr.bisquare,
  hatmatrix=TRUE,
  se.fit=TRUE,
  coords=cbind(LyonIris$X,LyonIris$Y),
  longlat=FALSE)
```

Le code ci-dessous permet de renvoyer les statistiques univariées des coefficients des 506 régressions locales, réalisées pour chacune des 506 entités spatiales (IRIS), et les statistiques d'ajustement du modèle du modèle GWR (AIC, R^2 quasi-global, etc.).

```
Modele.GWR
```

Call:

```
gwr(formula = N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
  NivVieMed, data = LyonIris, coords = cbind(LyonIris$X, LyonIris$Y),
  gweight = gwr.bisquare, adapt = bwCV.voisins, hatmatrix = TRUE,
  longlat = FALSE, se.fit = TRUE)
```

Kernel function: `gwr.bisquare`

Adaptive quantile: 0.1818192 (about 92 of 506 data points)

Summary of GWR coefficient estimates at data points:

	Min.	1st Qu.	Median	3rd Qu.	Max.	Global
X.Intercept.	12.429518	30.249511	38.619342	48.038863	60.098584	49.4330
Pct0_14	-1.094802	-0.360556	-0.215643	-0.047687	0.382801	-0.5335
Pct_65	-0.715331	-0.158253	-0.031353	0.076086	0.464992	-0.1505
Pct_Img	-0.331892	-0.049146	0.077177	0.240755	0.670433	0.2829
Pct_brevet	-0.655221	-0.221655	-0.084954	0.047835	0.598456	-0.2400
NivVieMed	-1.140895	-0.560649	-0.214717	0.193768	1.311228	-0.3162

Number of data points: 506
 Effective number of parameters (residual: 2traceS - traceS'S): 107.1278
 Effective degrees of freedom (residual: 2traceS - traceS'S): 398.8722
 Sigma (residual: 2traceS - traceS'S): 4.118116
 Effective number of parameters (model: traceS): 81.53263
 Effective degrees of freedom (model: traceS): 424.4674
 Sigma (model: traceS): 3.992025
 Sigma (ML): 3.656286
 AICc (GWR p. 61, eq 2.33; p. 96, eq. 4.21): 2945.674
 AIC (GWR p. 96, eq. 4.22): 2829.504
 Residual sum of squares: 6764.424
 Quasi-global R2: 0.783008

4.2.1.3 Comparaison des modèles MCO et GWR

Le R^2 du modèle GWR est bien supérieur à celui du modèle classique MCO (0,783 contre 0,283).

```
R2.global <- summary(Modele.Global)$r.squared
rss <- sum((Modele.GWR$lm$y - Modele.GWR$SDF$pred)^2)
tss <- sum((Modele.GWR$lm$y - mean(Modele.GWR$SDF$pred))^2)
R2.GWRquasiglobal <- 1 - (rss / tss)
cat("R2 global (LM) = ", round(R2.global, 3),
    "\nR2 quasi-global (GWR) : ", round(R2.GWRquasiglobal, 3))
```

R2 global (LM) = 0.283

R2 quasi-global (GWR) : 0.784

Fotheringham *et al.* (2003) proposent plusieurs tests pour comparer les modèles GWR et classique qui sont implémentés dans le *package* *spgwr* : fonctions `BFC99.gwr.test(Modele.GWR)`, `BFC02.gwr.test(Modele.GWR)`, `LMZ.F1GWR.test(Modele.GWR)`, `LMZ.F2GWR.test(Modele.GWR)`. Si les valeurs de p de ces tests sont inférieures à 0,05, alors le modèle GWR améliore de façon significative la capacité prédictive du modèle de régression globale, ce que confirment les résultats ci-dessous.

```
anova(Modele.GWR)
```

Analysis of Variance Table

	Df	Sum Sq	Mean Sq	F value
OLS Residuals	6.00	22346.0		
GWR Improvement	101.13	15581.6	154.078	
GWR Residuals	398.87	6764.4	16.959	9.0854

`BFC99.gwr.test(Modele.GWR)`

Brunsdon, Fotheringham & Charlton (1999) ANOVA

data: Modele.GWR
 F = 9.0854, df1 = 350.93, df2 = 430.81, p-value < 2.2e-16
 alternative hypothesis: greater
 sample estimates:
 SS GWR improvement SS GWR residuals
 15581.596 6764.424

`BFC02.gwr.test(Modele.GWR)`

Brunsdon, Fotheringham & Charlton (2002, pp. 91-2) ANOVA

data: Modele.GWR
 F = 3.3035, df1 = 500.00, df2 = 398.87, p-value < 2.2e-16
 alternative hypothesis: greater
 sample estimates:
 SS OLS residuals SS GWR residuals
 22346.021 6764.424

`LMZ.F1GWR.test(Modele.GWR)`

Leung et al. (2000) F(1) test

data: Modele.GWR
 F = 0.37946, df1 = 430.81, df2 = 500.00, p-value < 2.2e-16
 alternative hypothesis: less
 sample estimates:
 SS OLS residuals SS GWR residuals
 22346.021 6764.424

`LMZ.F2GWR.test(Modele.GWR)`

Leung et al. (2000) F(2) test

```
data: Modele.GWR
F = 3.4476, df1 = 142.92, df2 = 500.00, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:
  SS OLS residuals SS GWR improvement
      22346.02      15581.60
```

Un autre test (`LMZ.F3GWR.test`) permet de répondre à la question suivante : est-ce que les coefficients de régression du modèle GWR varient spatialement de façon significative? Les résultats ci-dessous démontrent que c'est le cas pour toutes les variables indépendantes et la constante ($p < 0,001$).

```
LMZ.F3GWR.test(Modele.GWR)
```

Leung et al. (2000) F(3) test

	F statistic	Numerator d.f.	Denominator d.f.	Pr(>)
(Intercept)	2.2771	134.3880	430.81	1.629e-10 ***
Pct0_14	2.7767	141.7244	430.81	5.636e-16 ***
Pct_65	2.0918	169.0472	430.81	8.399e-10 ***
Pct_Img	1.9486	106.4400	430.81	1.550e-06 ***
Pct_brevet	2.4445	121.2830	430.81	1.629e-11 ***
NivVieMed	3.6926	138.8118	430.81	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Pour comparer **la dépendance spatiale** des deux modèles (MCO versus GWR), nous calculons le I de Moran sur leurs résidus avec une matrice de contiguïté standardisée selon le partage d'un segment (*Rook*). Avec une valeur du I de Moran de 0,587 ($p < 0,001$), les résidus du modèle global (MCO) sont fortement autocorrélés spatialement, traduisant ainsi un problème de dépendance spatiale du modèle. Bien qu'elle soit toujours significative, l'autocorrélation spatiale des résidus du modèle GWR est bien plus faible (0,275, $p < 0,001$). La cartographie des résidus des deux modèles à la figure 4.4 corrobore ces résultats.

```
library(spdep)
## Résidus des modèles global (MCO) et GWR
LyonIris$MCO.Residus <- Modele.Global$residuals
LyonIris$GWR.Residus <- Modele.GWR$SDF$pred - Modele.GWR$lm$y
## Matrice de contiguïté
Rook <- poly2nb(LyonIris, queen=TRUE)
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")
# I de Moran sur les résidus du modèle global (MCO)
moran.test(LyonIris$MCO.Residus, W.Rook, randomisation = FALSE)
```

Moran I test under normality

data: LyonIris\$MCO.Residus
weights: W.Rook

Moran I statistic standard deviate = 21.275, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:

Moran I statistic	Expectation	Variance
0.5775768402	-0.0019801980	0.0007420591

```
# I de Moran sur les résidus du modèle GWR (MCO)
moran.test(LyonIris$GWR.Residus, W.Rook, randomisation = FALSE)
```

Moran I test under normality

data: LyonIris\$GWR.Residus
weights: W.Rook

Moran I statistic standard deviate = 10.171, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:

Moran I statistic	Expectation	Variance
0.2750946577	-0.0019801980	0.0007420591

```
## Cartographie des résidus du modèle MCO
tmap_mode("plot")
Carte1 <- tm_shape(LyonIris)+
  tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="MCO.Residus", n = 6, style = "pretty",
    legend.format = list(text.separator = "à",
      decimal.mark = ","),
    midpoint = 0,
    palette = "-RdBu",
    title = "MCO") +
  tm_layout(frame=FALSE)
## Cartographie des résidus du modèle GWR
Carte2 <- tm_shape(LyonIris)+
  tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.Residus", n = 6, style = "pretty",
    legend.format = list(text.separator = "à",
      decimal.mark = ","),
    midpoint = 0,
    palette = "-RdBu",
```

```

title = "GWR") +
tm_layout(frame=FALSE) +
tm_scale_bar(breaks = c(0,5))
tmap_arrange(Carte1, Carte2, nrow = 1)

```

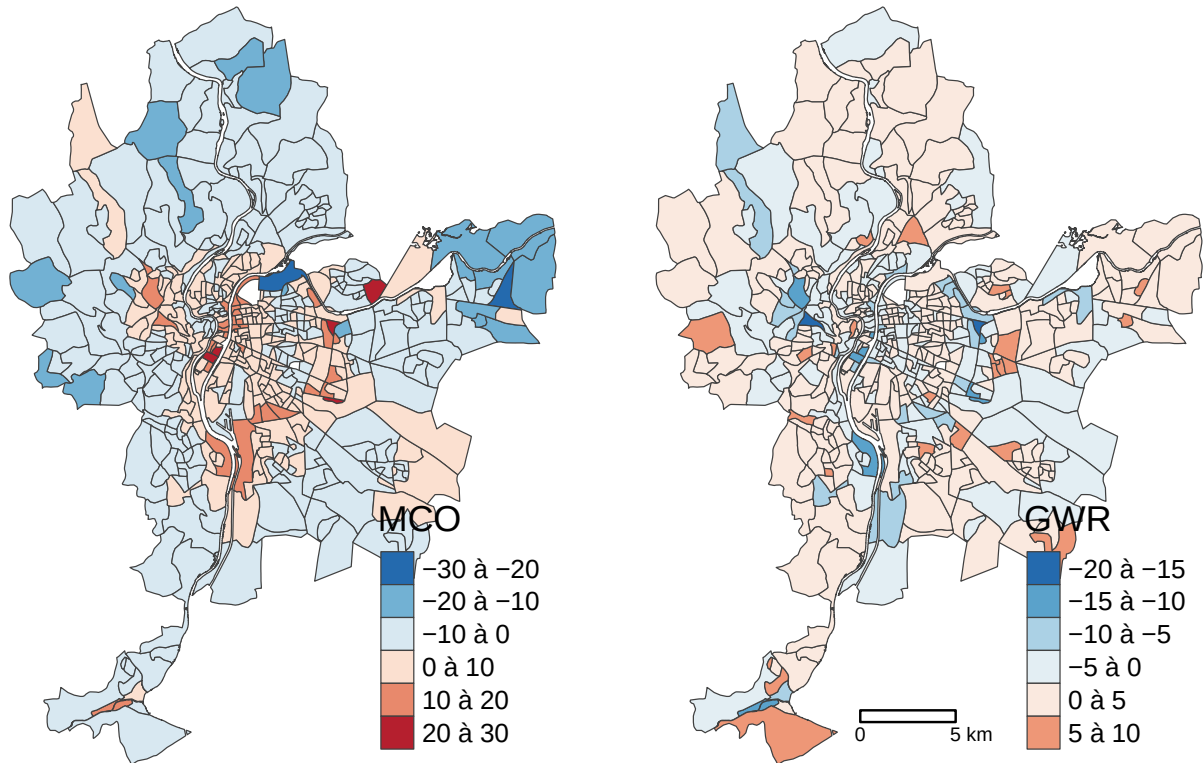


Figure 4.4: Cartographie des résidus des modèles MCO et GWR

4.2.1.4 Cartographie des résultats du modèle GWR

Dans un premier temps, nous ajoutons les valeurs locales des R^2 , des coefficients de régression et des valeurs de t dans la couche `sf`. Notez que les résultats locaux de la GWR sont stockés dans l'objet `Modele.GWR$SDF`.

```

## Récupération du R carré local
LyonIris$GWR.R2 <- Modele.GWR$SDF$localR2
## Récupération des coefficients de régression et calcul des valeurs de t locales
names(Modele.GWR$SDF)

```

[1] "sum.w"	"(Intercept)"	"Pct0_14"
[4] "Pct_65"	"Pct_Img"	"Pct_brevet"
[7] "NivVieMed"	"(Intercept)_se"	"Pct0_14_se"
[10] "Pct_65_se"	"Pct_Img_se"	"Pct_brevet_se"
[13] "NivVieMed_se"	"gwr.e"	"pred"

```
[16] "pred.se"          "localR2"          "(Intercept)_se_EDF"
[19] "Pct0_14_se_EDF"   "Pct_65_se_EDF"    "Pct_Img_se_EDF"
[22] "Pct_brevet_se_EDF" "NivVieMed_se_EDF" "pred.se"
```

```
VarsIndep <- c("Pct0_14", "Pct_65", "Pct_Img", "Pct_brevet", "NivVieMed")
for(e in VarsIndep){
  # Nom des nouvelles variables
  var.coef <- paste0("GWR.", "B_", e)
  var.t     <- paste0("GWR.", "T_", e)
  # Récupération des coefficients pour les variables indépendantes
  LyonIris[[var.coef]] <- Modele.GWR$SDF[[e]]
  # Calcul des valeurs de t pour les variables indépendantes
  LyonIris[[var.t]]    <- Modele.GWR$SDF[[e]] / Modele.GWR$SDF[[paste0(e, "_se")]]
}
```

Cartographie et analyse des R^2 locaux

Les deux lignes de code ci-dessous permettent d'obtenir un résumé sommaire des statistiques descriptives (minimum, quartiles 1 à 3, moyenne et maximum) des valeurs locales de R^2 , mais aussi de constater qu'elles varient de 0,19 à 0,74 pour 95 % des 506 observations (IRIS).

```
summary(LyonIris$GWR.R2)
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.0888  0.4205  0.5281  0.5081  0.6322  0.7774
```

```
round(quantile(LyonIris$GWR.R2, probs = c(0.025, 0.975)), 4)
```

```
      2.5% 97.5%
0.1876 0.7390
```

Le code ci-dessous permet ensuite de cartographier les R^2 locaux de la GWR (figure 4.5).

```
library(tmap)
# Paramètres pour la légende
legende.parametres <- list(text.separator = "à",
                           decimal.mark = ",")

tm_shape(LyonIris)+
  tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.R2",
          palette="YlOrBr",
          n=5, style="quantile",
          legend.format = legende.parametres,
```

```
    title = "R2 locaux")+  
tm_layout(frame=FALSE)+  
tm_scale_bar(breaks=c(0,5))
```

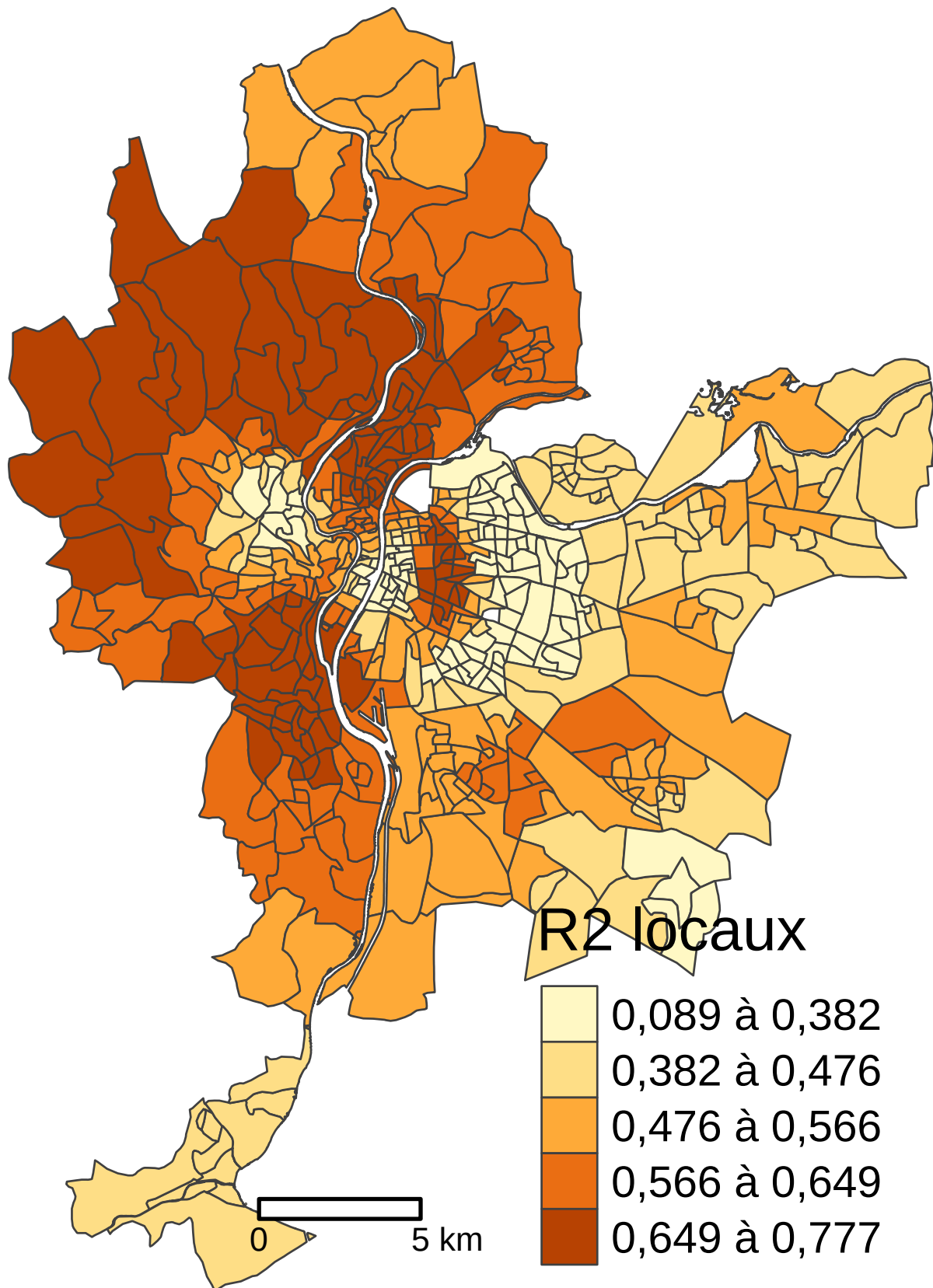


Figure 4.5: Cartographie des R carrés locaux de la GWR

Cartographie des coefficients de régression locaux

Le code ci-dessous permet ensuite de cartographier les coefficients locaux de la GWR (figure 4.6).

```
# Paramètres pour la légende
legende.parametres <- list(text.separator = "à",
                           decimal.mark = ",",
                           big.mark = " ")

Carte1 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.B_Pct0_14", palette="YlOrBr", n=4, style="pretty",
          legend.format = legende.parametres,
          title = "Moins de 15 ans (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte2 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.B_Pct_65", palette="YlOrBr", n=4, style="pretty",
          legend.format = legende.parametres,
          title = "65 ans et plus (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte3 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.B_Pct_Img", palette="YlOrBr", n=4, style="pretty",
          legend.format = legende.parametres,
          title = "Immigrants (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte4 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.B_Pct_brevet", palette="YlOrBr", n=4, style="pretty",
          legend.format = legende.parametres,
          title = "Faible scolarité (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte5 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.B_NivVieMed", palette="YlOrBr", n=4, style="pretty",
          legend.format = legende.parametres,
          title = "Niveau de vie (€1000)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)+
  tm_scale_bar(breaks=c(0,5))

tmap_arrange(Carte1, Carte2, Carte3, Carte4, Carte5, ncol = 2, nrow=3)
```

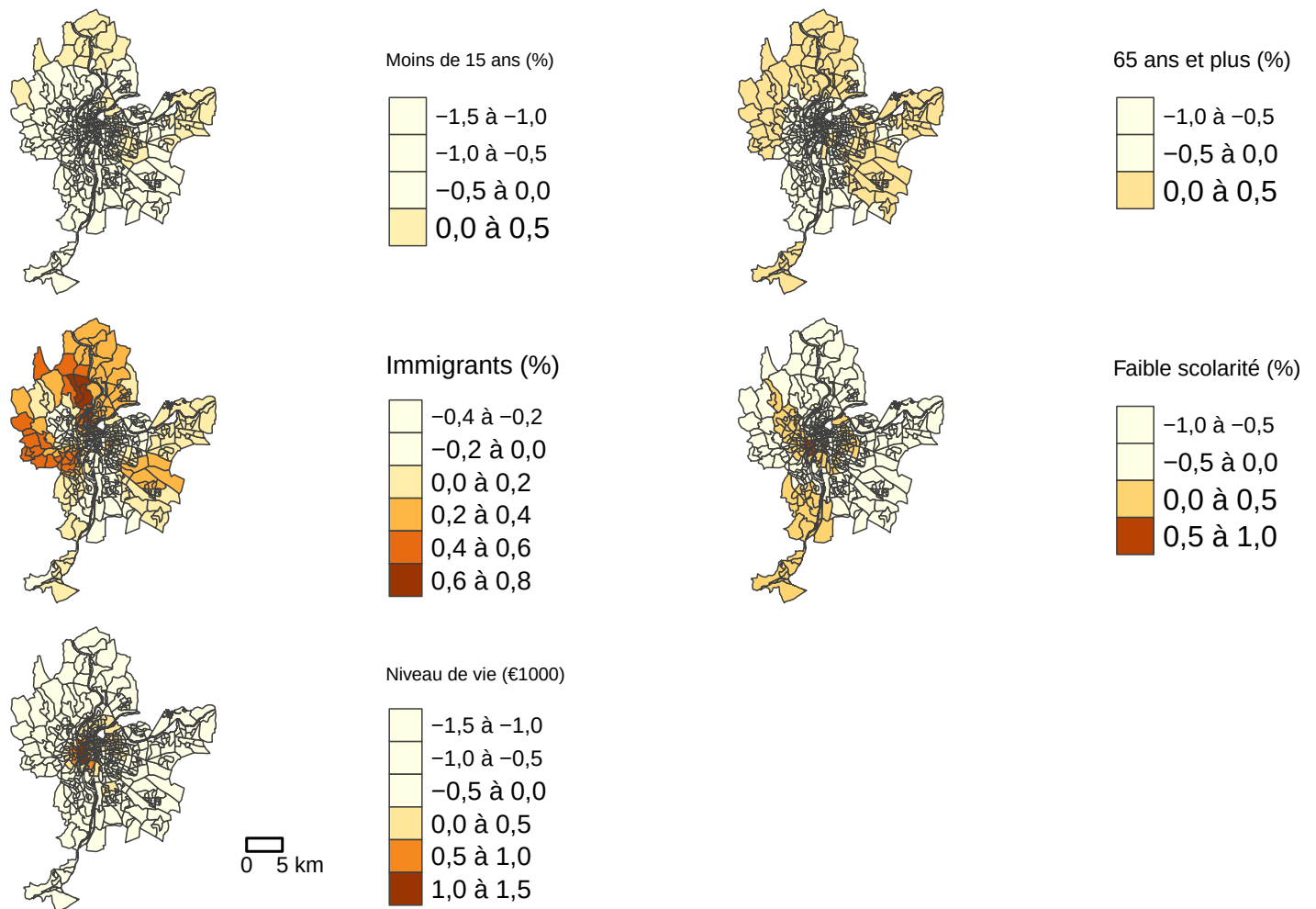



Figure 4.6: Cartographie des coefficients de régression de la GWR

Cartographie des valeurs de t locales

Pour cartographier les valeurs de t , nous utilisons les seuils de $\pm 1,96$, $2,58$ et $3,29$, indiquant des seuils de signification à 5 %, 1 % et 0,1 % (figure 4.7).

```
# Paramètres pour la légende
legende.parametres <- list(text.separator = "à",
                           text.less.than = "Moins de",
                           text.or.more = "et plus",
                           decimal.mark = ",",
                           big.mark = " ")

classes.intervalles = c(-Inf, -3.29, -2.58, -1.96, 1.96, 2.58, 3.29, Inf)
Cartel <- tm_shape(LyonIris) + tm_borders(col="gray25", lwd=.5) +
  tm_fill(col="GWR.T_Pct0_14", palette="-RdBu",
          midpoint = NA,
```

```

        breaks = classes.intervalles,
        legend.format = legende.parametres,
        title = "Moins de 15 ans (%)")+
    tm_layout(frame=FALSE, legend.outside = TRUE)

Carte2 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
    tm_fill(col="GWR.T_Pct_65", palette="-RdBu",
            midpoint = NA,
            breaks = classes.intervalles,
            legend.format = legende.parametres,
            title = "65 ans et plus (%)")+
    tm_layout(frame=FALSE, legend.outside = TRUE)

Carte3 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
    tm_fill(col="GWR.T_Pct_Img", palette="-RdBu",
            midpoint = NA,
            breaks = classes.intervalles,
            legend.format = legende.parametres,
            title = "Immigrants (%)")+
    tm_layout(frame=FALSE, legend.outside = TRUE)

Carte4 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
    tm_fill(col="GWR.T_Pct_brevet", palette="-RdBu",
            midpoint = NA,
            breaks = classes.intervalles,
            legend.format = legende.parametres,
            title = "Faible scolarité (%)")+
    tm_layout(frame=FALSE, legend.outside = TRUE)

Carte5 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
    tm_fill(col="GWR.T_NivVieMed", palette="-RdBu",
            midpoint = NA,
            breaks = classes.intervalles,
            legend.format = legende.parametres,
            title = "Niveau de vie (€1000)")+
    tm_layout(frame=FALSE, legend.outside = TRUE)+
    tm_scale_bar(breaks=c(0,5))

tmap_arrange(Carte1, Carte2, Carte3, Carte4, Carte5, ncol = 2, nrow=3)

```

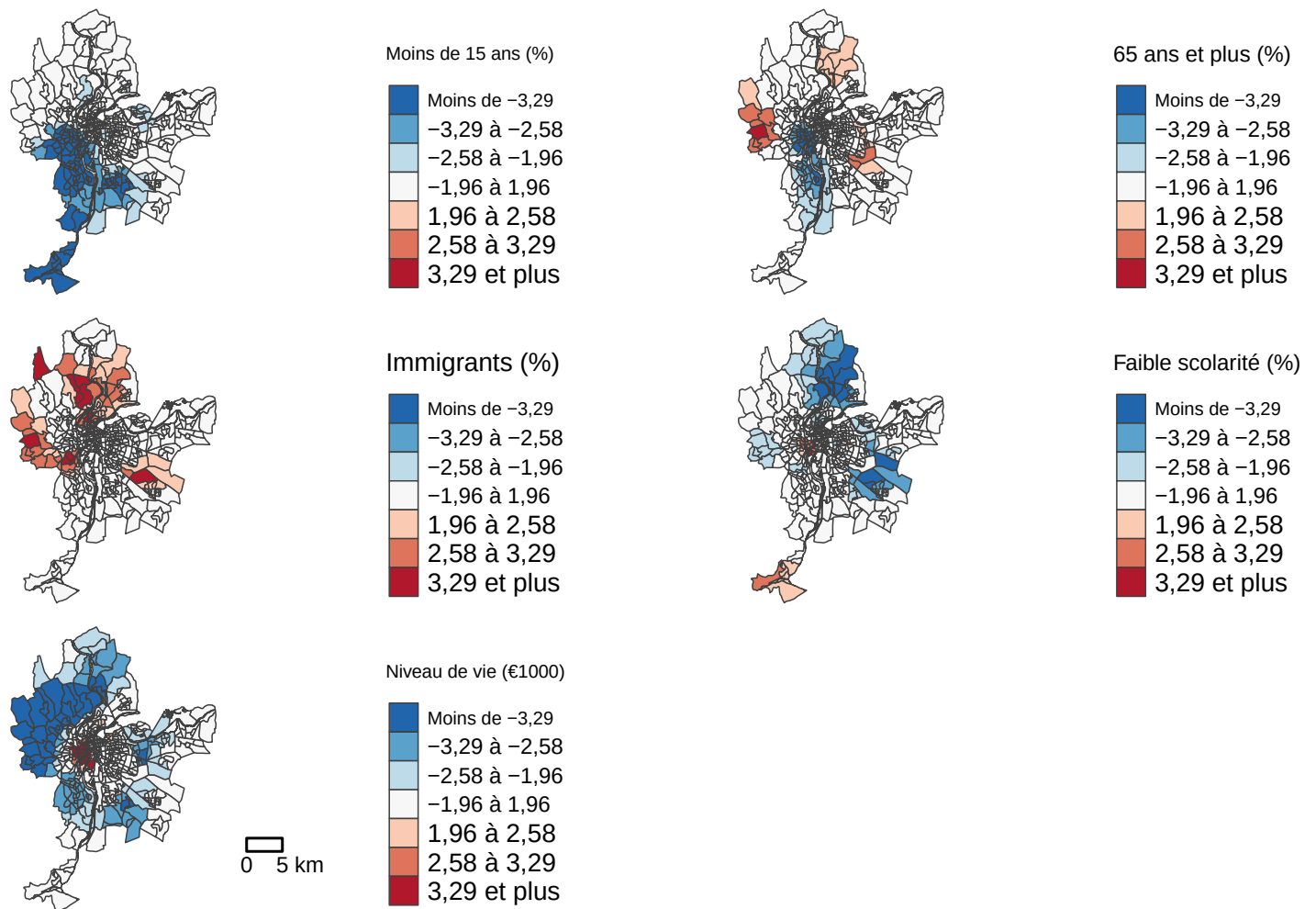


Figure 4.7: Cartographie des valeurs de t de la GWR

Cartographie du nombre de variables significatives

Nous pouvons aussi cartographier le nombre de variables localement significatives aux seuils de 5 % et 1 %.

```
## Identifier la variable plus significative avec les valeurs de t
VarsT <- paste0("GWR.T_", c("Pct0_14", "Pct_65", "Pct_Img", "Pct_brevet", "NivVieMed"))
Lyon.df <- st_drop_geometry(LyonIris)
Lyon.df <- abs(Lyon.df[,VarsT])
PlusSign <- VarsT[apply(Lyon.df[VarsT], 1, which.max)]
PlusSign <- substr(PlusSign, 7, nchar(PlusSign))
MaxAbsTvalue <- apply(Lyon.df[VarsT], 1, max)
PlusSign <- ifelse(MaxAbsTvalue < 1.96, "Aucune", PlusSign)
## Nombre de variables significatives au seuil de 5%, soit abs(t) = 1,96
LyonIris$NbSignif_1.96 <- as.factor(rowSums(Lyon.df > 1.96))
LyonIris$NbSignif_2.58 <- as.factor(rowSums(Lyon.df > 2.58))
LyonIris$PlusSign <- as.factor(PlusSign)
```

```
## Cartographie
Carte1 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="NbSignif_1.96", palette="Reds",
    title = "Sign. au seuil de 5%")+
  tm_layout(frame=FALSE)+ tm_scale_bar(breaks=c(0,5))
Carte2 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="NbSignif_2.58", palette="Reds",
    title = "Sign. au seuil de 1%")+
  tm_layout(frame=FALSE)
tmap_arrange(Carte1, Carte2, ncol=2, nrow=1)
```

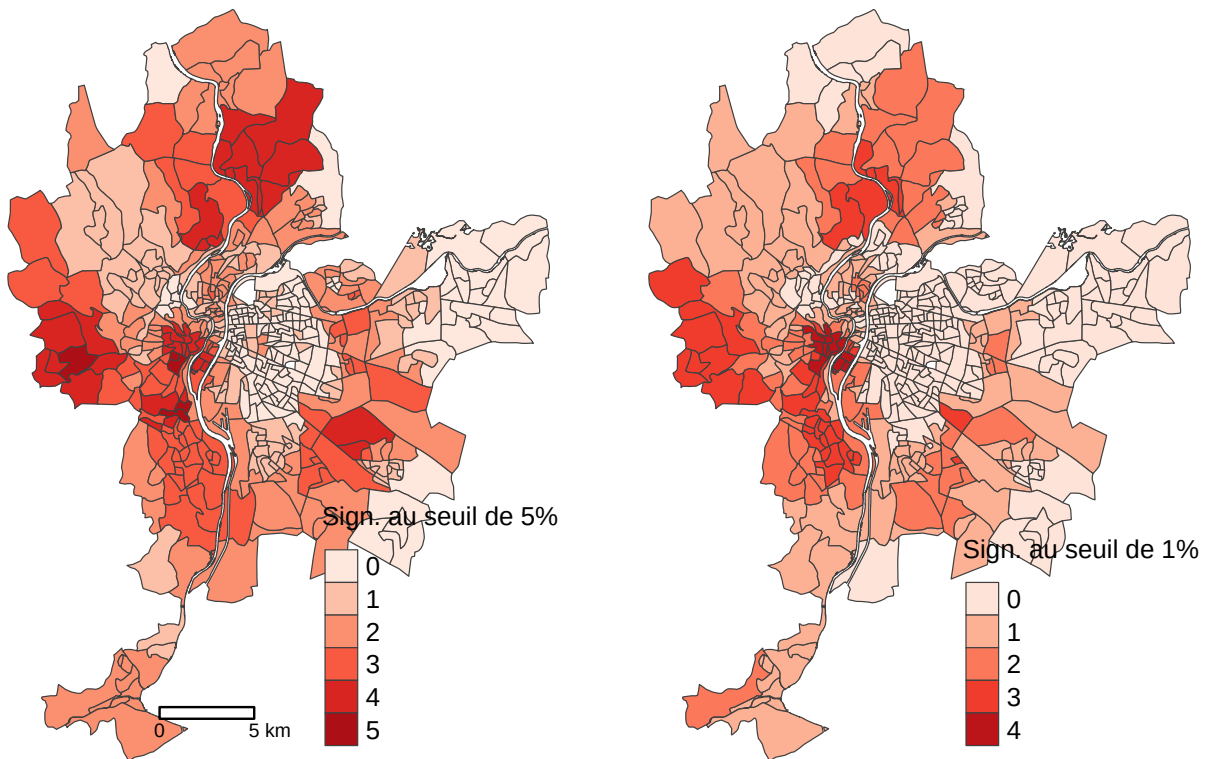


Figure 4.8: Nombre de variables significatives aux seuils de 5% et 1%

Cartographie de la variable la plus significative avec la valeur de t

Finalement, le code ci-dessous permet de repérer la variable la plus significative au seuil de 5 %, c'est-à-dire avec la plus forte valeur absolue pour la valeur de t .

```
tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="PlusSign", palette="Set1",
    title = "Variable la plus significative")+
  tm_layout(frame=FALSE)+ tm_scale_bar(breaks=c(0,5))
```

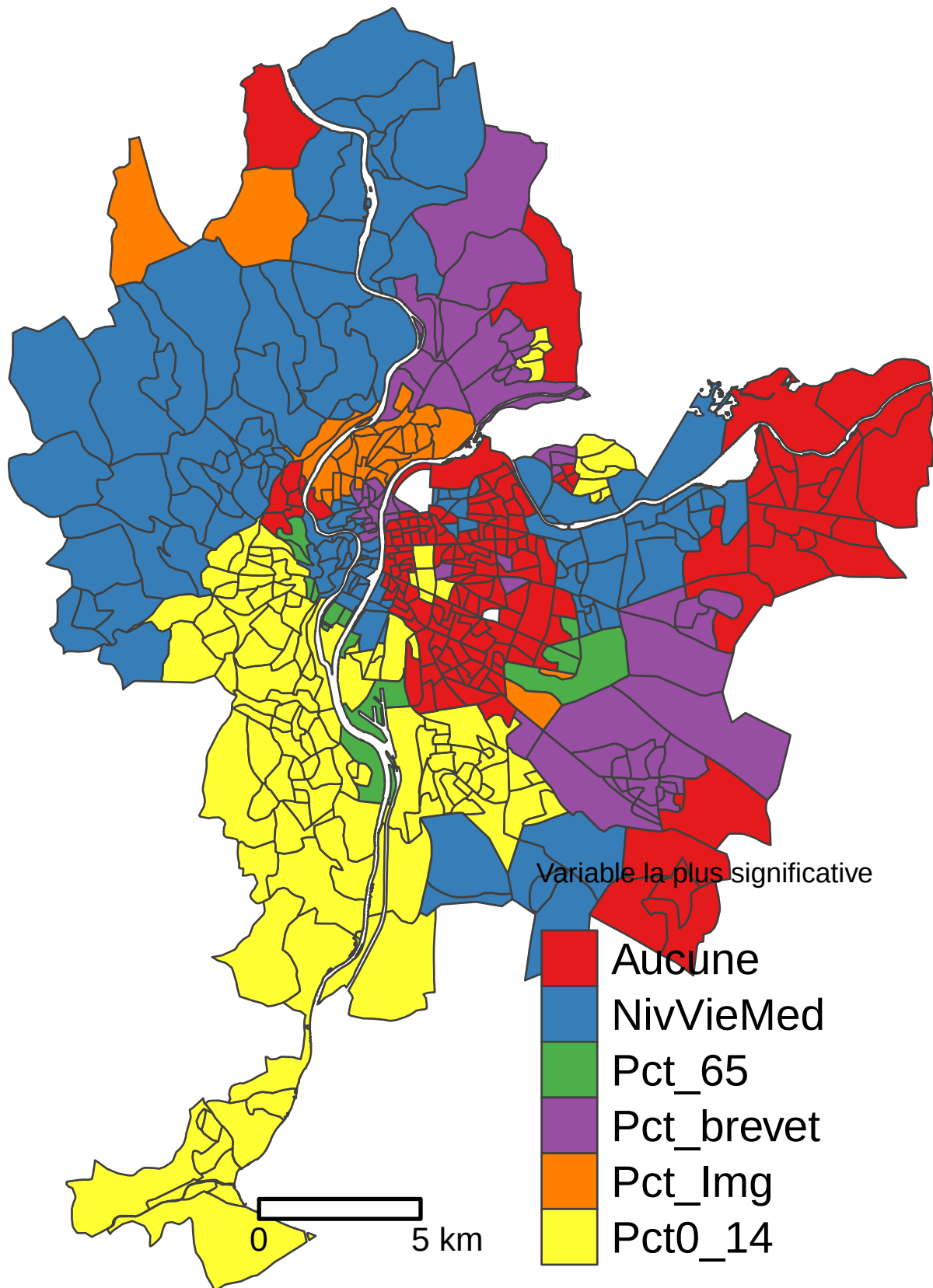


Figure 4.9: Variable indépendante la plus significative au seuil de 5 %

4.2.2 GWR avec le package `GWmodel`

La construction d'une régression GWR est certainement plus simple (moins verbeuse) avec les fonctions `bw.gwr` et `gwr.basic` du package `GWmodel`.

4.2.2.1 Définition de la taille de la zone d'influence

La fonction `bw.gwr` permet de trouver la valeur de la zone d'influence (b).

```
library(GWmodel)
load("data/Lyon.Rdata")
# Construction de la matrice de distance
xy <- st_coordinates(st_centroid(LyonIris))
MatriceDistance <- gw.dist(dp.locat=xy, p = 2, longlat=FALSE)
## Optimisation du nombre de voisins avec le CV
bwCV.voisins <- bw.gwr(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                      data = LyonIris,
                      dMat = MatriceDistance,
                      approach="CV",
                      kernel="bisquare",
                      adaptive=TRUE,
                      longlat=FALSE)
```

```
Adaptive bandwidth: 320 CV score: 17072.1
Adaptive bandwidth: 206 CV score: 14142.57
Adaptive bandwidth: 134 CV score: 12329.97
Adaptive bandwidth: 91 CV score: 11919.32
Adaptive bandwidth: 63 CV score: 12156.26
Adaptive bandwidth: 107 CV score: 11981.63
Adaptive bandwidth: 79 CV score: 12001.41
Adaptive bandwidth: 96 CV score: 11900.81
Adaptive bandwidth: 101 CV score: 11912.07
Adaptive bandwidth: 94 CV score: 11919.6
Adaptive bandwidth: 98 CV score: 11911.68
Adaptive bandwidth: 95 CV score: 11911.87
Adaptive bandwidth: 97 CV score: 11899.93
Adaptive bandwidth: 97 CV score: 11899.93
```

```
## Optimisation du nombre de voisins avec l'AIC
bwAIC.voisins <- bw.gwr(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                      data = LyonIris,
                      dMat = MatriceDistance,
                      approach="AIC",
                      kernel="bisquare",
                      adaptive=TRUE,
                      longlat=FALSE)
```

```
Adaptive bandwidth (number of nearest neighbours): 320 AICc value: 3181.763
Adaptive bandwidth (number of nearest neighbours): 206 AICc value: 3072.458
Adaptive bandwidth (number of nearest neighbours): 134 AICc value: 2989.605
Adaptive bandwidth (number of nearest neighbours): 91 AICc value: 2943.187
Adaptive bandwidth (number of nearest neighbours): 63 AICc value: 2925.002
Adaptive bandwidth (number of nearest neighbours): 47 AICc value: 2927.612
Adaptive bandwidth (number of nearest neighbours): 74 AICc value: 2929.903
Adaptive bandwidth (number of nearest neighbours): 57 AICc value: 2922.621
Adaptive bandwidth (number of nearest neighbours): 52 AICc value: 2922.879
Adaptive bandwidth (number of nearest neighbours): 58 AICc value: 2923.89
Adaptive bandwidth (number of nearest neighbours): 54 AICc value: 2922.612
Adaptive bandwidth (number of nearest neighbours): 54 AICc value: 2922.612
```

```
## Optimisation de la distance avec le CV
bwnCV.dist <- bw.gwr(NO2 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
  data = LyonIris,
  dMat = MatriceDistance,
  approach="CV",
  kernel="gaussian",
  adaptive=FALSE,
  longlat=FALSE)
```

```
Fixed bandwidth: 22632.98 CV score: 23240.62
Fixed bandwidth: 13990.75 CV score: 22942.89
Fixed bandwidth: 8649.556 CV score: 21987.02
Fixed bandwidth: 5348.517 CV score: 19330.61
Fixed bandwidth: 3308.362 CV score: 15656.55
Fixed bandwidth: 2047.478 CV score: 13534
Fixed bandwidth: 1268.208 CV score: 12883.62
Fixed bandwidth: 786.5929 CV score: 69868.32
Fixed bandwidth: 1565.863 CV score: 13058.8
Fixed bandwidth: 1084.247 CV score: 13320.38
Fixed bandwidth: 1381.902 CV score: 12891.23
Fixed bandwidth: 1197.941 CV score: 12968.27
Fixed bandwidth: 1311.635 CV score: 12869.97
Fixed bandwidth: 1338.475 CV score: 12872.81
Fixed bandwidth: 1295.047 CV score: 12872.27
Fixed bandwidth: 1321.887 CV score: 12870.16
Fixed bandwidth: 1305.299 CV score: 12870.46
Fixed bandwidth: 1315.551 CV score: 12869.91
Fixed bandwidth: 1317.971 CV score: 12869.95
Fixed bandwidth: 1314.055 CV score: 12869.91
Fixed bandwidth: 1316.475 CV score: 12869.92
Fixed bandwidth: 1314.98 CV score: 12869.9
Fixed bandwidth: 1314.627 CV score: 12869.91
Fixed bandwidth: 1315.198 CV score: 12869.9
```



```
## Optimisation de la distance avec l'AIC
bwAIC.dist <- bw.gwr(NO2 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
  data = LyonIris,
  dMat = MatriceDistance,
  approach="AIC",
  kernel="gaussian",
  adaptive=FALSE,
  longlat=FALSE)
```

```
Fixed bandwidth: 22632.98 AICc value: 3362.597
Fixed bandwidth: 13990.75 AICc value: 3354.75
Fixed bandwidth: 8649.556 AICc value: 3328.284
Fixed bandwidth: 5348.517 AICc value: 3241.508
Fixed bandwidth: 3308.362 AICc value: 3114.667
Fixed bandwidth: 2047.478 AICc value: 3043.093
Fixed bandwidth: 1268.208 AICc value: 3050.841
Fixed bandwidth: 2529.093 AICc value: 3065.054
Fixed bandwidth: 1749.823 AICc value: 3036.6
Fixed bandwidth: 1565.863 AICc value: 3036.749
Fixed bandwidth: 1863.517 AICc value: 3038.25
Fixed bandwidth: 1679.556 AICc value: 3036.195
Fixed bandwidth: 1636.129 AICc value: 3036.214
Fixed bandwidth: 1706.396 AICc value: 3036.289
Fixed bandwidth: 1662.969 AICc value: 3036.176
Fixed bandwidth: 1652.717 AICc value: 3036.181
Fixed bandwidth: 1669.305 AICc value: 3036.18
Fixed bandwidth: 1659.053 AICc value: 3036.177
Fixed bandwidth: 1665.389 AICc value: 3036.177
Fixed bandwidth: 1661.473 AICc value: 3036.176
Fixed bandwidth: 1660.549 AICc value: 3036.176
```

```
## Affichage des résultats d'optimisation
cat("Sélection de la taille de la zone optimale (bandwidth)",
  "\n avec le nombre de plus proches voisins :",
  "\n CV =", round(bwCV.voisins,4), "nombre de voisins =", round(bwCV.voisins*nrow(LyonIris)),
  "\n AIC =", round(bwAIC.voisins,4), "nombre de voisins =", round(bwAIC.voisins*nrow(LyonIris)),
  "\nSélection de la taille de la zone optimale (bandwidth) avec la distance :",
  "\n CV =", round(bwnCV.dist, 0), "mètres",
  "\n AIC =", round(bwAIC.dist, 0), "mètres")
```

Sélection de la taille de la zone optimale (bandwidth)
 avec le nombre de plus proches voisins :
 CV = 97 nombre de voisins = 49082
 AIC = 54 nombre de voisins = 27324
 Sélection de la taille de la zone optimale (bandwidth) avec la distance :

CV = 1315 mètres
AIC = 1661 mètres

4.2.2.2 Réalisation de la GWR

Comparativement à la fonction `gwr` du *package* `spgwr`, `gwr.basic` du *package* `GWmodel` offre l'avantage de fournir l'ensemble des résultats :

- Ceux de la régression linéaire multiple (modèle global).
- Un sommaire statistique des coefficients de régression de la GWR.
- Les statistiques de la qualité d'ajustement du modèle (AIC, AICc, R^2 , R^2 ajusté).
- Les tests pour comparer les modèles global et GWR (tests F1, F2).
- Le test F3 pour vérifier si les coefficients de régression varient ou non spatialement de façon significative.

```
# Calcul de la GWR
Modele.GWR <- gwr.basic(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                        data = LyonIris,
                        dMat = MatriceDistance, # Matrice de distances
                        bw = bwCV.voisins,      # Zone d'influence
                        kernel = "bisquare",    # Gaussien ou bicarrée
                        adaptive=TRUE,         # Nombre de voisins (TRUE) ou distance (FALSE)
                        F123.test = TRUE,      # Tests F123
                        longlat=FALSE)

Modele.GWR
```

```
*****
*                               Package    GWmodel                               *
*****
Program starts at: 2025-01-26 15:28:32.753671
Call:
gwr.basic(formula = N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet +
  NivVieMed, data = LyonIris, bw = bwCV.voisins, kernel = "bisquare",
  adaptive = TRUE, longlat = FALSE, dMat = MatriceDistance,
  F123.test = TRUE)

Dependent (y) variable:  N02
Independent variables:  Pct0_14 Pct_65 Pct_Img Pct_brevet NivVieMed
Number of data points: 506
*****
*                               Results of Global Regression                               *
*****

Call:
lm(formula = formula, data = data)
```

```

Residuals:
    Min       1Q   Median       3Q      Max
-27.733  -4.457  -0.499   3.507  29.160

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  49.43296    2.99550   16.502 < 2e-16 ***
Pct0_14      -0.53352    0.06305   -8.461 2.94e-16 ***
Pct_65       -0.15047    0.05627   -2.674 0.00774 **
Pct_Img       0.28287    0.05113    5.532 5.12e-08 ***
Pct_brevet  -0.24004    0.03721   -6.451 2.63e-10 ***
NivVieMed    -0.31625    0.10180   -3.107 0.00200 **

---Significance stars
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 6.685 on 500 degrees of freedom
Multiple R-squared:  0.2832
Adjusted R-squared:  0.276
F-statistic: 39.5 on 5 and 500 DF,  p-value: < 2.2e-16
***Extra Diagnostic information
Residual sum of squares: 22346.02
Sigma(hat): 6.658629
AIC: 3366.626
AICc: 3366.851
BIC: 2933.798
*****
*           Results of Geographically Weighted Regression           *
*****

*****Model calibration information*****
Kernel function: bisquare
Adaptive bandwidth: 97 (number of nearest neighbours)
Regression points: the same locations as observations are used.
Distance metric: A distance matrix is specified for this model calibration.

*****Summary of GWR coefficient estimates:*****
            Min.   1st Qu.   Median   3rd Qu.   Max.
Intercept 13.249694 30.566393 38.903053 48.035033 59.6124
Pct0_14   -1.094930 -0.371546 -0.224906 -0.053956  0.3386
Pct_65    -0.711775 -0.157939 -0.033630  0.073991  0.4514
Pct_Img   -0.319353 -0.042105  0.078841  0.243274  0.6657
Pct_brevet -0.624419 -0.224105 -0.092008  0.041021  0.5844
NivVieMed -1.108017 -0.555822 -0.213561  0.202347  1.2844
*****Diagnostic information*****
Number of data points: 506
Effective number of parameters (2trace(S) - trace(S'S)): 103.4065

```

```

Effective degrees of freedom (n-2trace(S) + trace(S'S)): 402.5935
AICc (GWR book, Fotheringham, et al. 2002, p. 61, eq 2.33): 2949.788
AIC (GWR book, Fotheringham, et al. 2002,GWR p. 96, eq. 4.22): 2839.039
BIC (GWR book, Fotheringham, et al. 2002,GWR p. 61, eq. 2.34): 2743.804
Residual sum of squares: 6933.276
R-square value: 0.7775915
Adjusted R-square value: 0.7203234
*****F test results of GWR calibration*****
---F1 test (Leung et al. 2000)
  F1 statistic Numerator DF Denominator DF      Pr(>)
      0.38534          Inf           500 < 2.2e-16 ***
---F2 test (Leung et al. 2000)
  F2 statistic Numerator DF Denominator DF Pr(>)
      3.5405      -31.0892           500    NaN
---F3 test (Leung et al. 2000)
      F3 statistic Numerator DF Denominator DF      Pr(>)
Intercept          2.3480          133.2095          Inf < 2.2e-16 ***
Pct0_14             2.8529          141.2525          Inf < 2.2e-16 ***
Pct_65              2.1479          168.6132          Inf 3.399e-16 ***
Pct_Img             1.9938          105.6136          Inf 5.756e-09 ***
Pct_brevet          2.4930          120.8152          Inf < 2.2e-16 ***
NivVieMed           3.8747          138.4279          Inf < 2.2e-16 ***
---F4 test (GWR book p92)
  F4 statistic Numerator DF Denominator DF      Pr(>)
      0.31027      402.59351           500 < 2.2e-16 ***

---Significance stars
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
*****
Program stops at: 2025-01-26 15:28:47.429748

```

4.2.2.3 Cartographie des résultats du modèle GWR

Une fois le modèle obtenu, nous disposons de l'ensemble des résultats locaux (coefficients, valeur prédite, résidus, valeurs de t et R^2) dans l'objet `Modele.GWR$SDF` qui sera une couche `sf` comme le jeu de données initial. Il est alors aisée de reproduire les cartes présentées à la section 4.2.1.4. En guise d'exemple, dans le code ci-dessous, nous cartographions les valeurs locales du R^2 et uniquement celles de t pour la variable relative au médiane du niveau de vie (figure 4.10).

```

GWR.Resultats <- Modele.GWR$SDF
names(GWR.Resultats)

```

```

[1] "Intercept"      "Pct0_14"        "Pct_65"         "Pct_Img"
[5] "Pct_brevet"     "NivVieMed"      "y"              "yhat"
[9] "residual"       "CV_Score"       "Stud_residual"  "Intercept_SE"
[13] "Pct0_14_SE"     "Pct_65_SE"      "Pct_Img_SE"     "Pct_brevet_SE"

```

```
[17] "NivVieMed_SE" "Intercept_TV" "Pct0_14_TV" "Pct_65_TV"
[21] "Pct_Img_TV" "Pct_brevet_TV" "NivVieMed_TV" "Local_R2"
[25] "geometry"
```

```
legende.parametres <- list(text.separator = "à",
                           text.less.than = "Moins de",
                           text.or.more = "et plus",
                           decimal.mark = ",",
                           big.mark = " ")

classes.intervalles = c(-Inf, -3.29, -2.58, -1.96, 1.96, 2.58, 3.29, Inf)

# Cartographie du R2
Carte1 <- tm_shape(GWR.Resultats)+
  tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="Local_R2",
          palette="YlOrBr",
          n=5, style="quantile",
          legend.format = legende.parametres,
          title = "R2 locaux")+
  tm_layout(frame=FALSE)+
  tm_scale_bar(breaks=c(0,5))

# Cartographie d'une valeur de t
Carte2 <- tm_shape(GWR.Resultats)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="NivVieMed_TV", palette="-RdBu",
          midpoint = NA,
          breaks = classes.intervalles,
          legend.format = legende.parametres,
          title = "Niveau de vie (€1000))+
  tm_layout(frame=FALSE)

tmap_arrange(Carte1, Carte2)
```

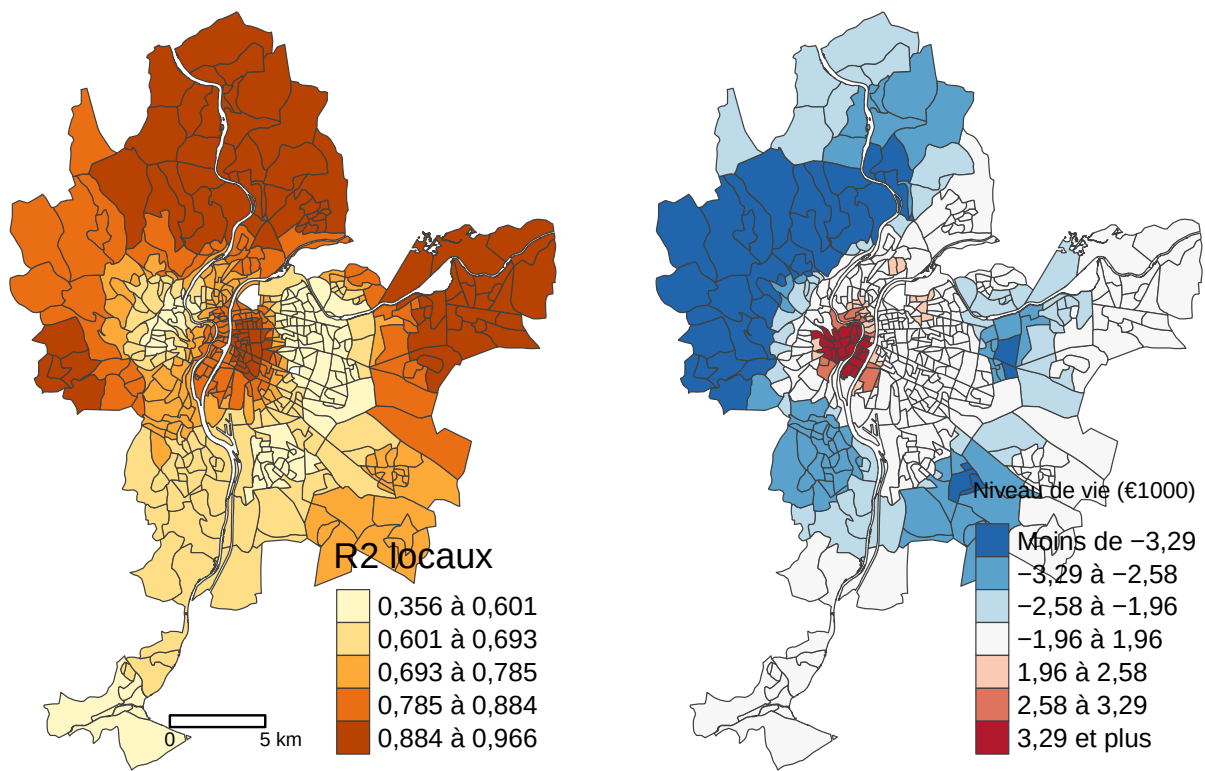


Figure 4.10: Exemple de cartographie avec les résultats de la GWR obtenus avec le *package GWmodel*

4.3 GWR classiques pour d'autres distributions

Aller plus loin

Modèles GWR avec d'autres distributions

Dans ce chapitre, nous avons décrit la GWR classique avec une distribution gaussienne qui est une extension de la régression linéaire multiple. Pour construire ce type de GWR classique, nous avons utilisé les fonctions des *packages* `spgwr` (`gwr.sel` et `gwr`) et `GWmodel` (`bw.gwr` et `gwr.basic`). De la même manière, il est possible de construire des modèles GWR qui sont des extensions des modèles linéaires généralisés (GLM), notamment pour des variables dépendantes qualitatives (modèle logistique binomial), de comptage (Poisson, quasi-poisson), continues (gaussien, inverse gaussienne, Gamma). Pour ce faire, il faut utiliser les fonctions `bw.ggwr` et `ggwr` du *package* `spgwr` et spécifier le type de distribution avec l'argument `family` dont la valeur par défaut est `gaussian()`. Cet argument permet de spécifier la famille de distribution et la fonction de lien en utilisant les familles de distribution disponibles dans la fonction de `glm` (*Generalized Linear Models* pour régressions linéaires généralisées) :

- `binomial(link = "logit")`
- `gaussian(link = "identity")`
- `Gamma(link = "inverse")`
- `inverse.gaussian(link = "1/mu^2")`
- `poisson(link = "log")`
- `quasi(link = "identity", variance = "constant")`
- `quasibinomial(link = "logit")`
- `quasipoisson(link = "log")`

Avec le *package* `GWmodel`, vous pourrez utiliser les fonctions `bw.ggwr` et `gwss` qui permettent de réaliser des régressions linéaires pondérées avec des distributions de Poisson et binomiale (avec l'argument `family = "poisson"` et `family = "binomial"`).

4.4 Limites et critiques des GWR

Bien que la GWR classique représente un outil d'exploration de données spatiales particulièrement intéressant pour évaluer l'importance de l'instabilité spatiale d'un modèle de régression (Apparicio, Séguin et Leloup 2007; Mennis et Jordan 2005; Calvo et Escolar 2003), elle comprend plusieurs limites :

- **Type de distance.** Dans le cadre de ce chapitre, nous avons calculé la GWR avec la distance euclidienne qui n'est pas adaptée à tous les phénomènes à l'étude. Par exemple, Lu, Charlton et Fotheringham (2011) utilisent des distances calculées sur un réseau de rues pour modéliser le prix de vente des maisons à Londres avec un modèle GWR. Pour ce faire, il est possible d'utiliser les fonctions `bw.gwr` et `gwr.basic` du *package* `GWmodel` dont l'argument `dmat` permet de spécifier une matrice de distance prédéfinie.
- **Choix du noyau (kernel) et la taille de la zone d'influence.** Ces deux paramètres de la GWR peuvent impacter grandement les résultats. En effet, la zone d'influence (défini avec une distance ou un nombre donné de plus proches voisins) peut entraîner localement un nombre insuffisant d'observations, compromettant ainsi la robustesse des équations de régression locales.

- **Prédicteurs locaux et globaux.** Nous avons vu que la fonction `LMZ.F3GWR.test` permet de déterminer si les coefficients de régression présentent des variations significatives dans l'espace. Or, certaines variables indépendantes peuvent être introduites globalement et non localement, comme nous le verrons dans le prochain chapitre (section 5.1).

La GWR fait aussi face à d'importantes critiques :

- **Validité des modèles locaux.** Pour une régression linéaire multiple, il est primordial de réaliser un diagnostic pour s'assurer du nombre suffisant d'observations, de l'absence de multicollinéarité excessive et de la normalité et l'homoscédasticité des résidus (lire par exemple la [section suivante](#) (Apparicio et Gelb 2022)). Or, ce diagnostic devrait être réalisé pour tous les modèles locaux obtenus pour les n observations du jeu de données. Par exemple, si le modèle global ne pose pas de problème de multicollinéarité excessive, rien ne garantit qu'elle ne soit pas présente dans plusieurs modèles locaux (Wheeler et Tiefelsdorf 2005).
- **Corrélation entre les coefficients locaux.** Dans un article intitulé *Multicollinearity and correlation among local regression coefficients in geographically weighted regression*, Wheeler et Tiefelsdorf (2005) signalent les coefficients de régression locaux peuvent être fortement corrélés entre eux, en raison d'un problème de multicollinéarité excessive des modèles locaux. En guise d'exemple, la figure 4.11 montre bien que certaines paires de coefficients locaux sont corrélées fortement.

```
library("GGally")
df <- data.frame(Modele.GWR$SDF)
df <- df[, 2:7]
names(df) <- c("Constante" , "Pct0_14", "Pct_65", "Pct_Img", "Pct_brevet", "NivVieMed")

# Graphique avec le package GGally
ggpairs(df, progress = FALSE)
# Il est aussi possible de réaliser un graphique avec le package corrplot
library("corrplot")
# p <- cor(df, method="pearson")
# couleurs <- colorRampPalette(c("#053061", "#2166AC" , "#4393C3", "#92C5DE",
#                               "#D1E5F0", "#FFFFFF", "#FDDBC7", "#F4A582",
#                               "#D6604D", "#B2182B", "#67001F"))
# corrplot.mixed(p, lower="number", lower.col = "black",
#               upper = "ellipse", upper.col=couleurs(100))
```

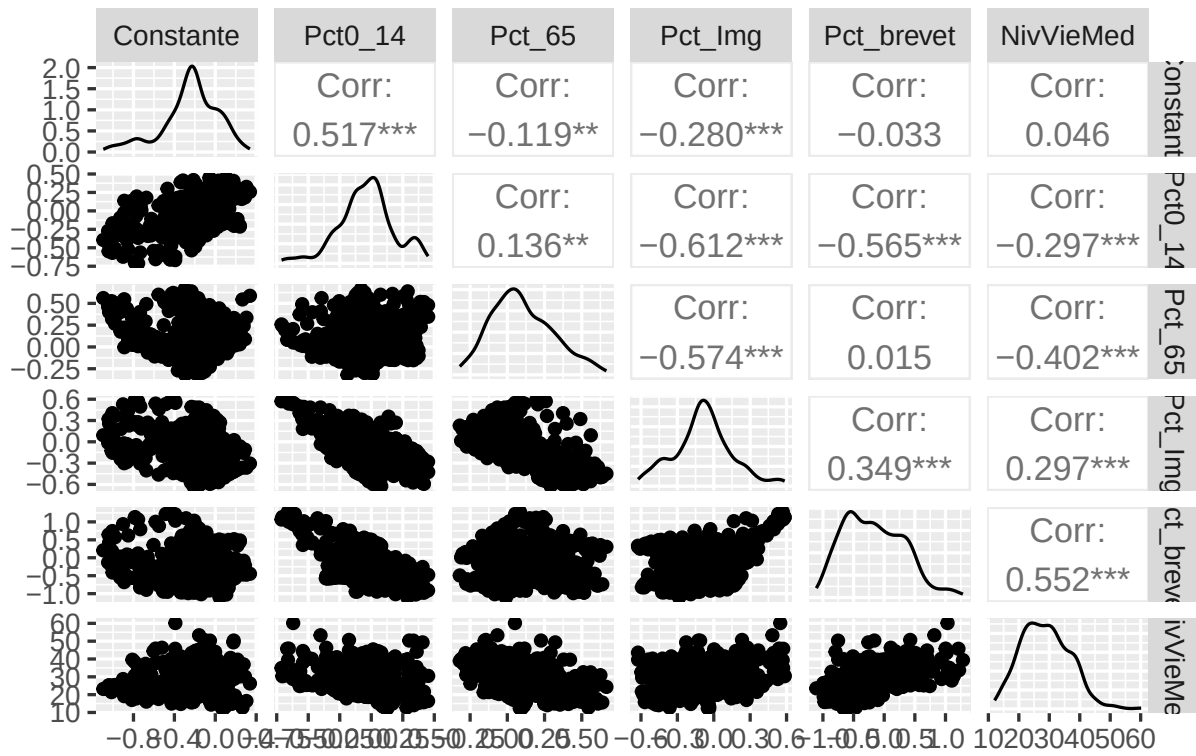


Figure 4.11: Corrélations de Pearson entre les coefficients de régression locaux de la GWR

4.5 Quiz de révision

4.6 Exercices de révision

Exercice

Exercice 1. Réalisation d'un GWR classique

```
library(sf)
library(spgwr)
load("data/Lyon.Rdata")
# Ajout des coordonnées x et y
xy <- à compléter
LyonIris$X <- à compléter
LyonIris$Y <- à compléter
# Optimisation du nombre de voisins avec le CV
formule <- "PM25 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed"
bwCV.voisins <- gwr.sel(à compléter)
# Réalisation de la GWR
Modele.GWR <- gwr(à compléter)
# Affichage des résultats
Modele.GWR
```

Correction à la section 8.4.1.

Exercice

Exercice 2. Comparaison des modèles MCO et GWR

```
library(sf)
library(spgwr)
# Modèle MCO
Modele.Global <- à compléter
# Comparaison des R2
R2.global <- à compléter
rss <- à compléter
tss <- à compléter
R2.GWRquasiglobal <- à compléter
cat("R2 global (LM) = ", round(R2.global, 3),
    "\nR2 quasi-global (GWR) : ", round(R2.GWRquasiglobal, 3))
# Comparaison des R2
à compléter
# Les coefficients du modèle GWR varient-ils significativement dans l'espace?
à compléter
```

Correction à la section 8.4.2.

 Exercice**Exercice 3.** Cartographie des R^2 et des valeurs de t

```
library(tmap)
# Récupération du R carré locaux et valeurs de t locales
à compléter

## Cartographie des R2 locaux
à compléter

# Cartographie des valeurs de t locales
## Paramètres pour la légende
legende.parametres <- list(text.separator = "à",
                           text.less.than = "Moins de",
                           text.or.more = "et plus",
                           decimal.mark = ",",
                           big.mark = " ")

## Cartographie
à compléter
```

Correction à la section 8.4.3.

5 Extensions de la régression géographiquement pondérée

Dans le chapitre 4, nous avons présenté les trois formes classiques de la régression géographiquement pondérée (GWR) qui permettent de modéliser des variables dépendantes continues (GWR gaussienne), dichotomiques (GWR logistique) ou des variables de comptage (GWR Poisson). Depuis la publication de l'ouvrage de référence de Steward Fotheringham, Chris Brunsdon et Martin Charlton (2003), plusieurs extensions ont vu le jour. Parmi celles-ci, nous abordons dans ce chapitre :

- la GWR mixte qui permet de spécifier des variables indépendantes variant spatialement et d'autres étant fixes (Fotheringham, Brunsdon et Charlton 2003).
- La régression géographiquement pondérée multiéchelle (*Multiscale Geographically Weighted Regression* – MGWR) (Fotheringham, Yang et Kang 2017).
- Les GWR mixtes intégrant des variables spatialement décalées (MGWR-SAR) (Geniaux et Martinetti 2018).

Objectif

Objectifs d'apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- Comprendre pourquoi utiliser différentes extensions de la GWR (GWR-mixte, MGWR, GTWR et MGWR-SAR).
- Assimiler les principes fondamentaux de ces différentes extensions de la GWR.
- Appréhender les différentes extensions de la GWR (GWR-mixte, MGWR, GTWR et MGWR-SAR).
- Analyser les résultats produits par ces différentes extensions.
- Mettre en pratique ces extensions de la GWR dans R.

Package

Liste des *packages* utilisés dans ce chapitre

- Pour importer et manipuler des fichiers géographiques :
 - `sf` pour importer et manipuler des données vectorielles.
 - `dplyr` pour manipuler les données.
- Pour construire des cartes et des graphiques :
 - `tmap` pour les cartes.
 - `ggplot2` pour construire des graphiques.
- Pour construire différentes extensions de la GWR :
 - `Gwmodel` pour construire des GWR mixtes, des MGWR et des GTWR.
 - `mgwrsar` pour construire des GWR avec des variables spatialement décalées.

5.1 Régression géographiquement pondérée mixte

5.1.1 Principe de base de la GWR mixte

5.1.1.1 Pourquoi recourir à une GWR mixte

5.1.1.2 Formulation de la GWR mixte

5.1.2 Mise en oeuvre de la GWR mixte dans R

5.2 Régression géographiquement pondérée multiéchelle

5.2.1 Principe de base de la MGWR

5.2.1.1 Pourquoi recourir à une GWR mixte

5.2.1.2 Formulation de la GWR mixte

5.2.2 Mise en oeuvre de la MGWR dans R

5.3 Régression géographiquement pondérée mixte avec des variables spatialement décalée

5.3.1 Principe de base de la MGWR-SAR

5.3.1.1 Pourquoi recourir à une MGWR-SAR

5.3.1.2 Formulation de la MGWR-SAR

5.3.2 Mise en oeuvre de la MGWR-SAR dans R

5.4 Quiz de révision

5.5 Exercices de révision

Exercice

Exercice 1. Réalisation d'un GWR classique

```
library(sf)
library(spgwr)
```

Correction à la ?@sec-06041.

Exercice

Exercice 2. Réalisation d'un GWR classique mixte

```
library(sf)
library(spgwr)
```

Correction à la ?@sec-06042.

Exercice

Exercice 3. Réalisation d'un GWR classique multiéchelle

```
library(sf)
library(spgwr)
```

Correction à la ?@sec-06043.

Partie 4. Autres approches de modélisations spatiales

6 Modèles généralisées additifs

Dans ce chapitre, nous abordons deux formes de modèles généralisées additifs (*Generalized additive model* en anglais – GAM) qui permettent d’introduire l’espace deux manières différentes : les modèles GAM avec une *spline* bivariée avec les coordonnées géographiques (x, y) pour capturer les variations continues dans l’espace; les modèles GAM avec un lissage par champ aléatoire de Markov (*random markov field* – RMF) pour modéliser la dépendance spatiale entre les unités spatiales voisines.

🎯 Objectif

Objectifs d’apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- Comprendre pourquoi utiliser un modèle GAM avec un *spline* bivariée sur les coordonnées géographiques ou avec un lissage par champ aléatoire de Markov.
- Comprendre les différences entre ces deux modèles : effets spatiaux lisses et globaux *versus* effets spatiaux locaux avec dépendance de voisinage.
- Analyser les résultats produits par ces deux types de modèles GAM.
- Les mettre en pratique dans R.

📦 Package

Liste des *packages* utilisés dans ce chapitre

- Pour importer et manipuler des fichiers géographiques :
 - `sf` pour importer et manipuler des données vectorielles.
- Pour construire des cartes et des graphiques :
 - `tmap` est certainement le meilleur *package* pour la cartographie.
 - `ggplot2` pour construire des graphiques.
- Pour construire des modèles GAM :
 - `mgcv` pour pratiquer ses GAM dans R!

6.1 Modèles généralisés additifs (GAM) avec une *spline* bivariée sur les coordonnées géographiques

Les modèles généralisés additifs (*Generalized additive models* en anglais) permettent d’intégrer à la fois des effets linéaires et des effets non linéaires avec des *splines*. Ils peuvent alors être utilisés en intégrant une *spline* bivariée sur les

coordonnées géographiques des centroïdes des entités spatiales.

⚠ Attention

Modèles généralisés additifs

Pour une description détaillée des modèles généralisés additifs, nous vous invitons vivement à lire le [chapitre suivant](#) (Apparicio et Gelb 2024).

6.1.1 Principe de base d'un GAM intégrant l'espace

6.1.1.1 Formulation d'un modèle GAM vec une spline bivariable sur les coordonnées géographiques

Avec une *spline* bivariable sur les coordonnées géographiques, l'équation d'un modèle généralisé additif s'écrit :

$$g(Y) = \beta_0 + X\beta + s(\text{Coord}X, \text{Coord}Y) + \epsilon \quad (6.1)$$

avec :

- y , la variable dépendante.
- β_0 , la constante.
- X , les variables indépendantes.
- β , les coefficients des variables dépendantes.
- $s(\text{Coord}X, \text{Coord}Y)$, *spline* bivariable sur les coordonnées x et y .
- ϵ , les résidus.

6.1.1.2 Pourquoi recourir à un modèle GAM avec une spline bivariable sur les coordonnées géographiques

L'intérêt de recourir à une *spline* bivariable sur les coordonnées géographiques est double :

1. Contrôler l'effet de la localisation sur la variable dépendante (y). Les coefficients des autres variables indépendantes sont ainsi obtenus une fois l'espace pris en compte.
2. Évaluer l'effet de la localisation (patron spatial), une fois les autres variables indépendantes contrôlées. Autrement dit, toutes choses étant égales par ailleurs, quel est l'effet de la localisation sur la variable dépendante?

6.1.2 Mise en oeuvre et analyse dans R

6.1.2.1 Réalisation du modèle GAM

Pour construire des modèles GAM dans R, nous utilisons la fonction `gam` du *package* `mgcv` (Wood 2011).

```
## Chargement des packages et des données
library(sf)
library(spdep)
library(mgcv)
load("data/Lyon.Rdata")
## Ajout des coordonnées X et Y dans LyonIris
xy <- st_coordinates(st_centroid(LyonIris))
LyonIris$X <- xy[,1]
LyonIris$Y <- xy[,2]
## Construction du modèle MCO
Modele.MCO <- lm(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed,
                 data = LyonIris)
## Construction du modèle GAM
Modele.GAM1 <- gam(N02 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed+
                   s(X, Y), # spline sur les coordonnées X, Y
                   family=gaussian(), # distribution
                   data = LyonIris) # dataframe
## Résultats du modèle
summary(Modele.GAM1)
```

Family: gaussian

Link function: identity

Formula:

N02 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed + s(X,
Y)

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	30.402313	2.156322	14.099	<2e-16 ***
Pct0_14	-0.053641	0.046224	-1.160	0.246
Pct_65	-0.056310	0.038476	-1.463	0.144
Pct_Img	0.008033	0.035397	0.227	0.821
Pct_brevet	0.043874	0.027518	1.594	0.112
NivVieMed	-0.043282	0.072461	-0.597	0.551

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	F	p-value
s(X,Y)	26.5	28.62	27.22	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.715 Deviance explained = 73.3%

GCV = 18.814 Scale est. = 17.605 n = 506

Les résultats ci-dessus signalent que la localisation a un effet très significatif puisque $s(X,Y) = 26,5$ avec $p < 0,001$. Notez que la valeur de p permet de déterminer si la *spline* bivariée (et donc l'espace) a ou non un effet significatif. Si la valeur de p est supérieure à 0,05, alors il n'est pas nécessaire de conserver la *spline* bivariée sur les coordonnées géographiques.

De plus, le code ci-dessous permet de constater que le modèle GAM est plus performant que le modèle linéaire multiple classique (MCO).

```
anova(Modele.MCO, Modele.GAM1)
```

Analysis of Variance Table

Model 1: NO2 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed

Model 2: NO2 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed + s(X,
Y)

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	500.0	22346.0				
2	473.5	8336.1	26.5	14010	30.029	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Nous pouvons aussi introduire une *spline* plus complexe en augmentant le nombre de nœuds à 40.

```
Modele.GAM2 <- gam(NO2 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed+
                    s(X, Y, k= 40),
                    family=gaussian(),
                    data = LyonIris)
summary(Modele.GAM2)
```

Family: gaussian

Link function: identity

Formula:

NO2 ~ Pct0_14 + Pct_65 + Pct_Img + Pct_brevet + NivVieMed + s(X,
Y, k = 40)

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	30.372093	2.088160	14.545	<2e-16 ***
Pct0_14	-0.055294	0.044285	-1.249	0.2124
Pct_65	-0.049122	0.036870	-1.332	0.1834
Pct_Img	0.002226	0.033722	0.066	0.9474
Pct_brevet	0.052229	0.026315	1.985	0.0478 *

```

NivVieMed    -0.050991    0.070081   -0.728    0.4672
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
      edf Ref.df      F p-value
s(X,Y) 35.68  38.51 24.27  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.747   Deviance explained = 76.7%
GCV = 17.014   Scale est. = 15.613       n = 506

```

La valeur plus faible d'AIC pour le second modèle GAM signale qu'il est plus performant que le premier.

```
AIC(Modele.MCO, Modele.GAM1, Modele.GAM2)
```

```

      df      AIC
Modele.MCO  7.00000 3366.626
Modele.GAM1 33.50046 2920.682
Modele.GAM2 42.67520 2868.355

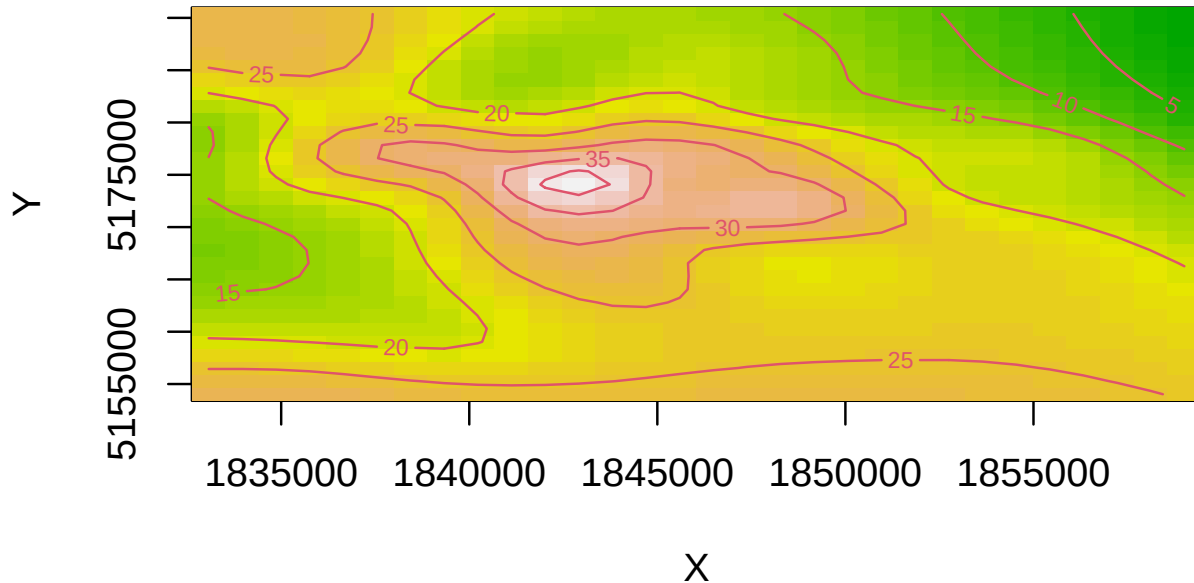
```

6.1.2.2 Visualisation de l'effet de l'espace

Pour visualiser les prédictions du modèle dans l'espace, toutes choses étant égales par ailleurs, nous utilisons la fonction `vis.gam` (figure 6.1). Les contours signalent qu'au centre de Lyon, les valeurs de dioxyde d'azote sont les plus élevées et dépassent même $40 \mu\text{g}/\text{m}^3$.

```
vis.gam(Modele.GAM2, view=c("X", "Y"), plot.type = "contour", color="terrain")
```

linear predictor

Figure 6.1: Visualisation des prédictions dans l'espace avec la fonction `vis.gam`

Toutefois, il est plus intéressant de la représenter dans un *raster*, une fois contrôlées les autres variables indépendantes. Pour ce faire, six étapes sont nécessaires :

1. Créer une *grid*.
2. Fixer les autres paramètres à leur moyenne respective.
3. Calculer la prédiction pour la localisation.
4. Centrer la prédiction.
5. Construire le *raster* avec les prédictions.
6. Découper et cartographier le *raster*.

```
library(raster)
library(terra)
## Étape 1 : création d'une grid pour la prédiction de 100 mètres de résolution spatiale
Xcoords <- seq(min(LyonIris$X-100), max(LyonIris$X+100), by=100)
Ycoords <- seq(min(LyonIris$Y-100), max(LyonIris$Y+100), by=100)
PredDF <- expand.grid(Xcoords,Ycoords)
names(PredDF) <- c("X","Y")
## Étape 2 : fixation de tous les autres paramètres à leur moyenne
for(Var in c("VegHautPrt","Pct0_14","Pct_65","Pct_Img","Pct_brevet", "NivVieMed")){
  PredDF[[Var]] <- mean(LyonIris[[Var]])
}
## Étape 3 : calcul de la prédiction
PredDF$PM25 <- predict(Modele.GAM2,newdata=PredDF)
## Étape 4 : centrage de la prédiction (sans la constante)
```

```

PredDF$CenterPredPM25 <- PredDF$PM25 - mean(PredDF$PM25)
### Étape 5 : construction du raster
rasterGAM <- rasterFromXYZ(PredDF[, c("X", "Y", "CenterPredPM25")])
crs(rasterGAM) <- crs(as(LyonIris, "Spatial"))
rasterGAM <- rast(rasterGAM)
### Étape 6 : découpage et cartographie du raster
LyonIris.SpatVector <- vect(LyonIris)
rasterGAM <- terra::mask(rasterGAM, LyonIris.SpatVector)
terra::plot(rasterGAM)

```

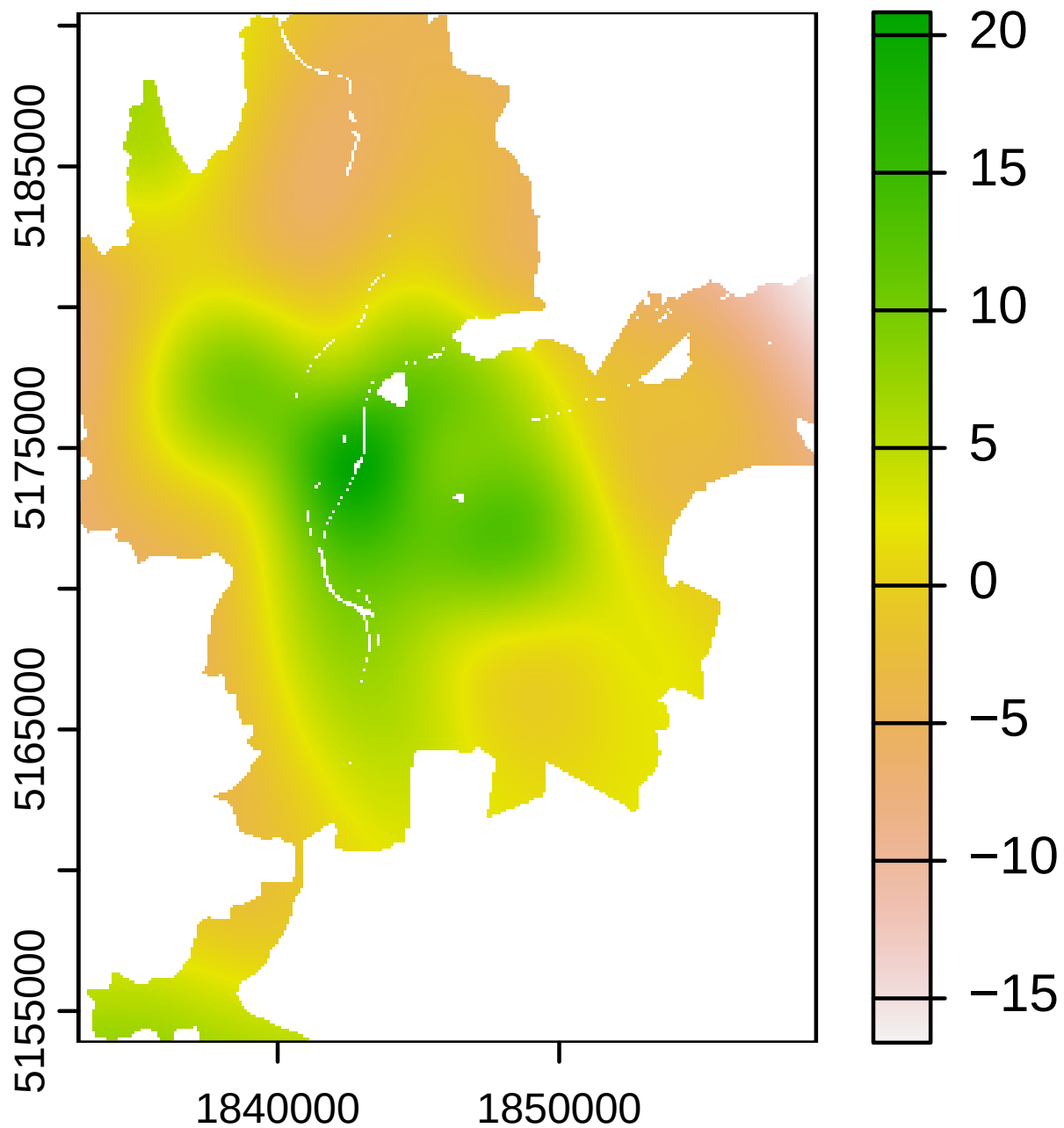


Figure 6.2: Visualisation de l'effet de la localisation centrée sur zéro

La figure 6.2 signale que dans le centre de Lyon, le dioxyde d'azote est plus élevé de 10 à 20 $\mu\text{g}/\text{m}^3$, toutes choses étant égales par ailleurs. À l'inverse, dans les zones périphériques, il est faible. Cela signale un net patron spatial décroissant du centre vers la périphérie.

6.1.2.3 Dépendance spatiale du modèle GAM

Par contre, bien que l'autocorrélation spatiale des résidus du modèle GAM soit plus faible que pour le modèle MCO (I de Moran de 0,337 contre 0,570 avec $p < 0,001$), il reste que le problème de la dépendance spatiale n'est pas corrigé.

```
## Matrice de contiguïté selon le partage d'un segment (Rook)
Rook <- poly2nb(LyonIris, queen=FALSE)
W.Rook <- nb2listw(Rook, zero.policy=TRUE, style = "W")
## Autocorrélation sur les résidus du modèle GAM
moran.mc(resid(Modele.GAM2), W.Rook, nsim=999)
```

Monte-Carlo simulation of Moran I

```
data: resid(Modele.GAM2)
weights: W.Rook
number of simulations + 1: 1000
```

```
statistic = 0.33664, observed rank = 1000, p-value = 0.001
alternative hypothesis: greater
```

6.2 GAM avec un lissage par champ aléatoire de Markov

6.2.1 Principe de base d'un avec un champ aléatoire de Markov

6.2.2 Mise en oeuvre et analyse dans R

<https://fromthebottomoftheheap.net/2017/10/19/first-steps-with-mrf-smooths/>

Aller plus loin

Modèles généralisées additifs avec d'autres distributions pour la variable dépendante

Dans ce chapitre, nous avons créé des modèles GAM en introduisant l'espace de deux manières différentes, et ce, avec une distribution gaussienne. Or, il est aussi possible de construire ces deux types de modèles avec des variables dépendantes qualitatives (modèles logistique, probit, multinomial, ordinal), de comptage (modèles Poisson, binomial négatif, avec excès fixe de zéros, avec excès ajusté de zéros), continues (gaussien, Student, Gamma, bêta).

La fonction `gam` du *package* `mgcv` comprend un argument `family` dont la valeur par défaut est `gaussian()`. Cet argument permet de spécifier la famille de distribution et la fonction de lien en utilisant les familles de distribution disponibles dans la fonction de `glm` (*Generalized Linear Models* pour régressions linéaires généralisées) :

- `binomial(link = "logit")`
- `gaussian(link = "identity")`
- `Gamma(link = "inverse")`
- `inverse.gaussian(link = "1/mu^2")`
- `poisson(link = "log")`
- `quasi(link = "identity", variance = "constant")`
- `quasibinomial(link = "logit")`
- `quasipoisson(link = "log")`

D'autres familles de distribution sont aussi spécifiques la fonction `gam` de `mgcv` :

- `betar(link="logit")`
- `cnorm(theta=NULL,link="identity")`
- `nb(theta = NULL, link = "log")`
- `ocat(theta=NULL,link="identity",R=NULL)`
- `scat(theta = NULL, link = "identity",min.df=3)`
- `tw(theta = NULL, link = "log",a=1.01,b=1.99)`
- `zip(theta = NULL, link = "identity",b=0)`

6.3 Quiz de révision

6.4 Exercices de révision

Exercice

Exercice 1. Réalisation d'un modèle GAM

```
library(sf)
library(mgcv)
load("data/chap07/DonneesLyon.Rdata")
# Ajout des coordonnées x et y
xy <- à compléter
LyonIris$X <- à compléter
LyonIris$Y <- à compléter
# Construction du modèle
formule <- "PM25 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed"
Modele.GAM2 <- gam(NO2 ~ à compléter
                  à compléter,
                  data = LyonIris)
summary(Modele.GAM2)
```

Correction à la ?@sec-12072.

7 Modèles avec des vecteurs propres spatiaux

Dans ce chapitre, nous abordons les modèles linéaires généralisés avec des vecteurs spatiaux proposés par Daniel Griffith (2003; Griffith, Chun et Li 2019).

Objectif

Objectifs d'apprentissage visés dans ce chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

- À compléter.
- À compléter.
- Analyser les résultats produits par ce type de modèle.
- Les mettre en pratique dans R.

Package

Liste des *packages* utilisés dans ce chapitre

- Pour importer et manipuler des fichiers géographiques :
 - `sf` pour importer et manipuler des données vectorielles.
- Pour construire des cartes et des graphiques :
 - `tmap` est certainement le meilleur *package* pour la cartographie.
 - `ggplot2` pour construire des graphiques.
- Pour construire des modèles GAM :
 - `spfilter` est un package dédié à ce type de modèles.

Partie 7. Exercices et bibliographie

8 Correction des exercices

8.1 Exercices du chapitre 1

8.1.1 Exercice 1

8.1.2 Exercice 2

8.1.3 Exercice 3

8.2 Exercices du chapitre 2

8.2.1 Exercice 1

8.2.2 Exercice 2

8.2.3 Exercice 3

8.3 Exercices du chapitre 3

8.3.1 Exercice 1

8.3.2 Exercice 2

8.3.3 Exercice 3

8.4 Exercices du chapitre 4

8.4.1 Exercice 1

```
library(sf)
library(spgwr)
load("data/Lyon.Rdata")
# Ajout des coordonnées x et y
xy <- st_coordinates(st_centroid(LyonIris))
LyonIris$X <- xy[,1]
LyonIris$Y <- xy[,2]
```

```

# Optimisation du nombre de voisins avec le CV
formule <- "PM25 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed"
bwaCV.voisins <- gwr.sel(formule,
                        data = LyonIris,
                        method = "cv",
                        gweight=gwr.bisquare,
                        adapt=TRUE,
                        verbose = FALSE,
                        RMSE = TRUE,
                        longlat = FALSE,
                        coords=cbind(LyonIris$X,LyonIris$Y))

# Optimisation du nombre de voisins avec l'AIC
formule <- "PM25 ~ Pct0_14+Pct_65+Pct_Img+Pct_brevet+NivVieMed"
bwaCV.voisins <- gwr.sel(formule,
                        data = LyonIris,
                        method = "CV",
                        gweight=gwr.bisquare,
                        adapt=TRUE,
                        verbose = FALSE,
                        RMSE = TRUE,
                        longlat = FALSE,
                        coords=cbind(LyonIris$X,LyonIris$Y))

# Réalisation de la GWR
Modele.GWR <- gwr(formule,
                  data = LyonIris,
                  adapt=bwaCV.voisins,
                  gweight=gwr.bisquare,
                  hatmatrix=TRUE,
                  se.fit=TRUE,
                  coords=cbind(LyonIris$X,LyonIris$Y),
                  longlat=FALSE)

# Affichage des résultats
Modele.GWR

```

8.4.2 Exercice 2

```

# Modèle MCO
Modele.Global <- lm(formule, data = LyonIris)
# Comparaison des R2
R2.global <- summary(Modele.Global)$r.squared
rss <- sum((Modele.GWR$lm$y - Modele.GWR$SDF$pred)^2)
tss <- sum((Modele.GWR$lm$y - mean(Modele.GWR$SDF$pred))^2)
R2.GWRquasiglobal <- 1 - (rss / tss)
cat("R2 global (LM) = ", round(R2.global, 3),

```

```

  "\nR2 quasi-global (GWR) : ", round(R2.GWRquasiglobal, 3))
# Comparaison des R2
anova(Modele.GWR)
LMZ.F1GWR.test(Modele.GWR)
LMZ.F2GWR.test(Modele.GWR)
# Les coefficients du modèle GWR varient-ils significativement dans l'espace?
LMZ.F3GWR.test(Modele.GWR)

```

8.4.3 Exercice 3

```

library(tmap)
# Récupération du R carré locaux et valeurs de t locales
LyonIris$GWR.R2 <- Modele.GWR$SDF$localR2
VarsIndep <- c("Pct0_14", "Pct_65", "Pct_Img", "Pct_brevet", "NivVieMed")
for(e in VarsIndep){
  # Nom des nouvelles variables
  var.coef <- paste0("GWR.", "B_", e)
  var.t <- paste0("GWR.", "T_", e)
  # Récupération des coefficients pour les variables indépendantes
  LyonIris[[var.coef]] <- Modele.GWR$SDF[[e]]
  # Calcul des valeurs de t pour les variables indépendantes
  LyonIris[[var.t]] <- Modele.GWR$SDF[[e]] / Modele.GWR$SDF[[paste0(e, "_se")]]
}

# Cartographie des R2 locaux
legendes.parametres <- list(text.separator = "à",
                             decimal.mark = ",",
                             big.mark = " ")

tm_shape(LyonIris)+
  tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.R2",
          palette="YlOrBr",
          n=5, style="quantile",
          legend.format = legendes.parametres,
          title = "R2 locaux")+
  tm_layout(frame=FALSE)+
  tm_scale_bar(breaks=c(0,5))

# Cartographie des valeurs de t locales
## Paramètres pour la légende
legende.parametres <- list(text.separator = "à",
                           text.less.than = "Moins de",
                           text.or.more = "et plus",
                           decimal.mark = ",",

```

```

        big.mark = " ")

## Cartographie
classes.intervalles = c(-Inf, -3.29, -2.58, -1.96, 1.96, 2.58, 3.29, Inf)
Carte1 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.T_Pct0_14", palette="-RdBu",
    midpoint = NA,
    breaks = classes.intervalles,
    legend.format = legende.parametres,
    title = "Moins de 15 ans (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte2 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.T_Pct_65", palette="-RdBu",
    midpoint = NA,
    breaks = classes.intervalles,
    legend.format = legende.parametres,
    title = "65 ans et plus (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte3 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.T_Pct_Img", palette="-RdBu",
    midpoint = NA,
    breaks = classes.intervalles,
    legend.format = legende.parametres,
    title = "Immigrants (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte4 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.B_Pct_brevet", palette="-RdBu",
    midpoint = NA,
    breaks = classes.intervalles,
    legend.format = legende.parametres,
    title = "Faible scolarité (%)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)

Carte5 <- tm_shape(LyonIris)+ tm_borders(col="gray25", lwd=.5)+
  tm_fill(col="GWR.T_NivVieMed", palette="-RdBu",
    midpoint = NA,
    breaks = classes.intervalles,
    legend.format = legende.parametres,
    title = "Niveau de vie (€1000)")+
  tm_layout(frame=FALSE, legend.outside = TRUE)+
  tm_scale_bar(breaks=c(0,5))

tmap_arrange(Carte1, Carte2, Carte3, Carte4, Carte5, ncol = 2, nrow=3)

```


8.5 Exercices du chapitre 5

8.5.1 Exercice 1

8.5.2 Exercice 2

8.5.3 Exercice 3

Bibliographie

- Anselin, Luc. 1988. *Spatial econometrics: methods and models*. Kluwer Academic Publishers.
- . 2010. « Thirty years of spatial econometrics. » *Papers in regional science* 89 (1): 3-26. <https://doi.org/10.1111/j.1435-5957.2010.00279.x>.
- Anselin, Luc, Anil K Bera, Raymond Florax et Mann J Yoon. 1996. « Simple diagnostic tests for spatial dependence. » *Regional science and urban economics* 26 (1): 77-104. [https://doi.org/10.1016/0166-0462\(95\)02111-6](https://doi.org/10.1016/0166-0462(95)02111-6).
- Apparicio, Philippe et Jérémy Gelb. 2022. *Méthodes quantitatives en sciences sociales : un grand bol d'R*. FabriqueREL, Licence CC BY-SA. <https://serieboldr.github.io/MethodesQuantitatives/>.
- . 2024. *Méthodes d'analyse spatiales : un grand bol d'R*. FabriqueREL, Licence CC BY-SA. <https://serieboldr.github.io/MethodesAnalyseSpatiale/>.
- Apparicio, Philippe, Jérémy Gelb, Anne-Sophie Dubé, Simon Kingham, Lise Gauvin et Éric Robitaille. 2017. « The approaches to measuring the potential spatial access to urban health services revisited: distance types and aggregation-error issues. » *International journal of health geographics* 16 (1): 32. <https://doi.org/10.1186/s12942-017-0105-9>.
- Apparicio, Philippe, Anne-Marie Séguin et Xavier Leloup. 2007. « Modélisation spatiale de la pauvreté à Montréal: apport méthodologique de la régression géographiquement pondérée. » *The Canadian Geographer/Le Géographe canadien* 51 (4): 412-427. <https://doi.org/10.1111/j.1541-0064.2007.00189.x>.
- Binbin Lu, Paul Harris, Martin Charlton et Christopher Brunsdon. 2014. « The GWmodel R package: further topics for exploring spatial heterogeneity using geographically weighted models. » *Geo-spatial Information Science* 17 (2): 85-101. [10.1080/10095020.2014.917453](https://doi.org/10.1080/10095020.2014.917453).
- Bivand, Roger, Giovanni Millo et Gianfranco Piras. 2021. « A review of software for spatial econometrics in R. » *Mathematics* 9 (11): 1276. <https://doi.org/10.3390/math9111276>.
- Bivand, Roger et Danlin Yu. 2023. *spgwr: Geographically Weighted Regression*. s.n. <https://CRAN.R-project.org/package=spgwr>.
- Brunsdon, Chris, A Stewart Fotheringham et Martin Charlton. 1998. « Spatial nonstationarity and autoregressive models. » *Environment and Planning A* 30 (6). SAGE Publications Sage UK: London, England: 957-973.
- Brunsdon, Chris, A Stewart Fotheringham et Martin E Charlton. 1996. « Geographically weighted regression: a method for exploring spatial nonstationarity. » *Geographical analysis* 28 (4). Wiley Online Library: 281-298.
- Calvo, Ernesto et Marcelo Escolar. 2003. « The local voter: A geographically weighted approach to ecological inference. » *American Journal of Political Science* 47 (1): 189-204. <https://doi.org/10.1111/1540-5907.00013>.
- Chi, Guangqing et Jun Zhu. 2019. *Spatial regression models for the social sciences*. SAGE publications.
- Dubé, Jean et Diègo Legros. 2014. *Econométrie spatiale appliquée des microdonnées*. ISTE Group.
- Fotheringham, A Stewart, Chris Brunsdon et Martin Charlton. 2003. *Geographically weighted regression: the analysis of spatially varying relationships*. John Wiley & Sons.
- Fotheringham, A Stewart, Wenbai Yang et Wei Kang. 2017. « Multiscale geographically weighted regression (MGWR). » *Annals of the American Association of Geographers* 107 (6). Taylor & Francis: 1247-1265. <https://doi.org/10.1080/24694452.2017.1352480>.
- Geary, Robert C. 1954. « The contiguity ratio and statistical mapping. » *The incorporated statistician* 5 (3): 115-146. <https://doi.org/10.2307/2986645>.
- Geniaux, Ghislain et Davide Martinetti. 2018. « A new method for dealing simultaneously with spatial autocorrelation and spatial heterogeneity in regression models. » *Regional Science and Urban Economics* 72: 74-85. <https://doi.org/10.1016/j.regsci.2018.05.001>.

- [1016/j.regsciurbeco.2017.04.001](#).
- Griffith, Daniel A. 1988. *Advanced Spatial Statistics: Special Topics in the Exploration of Quantitative Spatial Data Series*. Kluwer Academic Publishers.
- . 2003. *Spatial autocorrelation and spatial filtering: gaining understanding through theory and scientific visualization*. Springer.
- Griffith, Daniel A., Yongwan Chun et Bin Li. 2019. *Spatial regression analysis using eigenvector spatial filtering*. Academic Press.
- Isabella Gollini, Binbin Lu, Martin Charlton, Christopher Brunsdon et Paul Harris. 2015. « GWmodel: An R Package for Exploring Spatial Heterogeneity Using Geographically Weighted Models. » *Journal of Statistical Software* 63 (17): 1-50. [10.18637/jss.v063.i17](#).
- Lu, Binbin, Martin Charlton et A Stewart Fotheringham. 2011. « Geographically weighted regression using a non-Euclidean distance metric with a study on London house price data. » *Procedia Environmental Sciences* 7: 92-97. [https://doi.org/10.1016/j.proenv.2011.07.017](#).
- Mantel, Nathan. 1967. « The detection of disease clustering and a generalized regression approach. » *Cancer research* 27 (2_Part_1): 209-220.
- Mennis, Jeremy L et Lisa Jordan. 2005. « The distribution of environmental equity: Exploring spatial nonstationarity in multivariate models of air toxic releases. » *Annals of the Association of American Geographers* 95 (2): 249-268. [https://doi.org/10.1111/j.1467-8306.2005.00459.x](#).
- Moran, Patrick. 1948. « The interpretation of statistical maps. » *Journal of the Royal Statistical Society. Series B (Methodological)* 10 (2): 243-251. [https://www.jstor.org/stable/2983777](#).
- . 1950. « A test for the serial independence of residuals. » *Biometrika* 37 (1/2): 178-181. [https://doi.org/10.2307/2332162](#).
- Paelinck, Jean. 1978. « Spatial econometrics. » *Economics Letters* 1 (1): 59-63. [https://doi.org/10.1016/0165-1765\(78\)90097-6](#).
- Stewart Fotheringham, A, Martin Charlton et Chris Brunsdon. 1996. « The geography of parameter space: an investigation of spatial non-stationarity. » *International journal of geographical information systems* 10 (5). Taylor & Francis: 605-627.
- Tobler, Waldo R. 1970. « A computer movie simulating urban growth in the Detroit region. » *Economic geography* 46 (sup1): 234-240. [https://doi.org/10.2307/143141](#).
- Wheeler, David et Michael Tiefelsdorf. 2005. « Multicollinearity and correlation among local regression coefficients in geographically weighted regression. » *Journal of Geographical Systems* 7 (2): 161-187. [https://doi.org/10.1007/s10109-005-0155-6](#).
- Wood, Simon N. 2011. « Fast stable restricted maximum likelihood and marginal likelihood estimation of semiparametric generalized linear models. » *Journal of the Royal Statistical Society Series B: Statistical Methodology* 73 (1): 3-36. [https://doi.org/10.1111/j.1467-9868.2010.00749.x](#).